ECOLE NORMALE SUPÉRIURE DE L'ENSEIGNEMENT TECHNIQUE DE MOHAMMEDIA
UNIVERSITÉ HASSAN II DE CASABLANCA

École Normale Supérieure de l'Enseignement Technique de Mohammedia
Université Hassan II de Casablanca

Filière SDIA, 2ème année
Module : Sécurité des systèmes distribués

RAPPORT DE PROJET

Authentification et autorisation avec Spring Security, JWT et React

Application SecIncidents : déclaration et suivi des incidents informatiques et de cybersécurité

Spring Boot · Spring Security · JWT · React · Axios · JPA · H2 / MySQL

Réalisé par
Hicham OUAOUCHE

Encadré par
Pr. Abdelmajid BOUSSELHAM

Année universitaire 2026 – 2027

Table des matières

1. Introduction
1.1 Contexte
1.2 Objectifs
1.3 Technologies utilisées
2. Analyse des besoins
2.1 Acteurs et rôles
2.2 Matrice des droits
2.3 Règles métier
3. Architecture générale
3.1 Architecture en couches du backend
3.2 Organisation des projets
4. Modèle de données
4.1 Entités
4.2 Travail de recherche : relations JPA
@ManyToOne et @OneToMany
LAZY et EAGER
Choix effectués et justification
5. Réalisation du backend, étape par étape
Étape 1 : création du projet et dépendances
Étape 2 : configuration (application.properties)
Étape 3 : entités, enums et repositories
Étape 4 : DTO et validation des données
Travail de recherche : @NotBlank, @Email, @Size, @Valid
Étape 5 : le service JWT
Travail de recherche : les claims JWT
Étape 6 : le filtre JwtAuthenticationFilter
Étape 7 : configuration de Spring Security
Étape 8 : sécurisation selon les rôles
Travail de recherche : hasRole, hasAuthority, @PreAuthorize, @EnableMethodSecurity
Étape 9 : contrôle du propriétaire d'un incident
Travail de recherche : Authentication, SecurityContextHolder, Principal
Étape 10 : règles métier et machine à états des statuts
Étape 11 : historique et notes de traitement
Étape 12 : gestion des exceptions
Travail de recherche : @ControllerAdvice et @ExceptionHandler
Travail de recherche : 400, 401, 403, 404
Étape 13 : refresh token
Travail de recherche : access token et refresh token

Étape 14 : initialisation des comptes
6. Réalisation du frontend React, étape par étape
Étape 1 : création du projet
Étape 2 : gestion du JWT côté React
Étape 3 : gestion de l'expiration (401)
Travail de recherche : axios.interceptors.response.use(...)
Étape 4 : protection des routes React
Étape 5 : pages et dashboard par rôle
7. Lancement de l'application
7.1 Backend
7.2 Frontend
7.3 Console H2 et stockage des mots de passe
8. Tests
8.1 Tests d'intégration (JUnit / MockMvc)
8.2 Outil de test en ligne de commande
8.3 Inscription, connexion et JWT
8.4 Les 9 tests de sécurité obligatoires
Test 1 : accès sans JWT
Test 2 : JWT invalide ou falsifié
Test 3 : JWT expiré
Test 4 : EMPLOYEE sur une route ADMIN
Test 5 : EMPLOYEE sur l'incident d'un autre utilisateur
Test 6 : TECHNICIAN sur un incident qui ne lui est pas affecté
Test 7 : utilisateur désactivé
Test 8 : inscription avec un email déjà existant
Test 9 : données invalides
8.5 Tests complémentaires
8.6 Collection Postman exécutée avec Newman
8.7 Tests réalisés dans l'application Postman
9. Scénario de test et captures de l'application
9.1 Pages publiques
9.2 Employé (EMPLOYEE)
9.3 Administrateur (ADMIN)
9.4 Technicien (TECHNICIAN)
9.5 Vérification finale et expiration de session
10. Bilan de sécurité
10.1 Mesures mises en place
10.2 Limites et améliorations possibles
11. Conclusion

Annexe : récapitulatif des endpoints.....
Livrables.....
Flux d'authentification.....

1. Introduction

1.1 Contexte

Les entreprises font face chaque jour à des incidents informatiques et de cybersécurité : postes infectés par un malware, tentatives de phishing, comptes bloqués, pannes réseau, fuites potentielles de données, problèmes logiciels ou matériels, comportements suspects sur un poste de travail. Pour les traiter efficacement, il faut un outil centralisé où les employés **déclarent** les incidents, où les techniciens les **traitent** et où l'administrateur **supervise** l'ensemble.

Ce projet consiste à développer une application web complète, **SecIncidents**, composée d'un backend **Spring Boot** sécurisé par **Spring Security** et des **JSON Web Tokens (JWT)**, et d'un frontend **React** qui consomme l'API REST avec **Axios**. Il prolonge l'atelier JWT réalisé en cours en y ajoutant les mécanismes nécessaires à une application réelle : gestion des rôles, contrôle fin des droits d'accès aux données, validation, gestion des erreurs, historique, refresh token, etc.

1.2 Objectifs

- mettre en place l'**authentification sans état (stateless)** avec JWT ;
- gérer **trois rôles** (EMPLOYEE, TECHNICIAN, ADMIN) et leurs droits respectifs ;
- protéger les **routes backend** (règles d'URL + sécurité au niveau des méthodes) ;
- protéger les **routes React** (ProtectedRoute, RoleRoute) ;
- contrôler l'**accès aux données** : un employé ne voit que ses incidents, un technicien que ceux qui lui sont affectés ;
- valider les données reçues et renvoyer des **erreurs propres et homogènes** ;
- proposer un **tableau de bord** adapté au rôle et un **suivi des incidents** (historique, notes de traitement) ;
- tester l'ensemble : tests d'intégration, tests Postman/Newman, tests curl et scénario fonctionnel complet.

1.3 Technologies utilisées

Tableau 1 : *Technologies utilisées*

Couche	Technologie	Rôle dans le projet
Backend	Java 21, Spring Boot 3.2.5	Socle applicatif, serveur Tomcat embarqué
Sécurité	Spring Security 6	Chaîne de filtres, authentification, autorisation, CORS
Jetons	JJWT 0.12.6 (io.jsonwebtoken)	Création et vérification des JWT signés HS256
Persistance	Spring Data JPA / Hibernate 6.4	Mapping objet-relationnel, repositories
Base de données	H2 (mémoire), MySQL (profil)	Stockage des utilisateurs, incidents, historique…
Validation	Jakarta Bean Validation	@NotBlank, @Email, @Size, @NotNull, @Valid
Frontend	React 18 + Vite 6	Interface utilisateur (SPA)
Routage	React Router 6	Routes publiques, protégées et par rôle
HTTP	Axios 1.7	Appels REST + intercepteurs JWT
Tests	JUnit 5, MockMvc, Postman/Newman, curl	Tests d'intégration et tests de sécurité

2. Analyse des besoins

2.1 Acteurs et rôles

L'application gère trois types d'utilisateurs. Le rôle est stocké dans l'entité User et recopié dans le JWT.

Tableau 2 : Les trois rôles de l'application

Rôle	Description	Création du compte
EMPLOYEE	Déclare des incidents et consulte uniquement les siens.	Inscription publique /auth/register (rôle imposé)
TECHNICIAN	Traite les incidents qui lui sont affectés : statut et notes.	Créé par l'initialisation des données
ADMIN	Gère les utilisateurs, affecte les incidents, consulte les statistiques.	Créé par l'initialisation (Hicham Ouaouche)

2.2 Matrice des droits

Tableau 3 : Matrice des droits par rôle

Fonctionnalité	EMPLOYEE	TECHNICIAN	ADMIN
Consulter / modifier son profil	Oui	Oui	Oui
Changer son mot de passe	Oui	Oui	Oui
Créer un incident	Oui	Non	Non
Consulter ses propres incidents	Oui	-	-
Consulter le détail d'un incident	Si propriétaire	Si affecté	Tous
Consulter les incidents affectés	Non	Oui	-
Modifier le statut	Non	Si affecté (IN_PROGRESS, RESOLVED)	Oui (y compris CLOSED)
Ajouter une note de traitement	Non	Si affecté	Oui
Affecter un incident à un technicien	Non	Non	Oui
Consulter tous les incidents	Non	Non	Oui
Lister / activer / désactiver les utilisateurs	Non	Non	Oui
Statistiques globales	Non	Non	Oui

2.3 Règles métier

1. Un nouvel utilisateur reçoit automatiquement le rôle **EMPLOYEE**.
2. Un utilisateur ne peut jamais choisir le rôle ADMIN lors de son inscription.
3. Les mots de passe sont encodés avec **BCrypt**.
4. Un employé ne peut consulter que ses propres incidents.
5. Un technicien ne peut modifier que les incidents qui lui sont affectés.
6. Seul un administrateur peut affecter un technicien.
7. Un utilisateur désactivé ne peut plus se connecter.
8. Lorsqu'un incident est **CLOSED**, son statut ne peut plus être modifié.

Chaque règle est appliquée **côté serveur**. Le frontend se contente d'adapter l'affichage : masquer un bouton améliore l'ergonomie mais ne constitue jamais une protection.

3. Architecture générale

L'application suit une architecture client-serveur en deux projets indépendants : un frontend React (SPA) servi par Vite sur le port **5173**, et une API REST Spring Boot sur le port **8085**. Ils communiquent en HTTP/JSON ; chaque requête protégée porte l'en-tête Authorization: Bearer <JWT>.

> **Remarque sur le port**
>
> Le sujet suggère le port 8080, mais il était déjà occupé sur la machine de développement par un conteneur Docker (serveur web). Le backend écoute donc sur **8085** (server.port=8085). Le frontend lit l'URL de l'API dans frontend/.env (VITE_API_URL), ce qui rend ce choix facile à modifier.

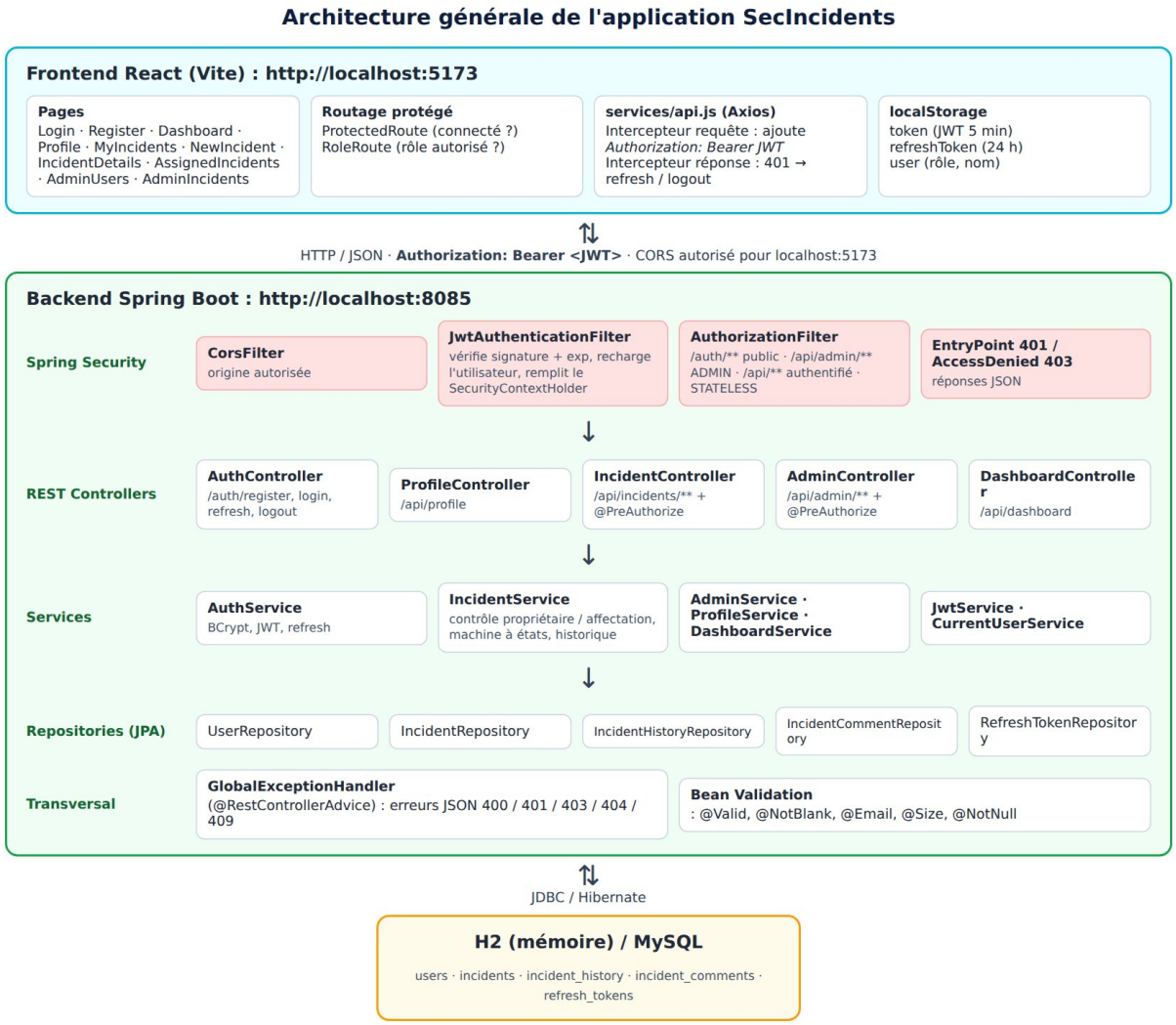


Figure 1 : *Diagramme de l'architecture générale*

3.1 Architecture en couches du backend

- **Spring Security** : chaîne de filtres (CORS, JwtAuthenticationFilter, autorisation) exécutée avant tout contrôleur ;
- **Controllers** : exposent les endpoints REST, valident les entrées (@Valid) et vérifient le rôle (@PreAuthorize) ;

- **Services** : contiennent la logique métier (contrôle du propriétaire, machine à états, historique) ;
- **Repositories** : interfaces Spring Data JPA pour l'accès aux données ;
- **DTO** : objets d'entrée et de sortie (records Java) ; les entités ne sont jamais exposées directement (le mot de passe n'est jamais renvoyé) ;
- **GlobalExceptionHandler** : transforme toutes les exceptions en réponses JSON homogènes.

3.2 Organisation des projets

Arborescence du projet

```
Projet_devoir/
├── README.md                  Guide de lancement, comptes, rôles, endpoints
├── backend/                   Projet Spring Boot (Maven)
│   ├── pom.xml
│   └── src/
│       ├── main/java/com/sdia/incidents/
│       │   ├── IncidentsApplication.java
│       │   ├── config/        SecurityConfig, DataInitializer
│       │   ├── controller/    AuthController, ProfileController, IncidentController,
│       │   │                  AdminController, DashboardController
│       │   ├── dto/           RegisterRequest, LoginRequest, AuthResponse, IncidentRequest, ...
│       │   ├── entity/        User, Incident, IncidentHistory, IncidentComment, RefreshToken
│       │   ├── enums/         Role, Category, Priority, Status
│       │   ├── exception/     GlobalExceptionHandler + exceptions métier
│       │   ├── repository/    UserRepository, IncidentRepository, ...
│       │   ├── security/      JwtService, JwtAuthenticationFilter, JwtAuthenticationEntryPoint,
│       │   │                  CustomAccessDeniedHandler, CustomUserDetailsService, CurrentUserService
│       │   └── service/       AuthService, IncidentService, AdminService, ProfileService, DashboardService
│       ├── main/resources/    application.properties, application-mysql.properties
│       └── test/java/com/sdia/incidents/   SecurityIntegrationTest.java
├── frontend/                  Projet React (Vite)
│   ├── package.json, vite.config.js, index.html, .env
│   ├── public/shield.svg
│   └── src/
│       ├── components/    Navbar, ProtectedRoute, RoleRoute, Badge, IncidentTable
│       ├── context/       AuthContext
│       ├── pages/         Login, Register, Dashboard, Profile, MyIncidents, NewIncident,
│       │                  IncidentDetails, AssignedIncidents, AdminUsers, AdminIncidents,
│       │                  Forbidden, NotFound
│       ├── services/      api.js, authService.js, incidentService.js, userService.js
│       ├── App.jsx, main.jsx, index.css
└── rapport/
    ├── Rapport_Projet_JWT_Spring_Security_React.docx
    ├── logo_enset.png
    ├── diagrammes/        architecture.png, entites.png, flux_jwt.png
    └── captures/          captures de l'application, du terminal et de Postman
        └── postman/       captures réalisées dans l'application Postman
```

4. Modèle de données

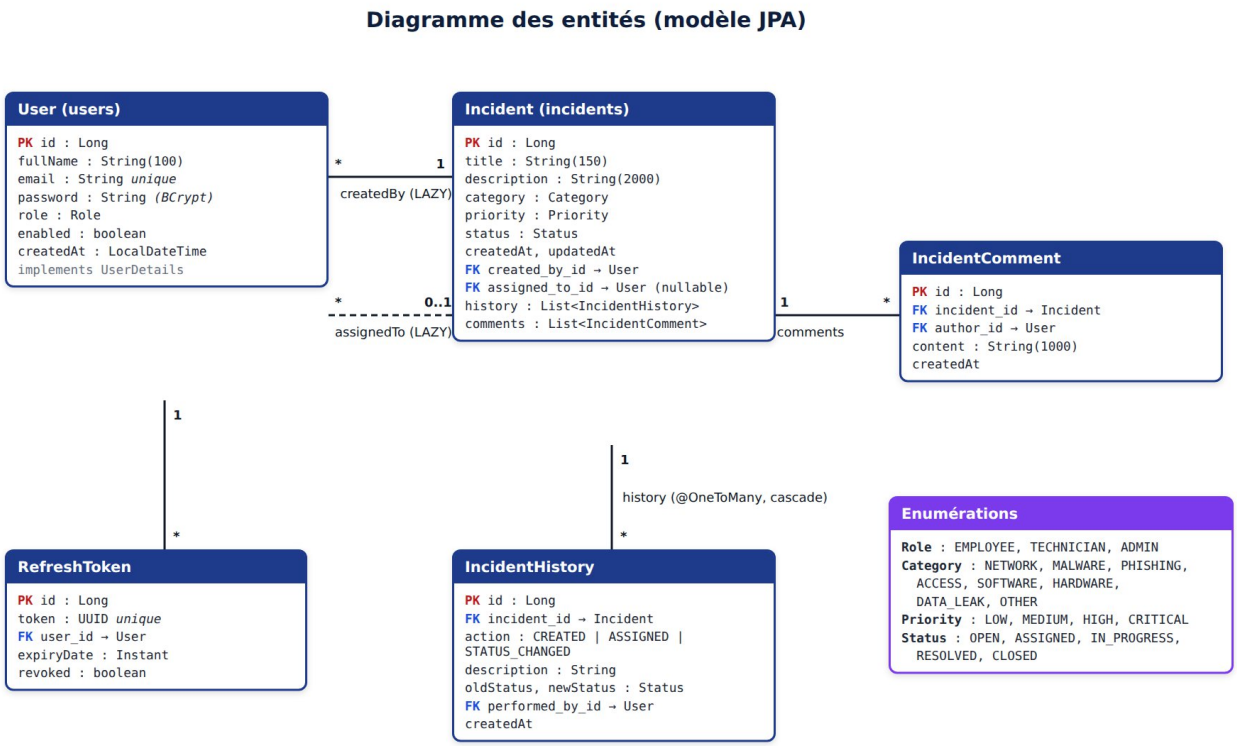


Figure 2 : Diagramme des entités

4.1 Entités

Tableau 4 : Entités JPA

Entité	Attributs principaux	Rôle
User	id, fullName, email (unique), password (BCrypt), role, enabled, createdAt	Compte utilisateur ; implémente UserDetails
Incident	id, title, description, category, priority, status, createdAt, updatedAt, createdBy, assignedTo	Incident déclaré
IncidentHistory	id, incident, action, description, oldStatus, newStatus, performedBy, createdAt	Trace des changements importants
IncidentComment	id, incident, author, content, createdAt	Notes de traitement du technicien
RefreshToken	id, token (UUID), user, expiryDate, revoked	Jeton de renouvellement révocable

Les énumérations sont stockées en base sous forme de chaîne (@Enumerated(EnumType.STRING)), ce qui rend la base lisible et résiste à un réordonnancement des constantes :

- **Role** : EMPLOYEE, TECHNICIAN, ADMIN
- **Category** : NETWORK, MALWARE, PHISHING, ACCESS, SOFTWARE, HARDWARE, DATA_LEAK, OTHER
- **Priority** : LOW, MEDIUM, HIGH, CRITICAL
- **Status** : OPEN, ASSIGNED, IN_PROGRESS, RESOLVED, CLOSED

entity/Incident.java (extrait)

```
@Entity
@Table(name = "incidents")
@Getter @Setter
@NoArgsConstructor @AllArgsConstructor @Builder
public class Incident {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 150)
    private String title;

    @Column(nullable = false, length = 2000)
    private String description;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private Category category;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private Priority priority;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private Status status;

    @Column(nullable = false, updatable = false)
    private LocalDateTime createdAt;

    private LocalDateTime updatedAt;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "created_by_id", nullable = false)
    private User createdBy;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "assigned_to_id")
    private User assignedTo;

    @OneToMany(mappedBy = "incident", cascade = CascadeType.ALL, orphanRemoval = true)
    @OrderBy("createdAt ASC")
    @Builder.Default
    private List<IncidentHistory> history = new ArrayList<>();

    @OneToMany(mappedBy = "incident", cascade = CascadeType.ALL, orphanRemoval = true)
    @OrderBy("createdAt ASC")
    @Builder.Default
    private List<IncidentComment> comments = new ArrayList<>();
```

4.2 Travail de recherche : relations JPA

@ManyToOne et @OneToMany

@ManyToOne exprime que **plusieurs** instances de l'entité courante référencent **une** instance d'une autre entité. Côté base, cela se traduit par une **clé étrangère** dans la table courante. C'est le côté **propriétaire** de la relation (celui qui contient @JoinColumn).

@OneToMany est la vue inverse : **une** entité possède une **collection** d'entités. Avec mappedBy, elle ne crée pas de colonne : elle indique simplement quel attribut de l'autre côté porte la clé étrangère.

LAZY et EAGER

Tableau 5 : Comparaison LAZY / EAGER

Stratégie	Fonctionnement	Défaut JPA	Avantages / risques
EAGER	L'association est chargée immédiatement avec l'entité (jointure ou requête supplémentaire).	@ManyToOne, @OneToOne	Simple, mais charge des données inutiles et provoque facilement le problème N+1.
LAZY	Un proxy est placé ; la donnée n'est chargée qu'au premier accès (dans une transaction ouverte).	@OneToMany, @ManyToMany	Performant ; nécessite une transaction active sinon LazyInitializationException.

Choix effectués et justification

- Incident.createdBy : **@ManyToOne(fetch = LAZY, optional = false)**. Un utilisateur crée plusieurs incidents. On force LAZY (au lieu du défaut EAGER) pour ne pas charger systématiquement l'utilisateur.
- Incident.assignedTo : **@ManyToOne(fetch = LAZY)**, nullable. Un incident est « éventuellement » affecté à un technicien (0..1).
- Incident.history et Incident.comments : **@OneToMany(mappedBy = "incident", cascade = ALL, orphanRemoval = true)**, LAZY par défaut. L'historique et les notes appartiennent à l'incident (cycle de vie lié) et ne sont chargés que dans la vue « détail ». @OrderBy("createdAt ASC") garantit l'ordre chronologique.
- Pas de @OneToMany côté User : une liste « tous les incidents d'un utilisateur » dans l'entité serait potentiellement énorme et inutile ; on passe par des requêtes ciblées du repository (findByCreatedByOrderByCreatedAtDesc).
- Pour éviter le problème N+1 dans les listes, les méthodes du repository utilisent @EntityGraph(attributePaths = {"createdBy", "assignedTo"}) : les associations restent LAZY par défaut mais sont chargées en une seule requête quand c'est nécessaire.
- spring.jpa.open-in-view=false : les entités ne sont pas manipulées dans la couche web ; la conversion en DTO est faite dans les services, à l'intérieur des transactions (@Transactional).

repository/IncidentRepository.java (extrait)

```
// @EntityGraph : charge createdBy/assignedTo en une seule requete (evite le probleme N+1
// tout en gardant LAZY comme strategie par defaut sur l'entite)
@EntityGraph(attributePaths = {"createdBy", "assignedTo"})
List<Incident> findByCreatedByOrderByCreatedAtDesc(User createdBy);

@EntityGraph(attributePaths = {"createdBy", "assignedTo"})
List<Incident> findByAssignedToOrderByCreatedAtDesc(User assignedTo);

@EntityGraph(attributePaths = {"createdBy", "assignedTo"})
List<Incident> findAllByOrderByCreatedAtDesc();
```

5. Réalisation du backend, étape par étape

Étape 1 : création du projet et dépendances

Le projet est un projet Maven basé sur spring-boot-starter-parent 3.2.5 et Java 21. Les dépendances principales sont :

backend/pom.xml (extrait)

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>
  <!-- JWT -->
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>${jjwt.version}</version>
  </dependency>
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>${jjwt.version}</version>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>${jjwt.version}</version>
    <scope>runtime</scope>
  </dependency>
  <!-- Bases de donnees -->
  <dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
```

Étape 2 : configuration (application.properties)

src/main/resources/application.properties

```
# ============================================================
#  Serveur
# ============================================================
server.port=8085
spring.application.name=incidents-backend

# ============================================================
#  Base de donnees H2 (profil par defaut)
```

```
#  -> pour MySQL : lancer avec --spring.profiles.active=mysql
# ============================================================
spring.datasource.url=jdbc:h2:mem:incidentsdb;DB_CLOSE_DELAY=-1
spring.datasource.username=sa
spring.datasource.password=
spring.datasource.driver-class-name=org.h2.Driver
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.open-in-view=false
spring.jpa.show-sql=false
spring.jpa.properties.hibernate.format_sql=true

# ============================================================
#  JWT
#  secret : cle HMAC-SHA256 encodee en Base64 (>= 256 bits)
#  access token  : 5 minutes  (300 000 ms)
#  refresh token : 24 heures  (86 400 000 ms)
# ============================================================
app.jwt.secret=c2RpYS1zZWN1cml0ZS1hcHBsaWNhdGlvbnMtand0LXNlY3JldC1rZXktMjAyNi1pbmNpZGVudHMtYXBw
app.jwt.access-expiration-ms=300000
app.jwt.refresh-expiration-ms=86400000

# ============================================================
#  CORS : application React (Vite)
# ============================================================
app.cors.allowed-origins=http://localhost:5173

# Messages d'erreur : ne pas exposer les stacktraces
server.error.include-stacktrace=never
server.error.include-message=never
```

La base H2 en mémoire est recréée à chaque démarrage (create-drop), ce qui garantit des tests reproductibles. Un profil mysql (application-mysql.properties) permet de basculer sur MySQL sans modifier le code.

Étape 3 : entités, enums et repositories

L'entité User implémente directement l'interface UserDetails de Spring Security : l'email sert d'identifiant (getUsername()), l'autorité est construite à partir du rôle avec le préfixe ROLE_, et isEnabled() renvoie le champ enabled. Spring Security refuse ainsi automatiquement la connexion d'un compte désactivé.

entity/User.java (extrait)

```
// ---------- UserDetails ----------
@Override
public Collection<? extends GrantedAuthority> getAuthorities() {
    return List.of(new SimpleGrantedAuthority("ROLE_" + role.name()));
}

@Override
public String getUsername() { return email; }

@Override
public boolean isAccountNonExpired() { return true; }

@Override
public boolean isAccountNonLocked() { return true; }
```

```
@Override
public boolean isCredentialsNonExpired() { return true; }

@Override
public boolean isEnabled() { return enabled; }
```

Étape 4 : DTO et validation des données

Travail de recherche : @NotBlank, @Email, @Size, @Valid

Tableau 6 : Annotations de validation

Annotation	Signification
@NotBlank	La chaîne ne doit être ni null, ni vide, ni composée uniquement d'espaces.
@NotNull	La valeur ne doit pas être null (utilisée pour les enums category, priority, status).
@Email	La chaîne doit respecter le format d'une adresse email.
@Size(min, max)	Longueur minimale et maximale d'une chaîne ou d'une collection.
@Valid	Placée sur le paramètre @RequestBody du contrôleur, elle déclenche la validation de l'objet. En cas d'échec, Spring lève MethodArgumentNotValidException (traduite en 400).

dto/RegisterRequest.java

```
/**
 * Donnees d'inscription.
 * Aucun champ "role" : l'utilisateur ne peut pas choisir son role.
 * Si le client envoie "role":"ADMIN" dans le JSON, le champ est simplement ignore.
 */
public record RegisterRequest(
        @NotBlank(message = "Le nom complet est obligatoire")
        @Size(min = 3, max = 100, message = "Le nom complet doit contenir entre 3 et 100 caracteres")
        String fullName,

        @NotBlank(message = "L'email est obligatoire")
        @Email(message = "L'email n'est pas valide")
        String email,

        @NotBlank(message = "Le mot de passe est obligatoire")
        @Size(min = 8, max = 72, message = "Le mot de passe doit contenir entre 8 et 72 caracteres")
        String password
) {}
```

Le DTO d'inscription ne contient **aucun champ role** : si un client malveillant envoie "role":"ADMIN" dans le JSON, Jackson ignore simplement ce champ inconnu. Le rôle est fixé côté serveur (règles 1 et 2).

dto/IncidentRequest.java

```
public record IncidentRequest(
        @NotBlank(message = "Le titre est obligatoire")
        @Size(min = 5, max = 150, message = "Le titre doit contenir entre 5 et 150 caracteres")
        String title,

        @NotBlank(message = "La description est obligatoire")
        @Size(min = 10, max = 2000, message = "La description doit contenir entre 10 et 2000 caracteres")
        String description,

        @NotNull(message = "La categorie est obligatoire")
        Category category,
```

```
    @NotNull(message = "La priorite est obligatoire")
    Priority priority
) {}
```

Étape 5 : le service JWT

Travail de recherche : les claims JWT

Un JWT est composé de trois parties encodées en Base64URL et séparées par des points : **header.payload.signature**. Le header indique l'algorithme (HS256), le payload contient les **claims**, et la signature (HMAC-SHA256 du header et du payload avec la clé secrète du serveur) garantit l'intégrité : toute modification du payload invalide la signature.

Un **claim** est une information (paire clé/valeur) affirmée dans le token. On distingue :

- **claims enregistrés** (standardisés par la RFC 7519) : sub (sujet, ici l'email), iat (issued at, date de création), exp (date d'expiration), iss, aud… ;
- **claims privés** : propres à l'application, ici role et fullName.

__Tableau 7 :__ Manipulation des claims avec JJWT

Question	Réponse dans le projet
Ajouter un rôle dans le token	Jwts.builder().claim("role", user.getRole().name())
Récupérer un claim	Jwts.parser().verifyWith(key).build().parseSignedClaims(token).getPayload() renvoie un objet Claims ; puis claims.getSubject() ou claims.get("role", String.class)
Vérifier l'expiration	parseSignedClaims() compare automatiquement exp à l'heure courante et lève ExpiredJwtException si le token est expiré. Le filtre l'intercepte et renvoie 401 « Token expire ».

security/JwtService.java (extrait)

```
public String generateToken(User user) {
   Date now = new Date();
   Date expiry = new Date(now.getTime() + accessExpirationMs);
   return Jwts.builder()
         .subject(user.getEmail())
         .claim(ROLE_CLAIM, user.getRole().name())
         .claim("fullName", user.getFullName())
         .issuedAt(now)
         .expiration(expiry)
         .signWith(signingKey, Jwts.SIG.HS256)
         .compact();
}

/**
 * Verifie la signature ET l'expiration puis retourne les claims.
 * Leve ExpiredJwtException si exp < maintenant,
 * SignatureException / MalformedJwtException si le token est falsifie ou mal forme.
 */
public Claims parseClaims(String token) {
   return Jwts.parser()
         .verifyWith(signingKey)
         .build()
         .parseSignedClaims(token)
         .getPayload();
}
```

Le token contient bien les quatre informations demandées : l'**email** (sub), la **date de création** (iat), la **date d'expiration** (exp, 5 minutes après iat) et le **rôle** (role). La clé secrète (256 bits, encodée en Base64) est lue depuis la configuration.

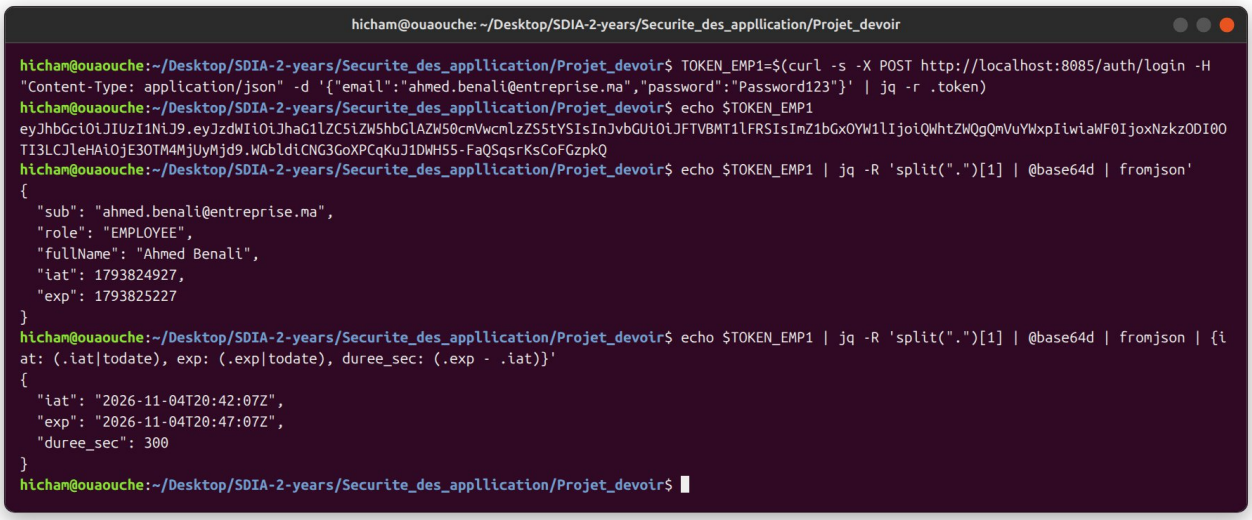


Figure 3 : *Décodage réel d'un JWT émis par l'API : claims sub, role, iat, exp (5 min)*

Étape 6 : le filtre JwtAuthenticationFilter

Ce filtre hérite de OncePerRequestFilter et est placé dans la chaîne Spring Security **avant** UsernamePasswordAuthenticationFilter. Pour chaque requête, il :

1. lit l'en-tête Authorization et vérifie qu'il commence par Bearer ;
2. vérifie la signature et l'expiration du token (parseClaims) ;
3. **recharge l'utilisateur en base** : un compte supprimé ou désactivé après l'émission du token est donc refusé immédiatement ;
4. crée un UsernamePasswordAuthenticationToken avec les autorités (ROLE_x) et le place dans le SecurityContextHolder ;
5. en cas d'erreur (token expiré, invalide, compte désactivé), mémorise la raison dans un attribut de requête : le point d'entrée 401 renverra ainsi un message précis.

security/JwtAuthenticationFilter.java (extrait)

```
String header = request.getHeader("Authorization");
if (header == null || !header.startsWith("Bearer ")) {
    filterChain.doFilter(request, response);
    return;
}

String token = header.substring(7);
try {
    Claims claims = jwtService.parseClaims(token);
    String email = claims.getSubject();

    if (email != null && SecurityContextHolder.getContext().getAuthentication() == null) {
        Optional<User> userOpt = userRepository.findByEmail(email);
        if (userOpt.isEmpty()) {
            request.setAttribute(JWT_ERROR_ATTRIBUTE, "Utilisateur du token introuvable");
        } else if (!userOpt.get().isEnabled()) {
            request.setAttribute(JWT_ERROR_ATTRIBUTE, "Compte desactive");
        } else {
            User user = userOpt.get();
```

```
            var authentication = new UsernamePasswordAuthenticationToken(
                    user, null, user.getAuthorities());
            authentication.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
            SecurityContextHolder.getContext().setAuthentication(authentication);
        }
    }
} catch (ExpiredJwtException e) {
    request.setAttribute(JWT_ERROR_ATTRIBUTE, "Token expire");
} catch (JwtException | IllegalArgumentException e) {
    request.setAttribute(JWT_ERROR_ATTRIBUTE, "Token invalide");
}

filterChain.doFilter(request, response);
```

Étape 7 : configuration de Spring Security

config/SecurityConfig.java (extrait)

```
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        // API REST stateless + JWT dans l'en-tete : pas de cookie de session -> CSRF inutile
        .csrf(AbstractHttpConfigurer::disable)
        .cors(cors -> cors.configurationSource(corsConfigurationSource()))
        // Aucune session HTTP : chaque requete doit porter son JWT
        .sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        // Necessaire pour afficher la console H2 (frames)
        .headers(h -> h.frameOptions(f -> f.sameOrigin()))
        .authorizeHttpRequests(auth -> auth
                // ---- Routes publiques ----
                .requestMatchers("/auth/**").permitAll()
                .requestMatchers("/h2-console/**").permitAll()
                .requestMatchers("/error").permitAll()
                // ---- Routes reservees a l'ADMIN (1er niveau : URL) ----
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                // ---- Toutes les autres routes /api/** : authentifie ----
                // (le controle fin par role est fait par @PreAuthorize)
                .requestMatchers("/api/**").authenticated()
                .anyRequest().denyAll()
        )
        .exceptionHandling(ex -> ex
                .authenticationEntryPoint(authenticationEntryPoint)   // 401
                .accessDeniedHandler(accessDeniedHandler))            // 403
        .authenticationProvider(authenticationProvider())
        .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);

    return http.build();
```

- **Routes publiques** : /auth/** et /h2-console/** (permitAll).
- **Routes protégées** : /api/** (authenticated) ; /api/admin/** exige en plus le rôle ADMIN ; toute autre URL est refusée (denyAll).
- **STATELESS** : SessionCreationPolicy.STATELESS. Aucune session HTTP n'est créée ni utilisée, chaque requête doit porter son JWT.
- **CSRF désactivé** : l'attaque CSRF exploite l'envoi automatique des cookies par le navigateur ; ici l'authentification passe par un en-tête Authorization que le navigateur n'ajoute jamais seul, la protection CSRF est donc inutile.
- **CORS** : seule l'origine http://localhost:5173 est autorisée, avec les méthodes et en-têtes nécessaires.
- **Erreurs** : JwtAuthenticationEntryPoint (401) et CustomAccessDeniedHandler (403) renvoient du JSON.

- **BCrypt** : BCryptPasswordEncoder (sel aléatoire intégré, facteur de coût 10).

Étape 8 : sécurisation selon les rôles

Travail de recherche : hasRole, hasAuthority, @PreAuthorize, @EnableMethodSecurity

Tableau 8 : Mécanismes d'autorisation de Spring Security

Mécanisme	Explication	Utilisation dans le projet
hasRole("ADMIN")	Vérifie l'autorité ROLE_ADMIN : le préfixe ROLE_ est ajouté automatiquement.	SecurityConfig : /api/admin/** ; @PreAuthorize
hasAuthority("ROLE_ADMIN")	Vérifie l'autorité exacte, sans ajout de préfixe. Équivalent ici à hasRole("ADMIN"). Utile pour des permissions fines (ex. INCIDENT_WRITE).	Non nécessaire : rôles simples
@EnableMethodSecurity	Active la sécurité au niveau des méthodes (@PreAuthorize, @PostAuthorize...). Remplace @EnableGlobalMethodSecurity depuis Spring Security 6.	Sur SecurityConfig
@PreAuthorize("...")	Évalue une expression SpEL avant l'exécution de la méthode ; si elle est fausse, AccessDeniedException → 403.	Sur chaque endpoint des contrôleurs

La sécurité est appliquée sur **deux niveaux** (défense en profondeur) : les règles d'URL dans SecurityConfig et @PreAuthorize sur chaque méthode. Ainsi, une erreur de configuration à un niveau ne suffit pas à ouvrir un accès.

controller/IncidentController.java (extrait)

```
@PostMapping
@ResponseStatus(HttpStatus.CREATED)
@PreAuthorize("hasRole('EMPLOYEE')")
public IncidentResponse create(@Valid @RequestBody IncidentRequest request) {
    return incidentService.create(request);
}

@GetMapping("/my")
@PreAuthorize("hasRole('EMPLOYEE')")
public List<IncidentResponse> myIncidents() {
    return incidentService.myIncidents();
}

@GetMapping("/assigned")
@PreAuthorize("hasRole('TECHNICIAN')")
public List<IncidentResponse> assigned() {
    return incidentService.assignedIncidents();
}

@GetMapping
@PreAuthorize("hasRole('ADMIN')")
public List<IncidentResponse> all(@RequestParam(required = false) Status status) {
    return incidentService.allIncidents(status);
}

/** Acces selon proprietaire (EMPLOYEE), affectation (TECHNICIAN) ou role (ADMIN). */
@GetMapping("/{id}")
@PreAuthorize("isAuthenticated()")
public IncidentDetailsResponse getById(@PathVariable Long id) {
    return incidentService.getById(id);
}

@PutMapping("/{id}/status")
```

```
@PreAuthorize("hasAnyRole('TECHNICIAN','ADMIN')")
public IncidentResponse updateStatus(@PathVariable Long id, @Valid @RequestBody StatusUpdateRequest request) {
    return incidentService.updateStatus(id, request);
}

@PutMapping("/{id}/assign/{technicianId}")
@PreAuthorize("hasRole('ADMIN')")
public IncidentResponse assign(@PathVariable Long id, @PathVariable Long technicianId) {
```

Étape 9 : contrôle du propriétaire d'un incident

Exigence de sécurité

Un employé peut appeler directement GET /api/incidents/12 avec Postman, même si l'incident ne lui appartient pas. Le backend doit donc vérifier que l'incident appartient réellement à l'utilisateur connecté **avant** de renvoyer les données. Masquer l'incident dans React ne suffit pas.

Travail de recherche : Authentication, SecurityContextHolder, Principal

- **SecurityContextHolder** : stocke le SecurityContext de la requête courante (par défaut dans un ThreadLocal). Le filtre JWT y place l'authentification.
- **Authentication** : représente l'utilisateur authentifié : getPrincipal(), getAuthorities(), isAuthenticated().
- **Principal** : l'identité de l'utilisateur. Dans notre cas, c'est directement l'entité User (qui implémente UserDetails). On peut aussi l'obtenir en paramètre de contrôleur (Principal principal ou @AuthenticationPrincipal User user).

Le service CurrentUserService récupère l'utilisateur connecté depuis le SecurityContextHolder, puis IncidentService.checkReadAccess le compare au propriétaire ou au technicien affecté :

security/CurrentUserService.java

```
public User getCurrentUser() {
    Authentication auth = SecurityContextHolder.getContext().getAuthentication();
    if (auth == null || !(auth.getPrincipal() instanceof User principal)) {
        throw new ResourceNotFoundException("Utilisateur introuvable");
    }
    return userRepository.findById(principal.getId())
            .orElseThrow(() -> new ResourceNotFoundException("Utilisateur introuvable"));
}
```

service/IncidentService.java : contrôle d'accès en lecture

```
private void checkReadAccess(Incident incident, User me) {
    boolean allowed = switch (me.getRole()) {
        case ADMIN -> true;
        case EMPLOYEE -> incident.getCreatedBy().getId().equals(me.getId());
        case TECHNICIAN -> isAssignedTo(incident, me);
    };
    if (!allowed) {
        throw new ForbiddenOperationException(
                "Acces interdit : vous n'avez pas le droit de consulter cet incident");
    }
}
```

Étape 10 : règles métier et machine à états des statuts

Les transitions de statut sont définies dans une table de transitions. CLOSED est un **état final** : aucune transition n'en sort (règle 8). L'affectation par l'administrateur fait passer l'incident de OPEN à **ASSIGNED**. Le technicien peut ensuite le passer à IN_PROGRESS puis RESOLVED, et l'administrateur peut le clôturer.

service/IncidentService.java : transitions

```
/** Transitions de statut autorisees (machine a etats). CLOSED est un etat final. */
private static final Map<Status, Set<Status>> TRANSITIONS = Map.of(
      Status.OPEN,       EnumSet.of(Status.IN_PROGRESS, Status.CLOSED),
      Status.ASSIGNED,   EnumSet.of(Status.IN_PROGRESS, Status.CLOSED),
      Status.IN_PROGRESS, EnumSet.of(Status.RESOLVED, Status.CLOSED),
      Status.RESOLVED,   EnumSet.of(Status.IN_PROGRESS, Status.CLOSED),
      Status.CLOSED,     EnumSet.noneOf(Status.class)
);

/** Statuts qu'un technicien a le droit de positionner. */
private static final Set<Status> TECHNICIAN_TARGETS = EnumSet.of(Status.IN_PROGRESS, Status.RESOLVED);
```

service/IncidentService.java : changement de statut

```
public IncidentResponse updateStatus(Long id, StatusUpdateRequest request) {
   User me = currentUserService.getCurrentUser();
   Incident incident = findIncident(id);
   Status current = incident.getStatus();
   Status target = request.status();

   // Regle n°5 : un technicien ne modifie que les incidents qui lui sont affectes
   if (me.getRole() == Role.TECHNICIAN && !isAssignedTo(incident, me)) {
      throw new ForbiddenOperationException(
            "Acces interdit : cet incident ne vous est pas affecte");
   }
   // Regle n°8 : un incident CLOSED ne peut plus changer de statut
   if (current == Status.CLOSED) {
      throw new BusinessRuleException("Incident cloture : son statut ne peut plus etre modifie");
   }
   if (current == target) {
      throw new BadRequestException("L'incident est deja au statut " + target);
   }
   if (me.getRole() == Role.TECHNICIAN && !TECHNICIAN_TARGETS.contains(target)) {
      throw new ForbiddenOperationException(
            "Un technicien ne peut positionner que IN_PROGRESS ou RESOLVED");
   }
   if (!TRANSITIONS.get(current).contains(target)) {
      throw new BusinessRuleException("Transition interdite : " + current + " -> " + target);
   }

   incident.setStatus(target);
   addHistory(incident, "STATUS_CHANGED", current + " -> " + target, current, target, me);
   return IncidentResponse.from(incidentRepository.save(incident));
```

Tableau 9 : *Machine à états des incidents*

Depuis	Vers (autorisé)	Qui
OPEN	ASSIGNED (via affectation), IN_PROGRESS, CLOSED	ADMIN (affectation) / technicien affecté
ASSIGNED	IN_PROGRESS, CLOSED	Technicien affecté (IN_PROGRESS), ADMIN
IN_PROGRESS	RESOLVED, CLOSED	Technicien affecté (RESOLVED), ADMIN
RESOLVED	IN_PROGRESS (réouverture), CLOSED	Technicien affecté (IN_PROGRESS), ADMIN
CLOSED	aucune (état final)	409 Conflict

Étape 11 : historique et notes de traitement

Chaque événement important ajoute une ligne IncidentHistory : création (« Incident cree par … »), affectation (« Affecte a Youssef Amrani », OPEN → ASSIGNED), et chaque changement de statut (« IN_PROGRESS -> RESOLVED »). L'historique conserve l'ancien et le nouveau statut, l'auteur et la date.

Les notes de traitement (IncidentComment) sont ajoutées par le technicien affecté (ou l'administrateur) via POST /api/incidents/{id}/comments. Elles sont liées à l'incident par une relation @ManyToOne (une note appartient à un incident, un incident a plusieurs notes) et à leur auteur. Elles sont visibles par l'employé propriétaire dans le détail de l'incident.

entity/IncidentHistory.java (extrait)

```
@Entity
@Table(name = "incident_history")
@Getter @Setter
@NoArgsConstructor @AllArgsConstructor @Builder
public class IncidentHistory {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "incident_id", nullable = false)
    private Incident incident;

    /** CREATED, ASSIGNED, STATUS_CHANGED */
    @Column(nullable = false, length = 30)
    private String action;

    @Column(nullable = false, length = 500)
    private String description;

    @Enumerated(EnumType.STRING)
    @Column(length = 20)
    private Status oldStatus;

    @Enumerated(EnumType.STRING)
    @Column(length = 20)
    private Status newStatus;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "performed_by_id", nullable = false)
    private User performedBy;

    @Column(nullable = false)
    private LocalDateTime createdAt;
```

Étape 12 : gestion des exceptions

Travail de recherche : @ControllerAdvice et @ExceptionHandler

@ControllerAdvice (ou @RestControllerAdvice, qui ajoute @ResponseBody) déclare un composant global qui intercepte les exceptions levées par **tous** les contrôleurs. Chaque méthode annotée

@ExceptionHandler(UneException.class) traite un type d'exception et construit la réponse HTTP. On centralise ainsi le format des erreurs et on évite de renvoyer des stack traces au client.

Travail de recherche : 400, 401, 403, 404

Tableau 10 : Codes HTTP d'erreur utilisés

Code	Signification	Exemples dans le projet
400 Bad Request	La requête est mal formée ou les données sont invalides. Le client doit corriger sa requête.	Échec @Valid, enum inconnue, ancien mot de passe incorrect
401 Unauthorized	Non authentifié : identité absente ou invalide. (« unauthenticated » serait plus juste.)	Pas de JWT, JWT invalide ou expiré, mauvais mot de passe
403 Forbidden	Authentifié, mais pas le droit d'accéder à la ressource.	EMPLOYEE sur /api/admin/**, incident d'un autre, compte désactivé
404 Not Found	La ressource demandée n'existe pas.	Incident ou utilisateur introuvable
409 Conflict	La requête entre en conflit avec l'état actuel de la ressource.	Email déjà utilisé, incident CLOSED

exception/GlobalExceptionHandler.java (extrait)

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    // ...
    // ---------- 404 ----------
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> notFound(ResourceNotFoundException ex, HttpServletRequest req) {
        return build(HttpStatus.NOT_FOUND, ex.getMessage(), req);
    }

    /** Levee par @PreAuthorize quand le role ne correspond pas. */
    @ExceptionHandler(AccessDeniedException.class)
    public ResponseEntity<ErrorResponse> accessDenied(AccessDeniedException ex, HttpServletRequest req) {
        return build(HttpStatus.FORBIDDEN, "Acces interdit : vous n'avez pas le role requis", req);
    }

    /** Echec de @Valid : on renvoie la liste des champs invalides. */
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> validation(MethodArgumentNotValidException ex, HttpServletRequest req) {
        Map<String, String> errors = new LinkedHashMap<>();
        ex.getBindingResult().getFieldErrors()
                .forEach(fe -> errors.putIfAbsent(fe.getField(), fe.getDefaultMessage()));
        return ResponseEntity.badRequest().body(new ErrorResponse(400, "Bad Request",
                "Donnees invalides", req.getRequestURI(), LocalDateTime.now(), errors));
    }
}
```

Toutes les erreurs ont le même format JSON, par exemple :

```
{
  "status": 404,
  "error": "Not Found",
  "message": "Incident introuvable",
  "path": "/api/incidents/9999",
  "timestamp": "2026-11-04T21:40:12.512"
}
```

Étape 13 : refresh token

Travail de recherche : access token et refresh token

Tableau 11 : Comparaison access token / refresh token

	Access token (JWT)	Refresh token
Rôle	Prouver l'identité et les droits à chaque requête API	Obtenir un nouvel access token sans ressaisir le mot de passe
Format	JWT signé, auto-porteur (stateless)	Chaîne aléatoire opaque (UUID) stockée en base
Durée de vie	Courte : 5 minutes	Longue : 24 heures
Envoyé à	Toutes les routes /api/** (en-tête Bearer)	Uniquement POST /auth/refresh et /auth/logout
Révocation	Impossible avant exp (sauf liste noire)	Possible : champ revoked en base

- **Renouvellement** : quand l'access token expire, le client appelle POST /auth/refresh avec son refresh token et reçoit un nouvel access token.
- **Rotation** : à chaque renouvellement, l'ancien refresh token est révoqué et un nouveau est émis. Un refresh token volé et déjà utilisé devient inutilisable.
- **Déconnexion** : POST /auth/logout révoque le refresh token. L'access token, de courte durée, expire de lui-même ; le client le supprime du stockage.
- **Désactivation** : quand l'administrateur désactive un compte, tous ses refresh tokens sont révoqués et ses access tokens sont refusés par le filtre.

service/AuthService.java : renouvellement

```
public AuthResponse refresh(RefreshRequest request) {
    RefreshToken stored = refreshTokenRepository.findByToken(request.refreshToken())
            .orElseThrow(() -> new InvalidTokenException("Refresh token invalide"));
    if (stored.isRevoked()) {
        throw new InvalidTokenException("Refresh token revoque");
    }
    if (stored.getExpiryDate().isBefore(Instant.now())) {
        throw new InvalidTokenException("Refresh token expire : veuillez vous reconnecter");
    }
    User user = stored.getUser();
    if (!user.isEnabled()) {
        throw new InvalidTokenException("Compte desactive");
    }
    stored.setRevoked(true);
    return buildAuthResponse(user, createRefreshToken(user));
}
```

Étape 14 : initialisation des comptes

L'inscription donne toujours le rôle EMPLOYEE : il est donc impossible d'obtenir un compte ADMIN ou TECHNICIAN par l'API publique. Un CommandLineRunner crée ces comptes au démarrage, avec des mots de passe hachés en BCrypt :

config/DataInitializer.java

```
public void run(String... args) {
    create("Hicham Ouaouche", "hichamouaouche@gmail.com", "Admin@2026", Role.ADMIN);
    create("Youssef Amrani", "tech1@incidents.ma", "Tech@2026", Role.TECHNICIAN);
    create("Karim Idrissi", "tech2@incidents.ma", "Tech@2026", Role.TECHNICIAN);
}
```

Tableau 12 : *Comptes de test*

Rôle	Nom	Email	Mot de passe
ADMIN	Hicham Ouaouche	hichamouaouche@gmail.com	Admin@2026
TECHNICIAN	Youssef Amrani	tech1@incidents.ma	Tech@2026
TECHNICIAN	Karim Idrissi	tech2@incidents.ma	Tech@2026
EMPLOYEE	Ahmed Benali (inscription)	ahmed.benali@entreprise.ma	Password123
EMPLOYEE	Sara El Amrani (inscription)	sara.elamrani@entreprise.ma	Password123

6. Réalisation du frontend React, étape par étape

Étape 1 : création du projet

Le projet est créé avec **Vite** (template React) puis complété avec react-router-dom et axios. Il démarre sur le port 5173, l'origine autorisée par la configuration CORS du backend.

```
npm create vite@latest frontend -- --template react
cd frontend
npm install react-router-dom axios
npm run dev
```

Étape 2 : gestion du JWT côté React

Après la connexion, authService.saveSession() stocke dans le **localStorage** le JWT (token), le refresh token et les informations d'affichage (rôle, nom). L'intercepteur de **requête** Axios ajoute automatiquement l'en-tête Authorization: Bearer <JWT> à toutes les requêtes protégées :

services/api.js : intercepteur de requête

```
api.interceptors.request.use((config) => {
  const token = getToken()
  if (token && !config.url.startsWith('/auth/')) {
    config.headers.Authorization = `Bearer ${token}`
  }
  return config
})
```

Étape 3 : gestion de l'expiration (401)

Travail de recherche : axios.interceptors.response.use(...)

axios.interceptors.response.use(onSuccess, onError) enregistre deux fonctions exécutées sur **chaque réponse** avant qu'elle ne soit transmise au code appelant. onError reçoit l'erreur (avec error.response.status et error.config, la requête d'origine). Elle peut corriger la situation et **rejouer** la requête (return api(originalConfig)), ou propager l'erreur (return Promise.reject(error)).

Dans le projet, à la réception d'un **401** sur une route protégée :

1. si un refresh token existe, POST /auth/refresh est appelé une seule fois (même si plusieurs requêtes échouent en parallèle) puis la requête d'origine est rejouée avec le nouveau token ;
2. sinon, ou si le refresh échoue : le JWT est **supprimé**, l'utilisateur est **redirigé vers /login** et le message **« Votre session a expiré »** est affiché.

services/api.js : intercepteur de réponse

```
let refreshPromise = null

function expireSession() {
  clearSession()
  sessionStorage.setItem('authMessage', 'Votre session a expiré. Veuillez vous reconnecter.')
  window.location.href = '/login?expired=1'
}

api.interceptors.response.use(
```

```
  (response) => response,
  async (error) => {
    const original = error.config
    const status = error.response?.status
    const isAuthCall = original?.url?.startsWith('/auth/')

    if (status === 401 && !isAuthCall && !original._retry) {
      original._retry = true
      const refreshToken = getRefreshToken()
      if (refreshToken) {
        try {
          // une seule requete de refresh meme si plusieurs appels echouent en parallele
          refreshPromise = refreshPromise || api.post('/auth/refresh', { refreshToken })
          const { data } = await refreshPromise
          saveSession(data)
          original.headers.Authorization = `Bearer ${data.token}`
          return api(original)
        } catch (e) {
          expireSession()
          return Promise.reject(e)
        } finally {
          refreshPromise = null
        }
      }
      expireSession()
    }
    return Promise.reject(error)
  }
)
```

Étape 4 : protection des routes React

ProtectedRoute empêche un utilisateur non authentifié d'accéder aux pages protégées : il est redirigé vers /login. RoleRoute vérifie que le rôle stocké fait partie des rôles autorisés, sinon redirection vers la page 403.

components/ProtectedRoute.jsx

```
export default function ProtectedRoute() {
  const location = useLocation()
  if (!isAuthenticated()) {
    return <Navigate to="/login" replace state={{ from: location }} />
  }
  return <Outlet />
}
```

components/RoleRoute.jsx

```
export default function RoleRoute({ roles }) {
  const user = getUser()
  if (!user) return <Navigate to="/login" replace />
  if (!roles.includes(user.role)) return <Navigate to="/forbidden" replace />
  return <Outlet />
}
```

App.jsx : déclaration des routes

```
{/* ---------- Routes publiques ---------- */}
<Route path="/login" element={<Login />} />
<Route path="/register" element={<Register />} />

{/* ---------- Routes protegees (utilisateur connecte) ---------- */}
<Route element={<ProtectedRoute />}>
```

```
  <Route path="/dashboard" element={<Dashboard />} />
  <Route path="/profile" element={<Profile />} />
  <Route path="/forbidden" element={<Forbidden />} />

  <Route element={<RoleRoute roles={['EMPLOYEE']} />}>
    <Route path="/incidents/my" element={<MyIncidents />} />
    <Route path="/incidents/new" element={<NewIncident />} />
  </Route>

  <Route element={<RoleRoute roles={['TECHNICIAN']} />}>
    <Route path="/incidents/assigned" element={<AssignedIncidents />} />
  </Route>

  {/* le controle proprietaire / affectation est fait par le backend */}
  <Route path="/incidents/:id" element={<IncidentDetails />} />

  <Route element={<RoleRoute roles={['ADMIN']} />}>
    <Route path="/admin/users" element={<AdminUsers />} />
    <Route path="/admin/incidents" element={<AdminIncidents />} />
  </Route>
</Route>

<Route path="/" element={<Navigate to="/dashboard" replace />} />
<Route path="*" element={<NotFound />} />
```

Important

La protection des routes React améliore l'expérience utilisateur, mais un utilisateur peut modifier le localStorage (par exemple écrire "role":"ADMIN") et afficher la page /admin/users. Cela ne lui donne **aucune donnée** : toutes les requêtes vers /api/admin/** sont refusées par le backend avec un 403, car le rôle réel vient du JWT signé et de la base de données.

Étape 5 : pages et dashboard par rôle

Tableau 13 : *Pages du frontend*

Page	Route	Accès	Description
Login	/login	Public	Connexion, messages « session expirée » et « compte créé »
Register	/register	Public	Inscription avec validation et affichage des erreurs serveur
Dashboard	/dashboard	Connecté	Statistiques adaptées au rôle
Profile	/profile	Connecté	Consultation, modification du nom et de l'email, changement de mot de passe
MyIncidents	/incidents/my	EMPLOYEE	Liste de mes incidents
NewIncident	/incidents/new	EMPLOYEE	Formulaire de déclaration
IncidentDetails	/incidents/:id	Connecté (contrôle serveur)	Détail, historique, notes, actions selon le rôle
AssignedIncidents	/incidents/assigned	TECHNICIAN	Incidents qui me sont affectés
AdminIncidents	/admin/incidents	ADMIN	Tous les incidents, filtre par statut, affectation
AdminUsers	/admin/users	ADMIN	Utilisateurs, activation et désactivation

Le tableau de bord appelle GET /api/dashboard, qui renvoie des statistiques calculées **côté serveur** selon le rôle de l'utilisateur connecté :

- **EMPLOYEE** : nombre total de ses incidents, OPEN, IN_PROGRESS, RESOLVED ou CLOSED, et ses 5 derniers incidents ;
- **TECHNICIAN** : incidents affectés, critiques, en cours et résolus ;
- **ADMIN** : nombre total d'utilisateurs, d'employés et de techniciens, nombre total d'incidents, OPEN, IN_PROGRESS, CLOSED, et répartition par statut.

7. Lancement de l'application

7.1 Backend

```
cd backend
mvn spring-boot:run        # ou : mvn clean package && java -jar target/incidents-backend-1.0.0.jar
```



Figure 4 : *Démarrage du backend : Tomcat sur le port 8085, chaîne de filtres de sécurité et création des comptes initiaux*

7.2 Frontend

```
cd frontend
npm install
```

```
npm run dev
```

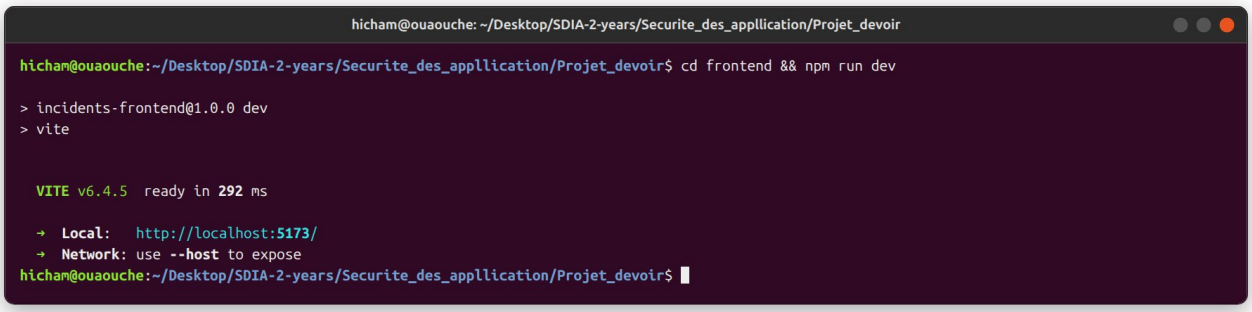


Figure 5 : Démarrage du serveur de développement Vite sur le port 5173

7.3 Console H2 et stockage des mots de passe

La console H2 (route publique /h2-console, URL JDBC jdbc:h2:mem:incidentsdb, utilisateur sa) montre que les mots de passe sont stockés sous forme de **hachés BCrypt** (préfixe $2a$10$: algorithme 2a, coût 10, puis 22 caractères de sel et 31 caractères de hachage). Le même mot de passe donne un hachage différent pour chaque utilisateur grâce au sel aléatoire.

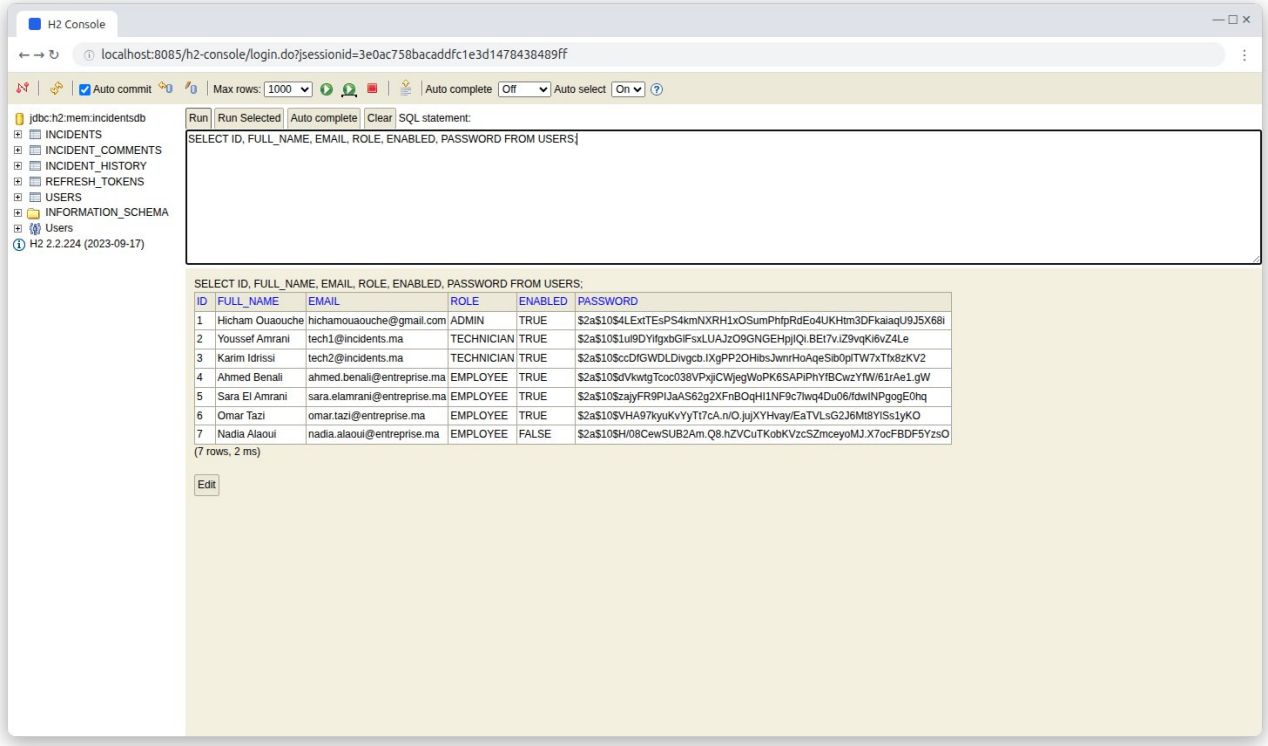


Figure 6 : Console H2 : table USERS, mots de passe hachés avec BCrypt

8. Tests

Les tests ont été menés sur quatre niveaux complémentaires. Toutes les captures de ce chapitre sont des **exécutions réelles** sur le projet, réalisées dans le terminal hicham@ouaouche:~/Desktop/SDIA-2-years/Securite_des_appllication/Projet_devoir$.

- **Tests d'intégration JUnit 5 + MockMvc** (mvn test) : 9 tests automatisés sur le contexte Spring complet ;
- **Tests manuels en ligne de commande** avec curl, via un petit script utilitaire api.sh ;
- **Collection Postman** (importable dans Postman) exécutée avec **Newman** : 53 requêtes et 94 assertions ;
- **Tests fonctionnels de l'interface** React (captures au chapitre 9).

8.1 Tests d'intégration (JUnit / MockMvc)

La classe SecurityIntegrationTest démarre le contexte Spring complet (H2 en mémoire) et envoie de vraies requêtes HTTP simulées avec MockMvc. Elle couvre l'attribution forcée du rôle EMPLOYEE, les 401 sans JWT et avec JWT invalide, les 403 (route ADMIN, incident d'un autre employé, compte désactivé), le 409 sur email existant, le 400 sur données invalides, et un cycle de vie complet (affectation, technicien non affecté refusé, IN_PROGRESS, RESOLVED, note, CLOSED puis modification refusée).

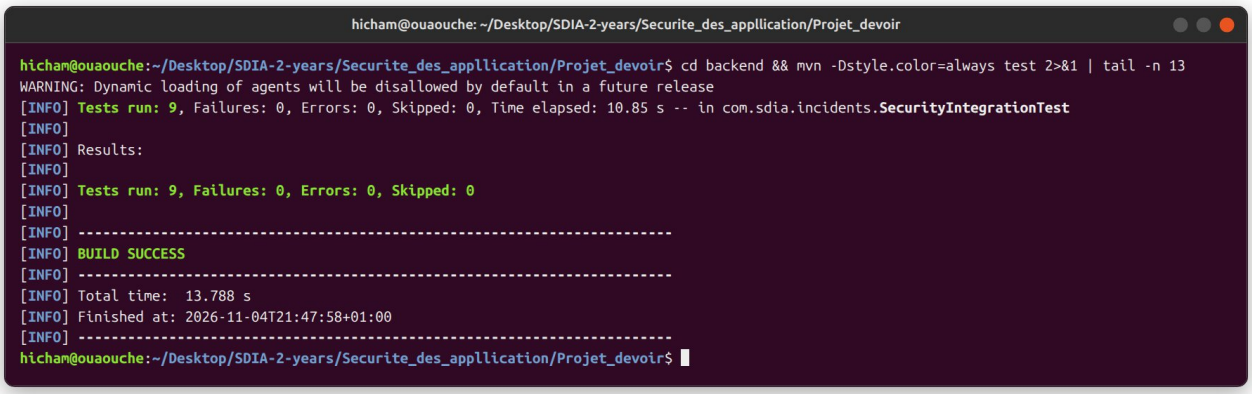


Figure 7 : mvn test : 9 tests exécutés, 0 échec, BUILD SUCCESS

8.2 Outil de test en ligne de commande

Pour obtenir des sorties lisibles pendant les tests, un petit script utilitaire api.sh (outil de test, non livré avec le projet) encapsule curl : il affiche la méthode et l'URL, le **code HTTP** (vert pour 2xx, rouge pour 4xx) et le corps JSON formaté avec jq.

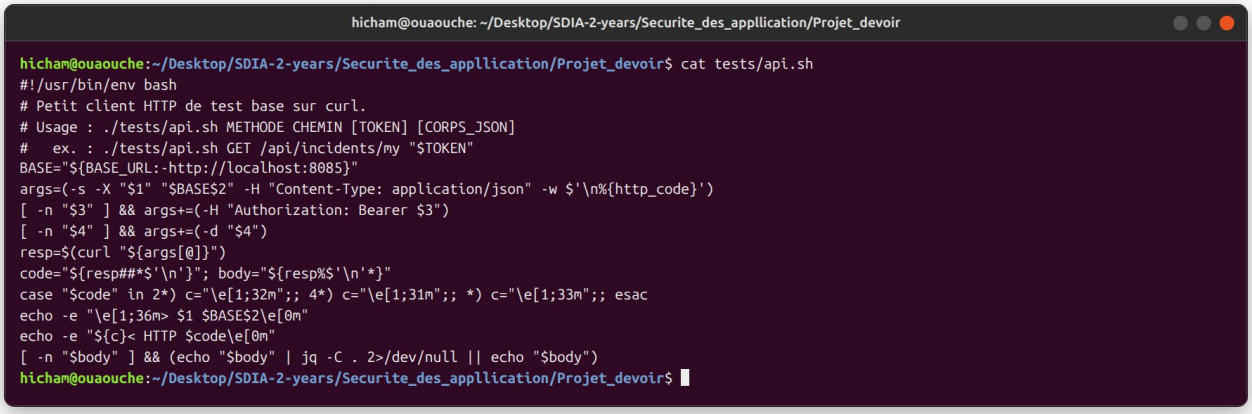


Figure 8 : *Le script utilitaire api.sh utilisé pour les tests curl*

8.3 Inscription, connexion et JWT

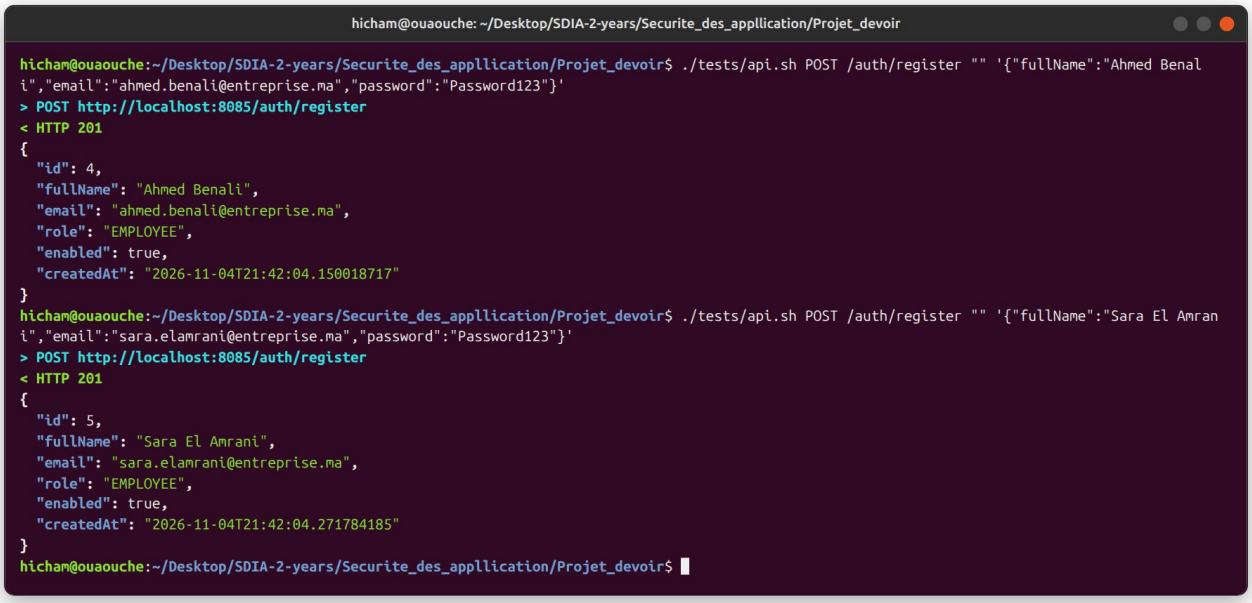


Figure 9 : *Inscription de deux employés : 201 Created, rôle EMPLOYEE, mot de passe jamais renvoyé*

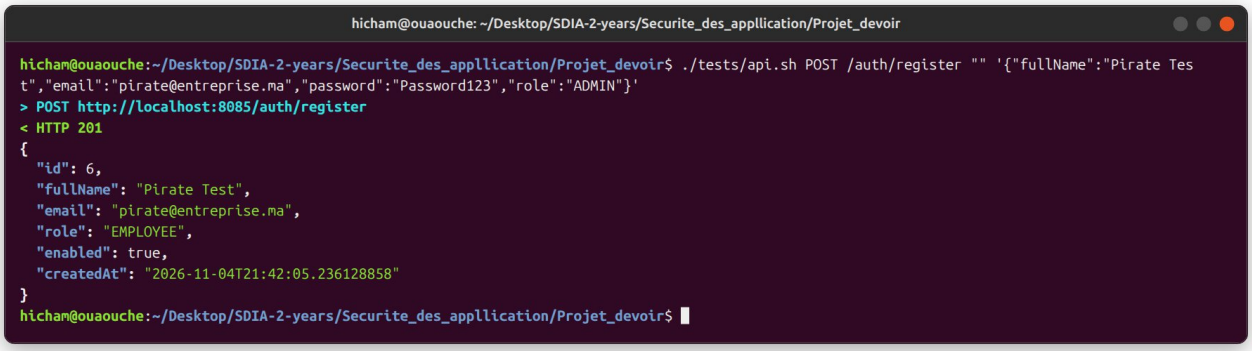


Figure 10 : *Tentative d'inscription avec "role":"ADMIN" : le champ est ignoré, le rôle reste EMPLOYEE*

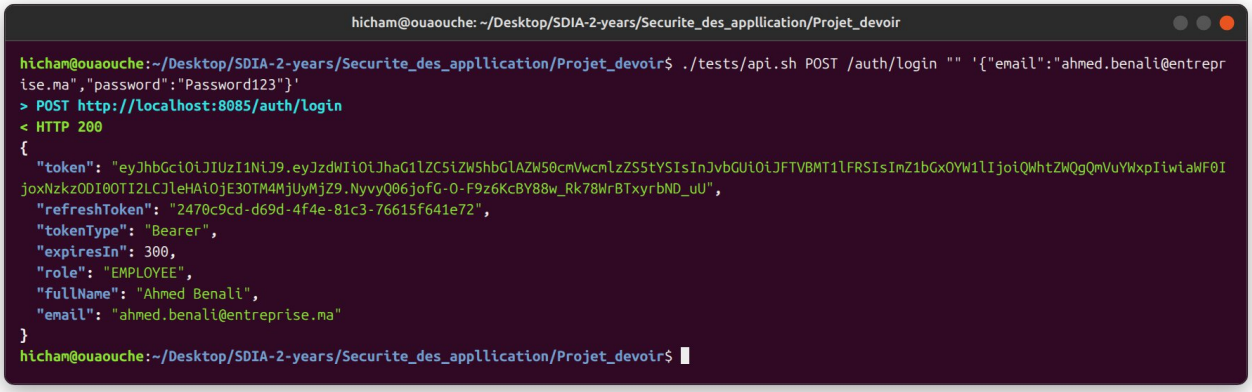


Figure 11 : *Connexion : la réponse contient token, refreshToken, role et fullName*

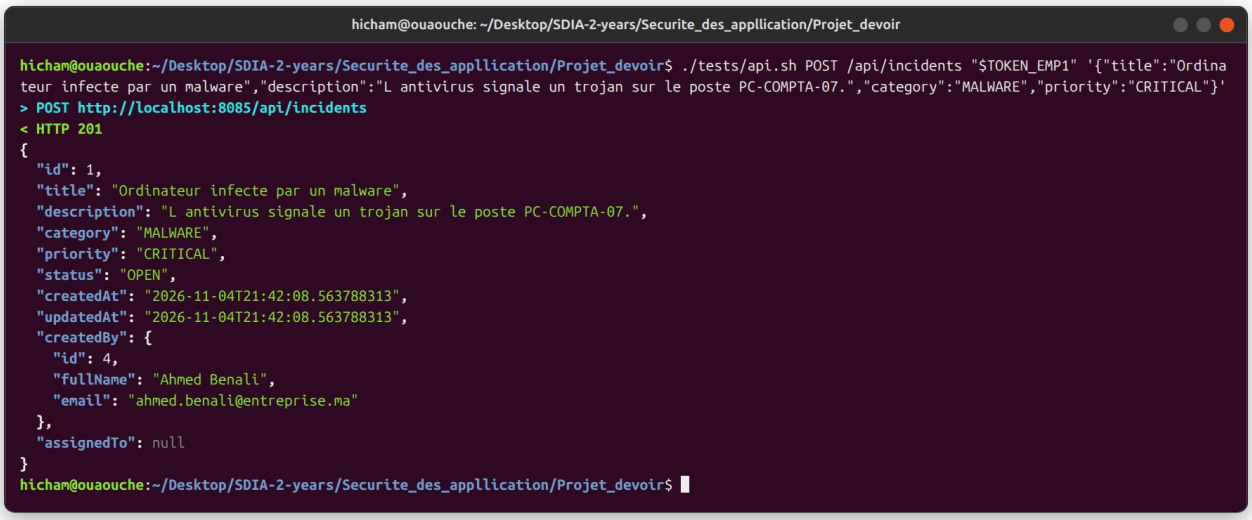


Figure 12 : *Création d'un incident par l'employé : statut initial OPEN*

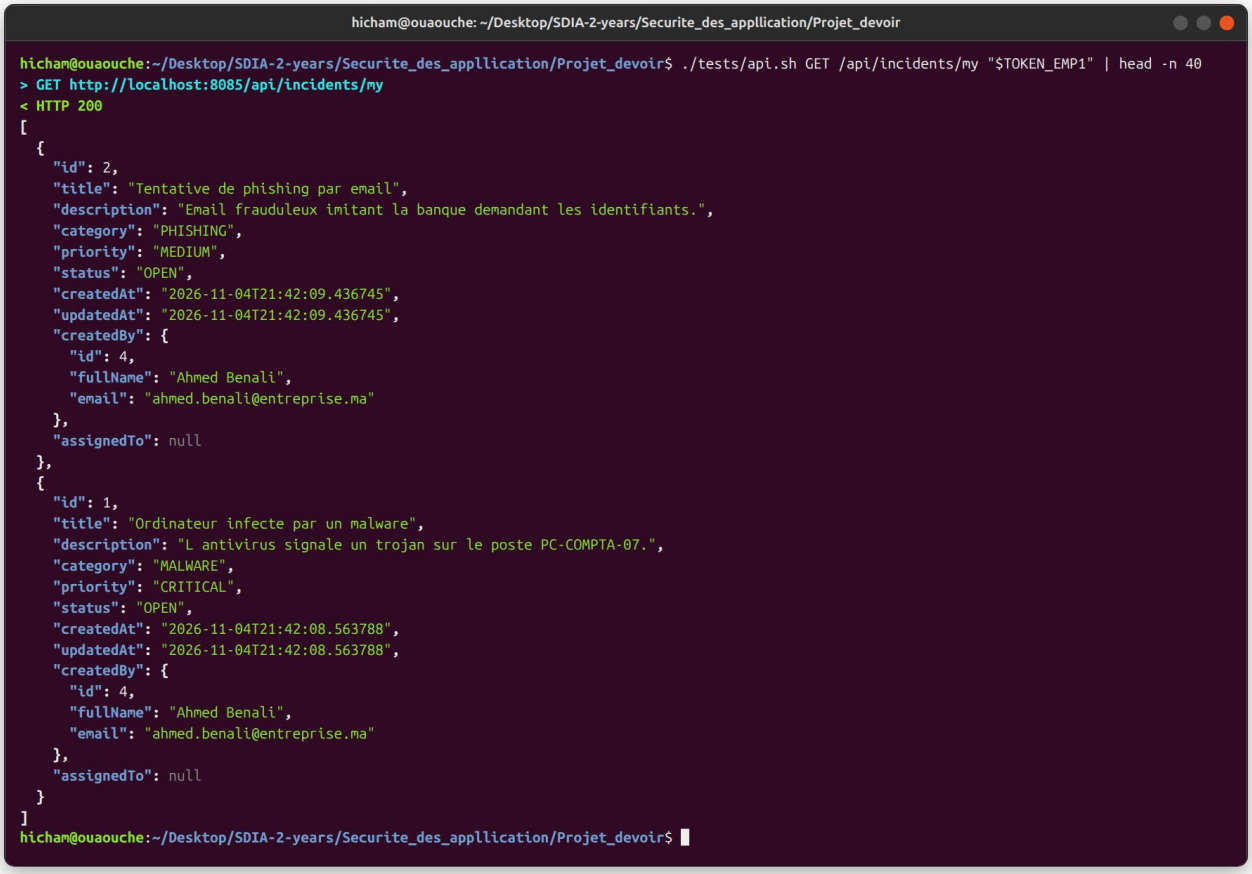


Figure 13 : *GET /api/incidents/my : l'employé ne reçoit que ses propres incidents*

8.4 Les 9 tests de sécurité obligatoires

Tableau 14 : *Synthèse des tests de sécurité obligatoires (tous réussis)*

N°	Situation testée	Résultat attendu	Résultat obtenu
1	Accès à une API protégée sans JWT	401	401 « Authentification requise : token JWT manquant »
2	Accès avec un JWT invalide (et falsifié)	401	401 « Token invalide »
3	Accès avec un JWT expiré	401	401 « Token expire »
4	EMPLOYEE sur une route ADMIN	403	403 « Accès interdit : vous n'avez pas le rôle requis »
5	EMPLOYEE sur l'incident d'un autre utilisateur	403	403 « … pas le droit de consulter cet incident »
6	TECHNICIAN sur un incident non affecté	403	403 « cet incident ne vous est pas affecté »
7	Utilisateur désactivé qui se connecte	403	403 « Compte désactivé » + ancien JWT refusé (401)
8	Inscription avec un email déjà existant	409	409 « Cet email est déjà utilisé »
9	Création d'un incident avec des données invalides	400	400 « Données invalides » + détail par champ

Test 1 : accès sans JWT

Sans en-tête Authorization, le filtre JWT ne crée aucune authentification ; la règle /api/** → authenticated() échoue et le point d'entrée renvoie **401**.

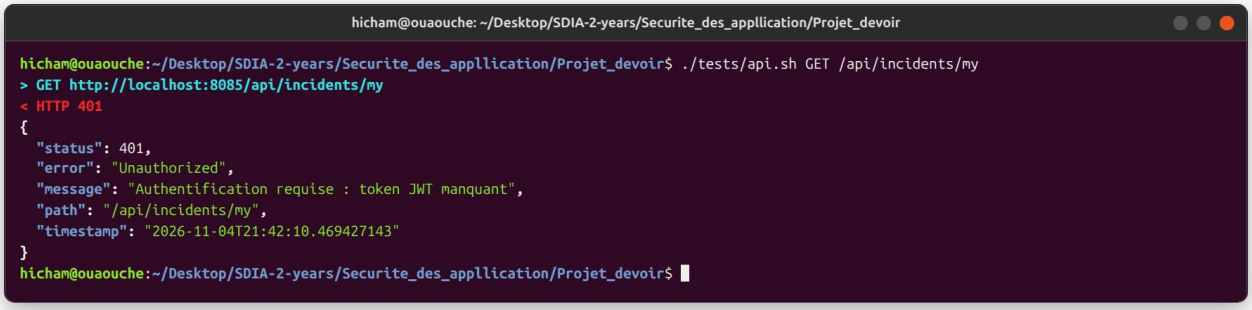


Figure 14 : *Test 1 : GET /api/incidents/my sans JWT → 401*

Test 2 : JWT invalide ou falsifié

Premier cas : une chaîne qui n'est pas un JWT. Second cas, plus intéressant : on prend le vrai token de l'employé et on remplace son payload par {"sub":...,"role":"ADMIN"} pour tenter une **élévation de privilèges**. N'ayant pas la clé secrète, l'attaquant ne peut pas recalculer la signature : la vérification HMAC échoue et la requête est rejetée.

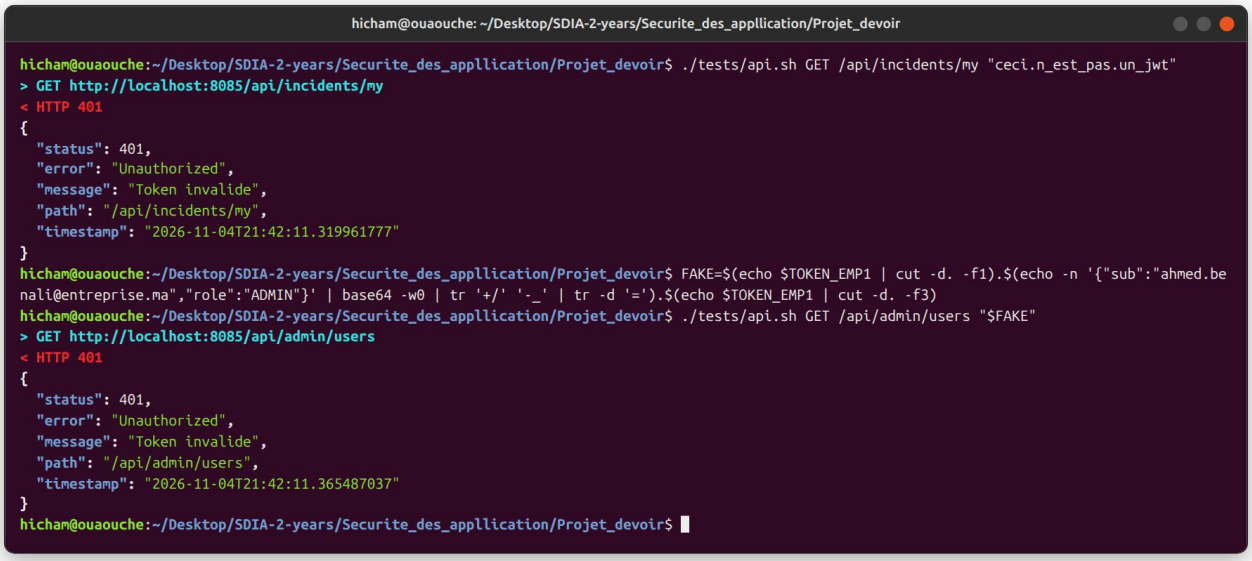


Figure 15 : *Test 2 : JWT invalide et JWT falsifié (rôle ADMIN injecté) → 401 Token invalide*

Test 3 : JWT expiré

Ce test a été réalisé **réellement** : un token obtenu au début de la session de tests a été réutilisé plus de 5 minutes après son émission. La capture montre la date d'émission, la date d'expiration et l'heure courante, puis la réponse **401 « Token expire »** (exception ExpiredJwtException levée par JJWT).

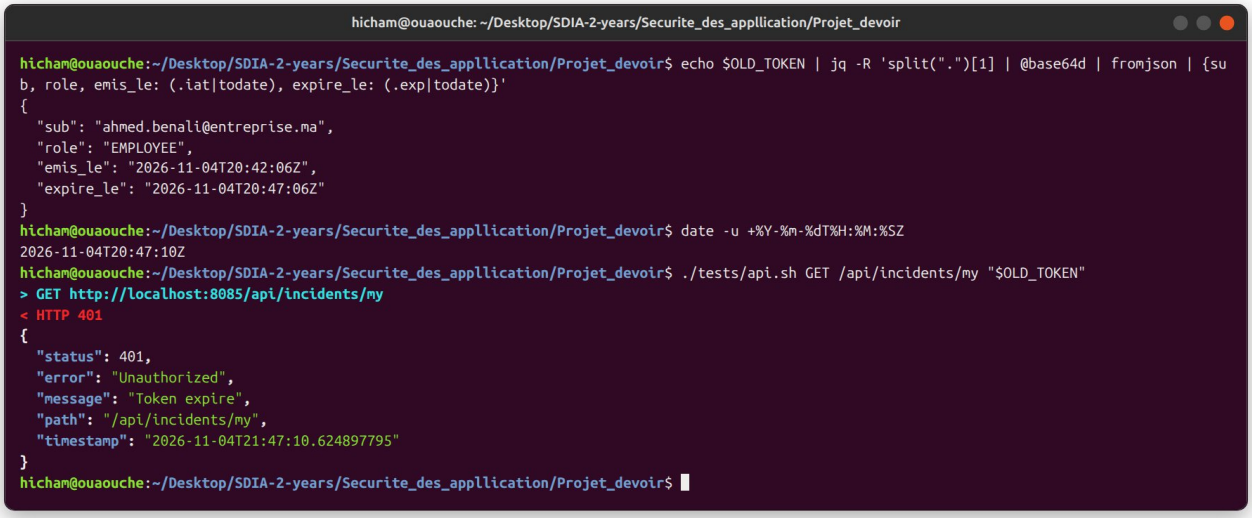


Figure 16 : *Test 3 : utilisation d'un JWT après son expiration réelle → 401 Token expire*

Test 4 : EMPLOYEE sur une route ADMIN

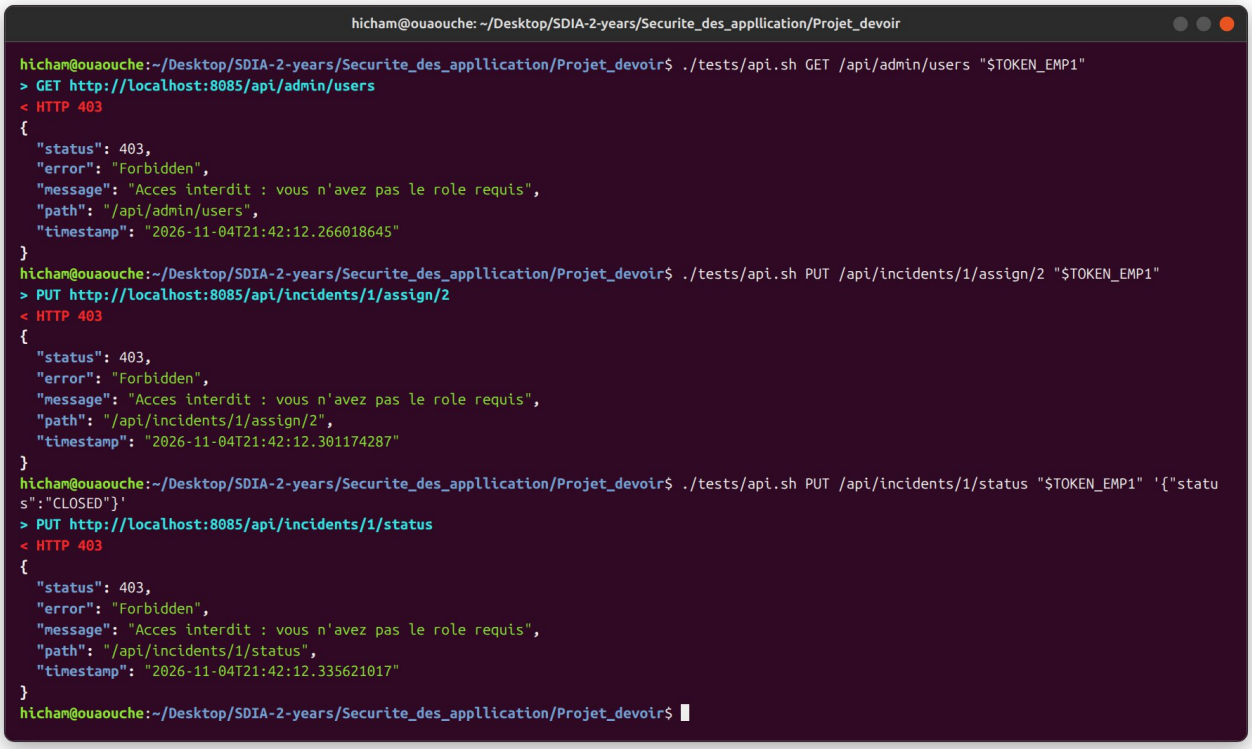


Figure 17 : *Test 4 : un EMPLOYEE appelle /api/admin/users, tente une affectation et un changement de statut → 403*

Test 5 : EMPLOYEE sur l'incident d'un autre utilisateur

L'incident n°3 appartient à Sara. Ahmed reçoit **403** alors que Sara, propriétaire, reçoit **200** : le contrôle du propriétaire est bien fait côté serveur.

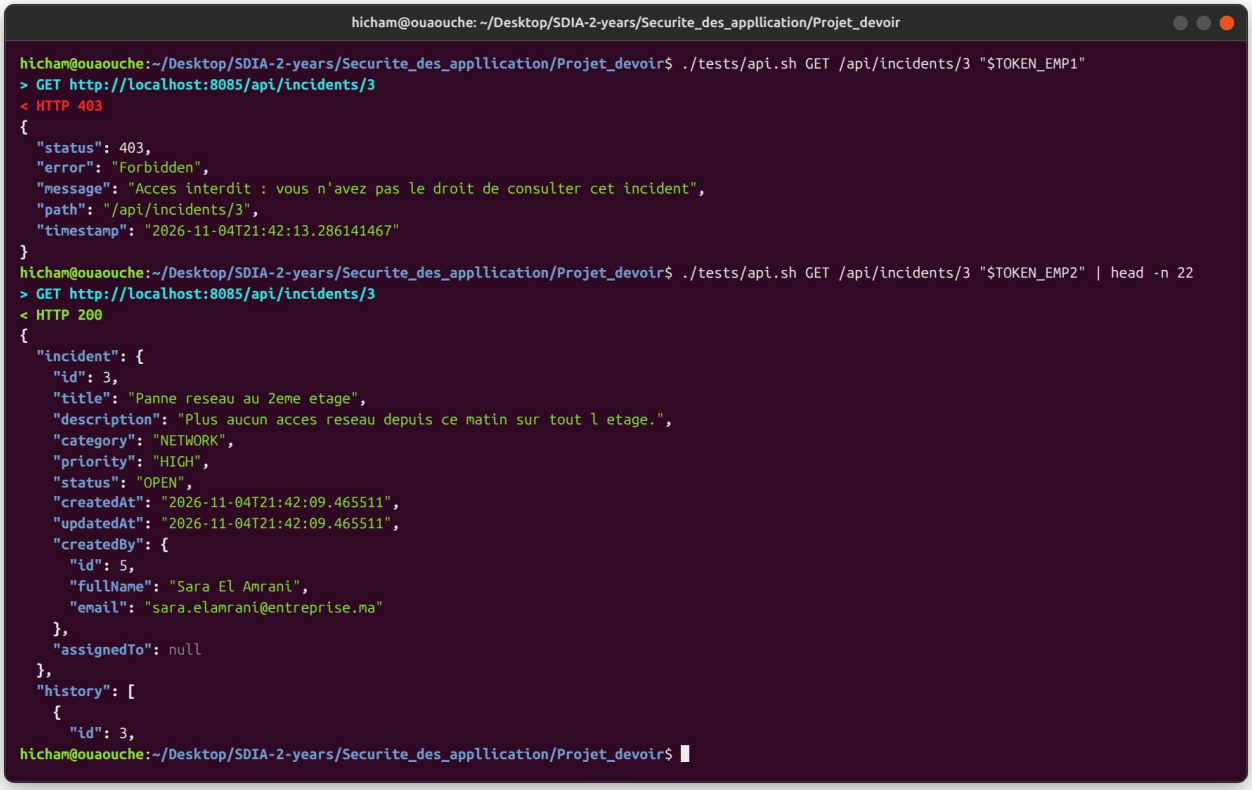


Figure 18 : *Test 5 : Ahmed → incident de Sara : 403 ; Sara → son propre incident : 200*

Test 6 : TECHNICIAN sur un incident qui ne lui est pas affecté

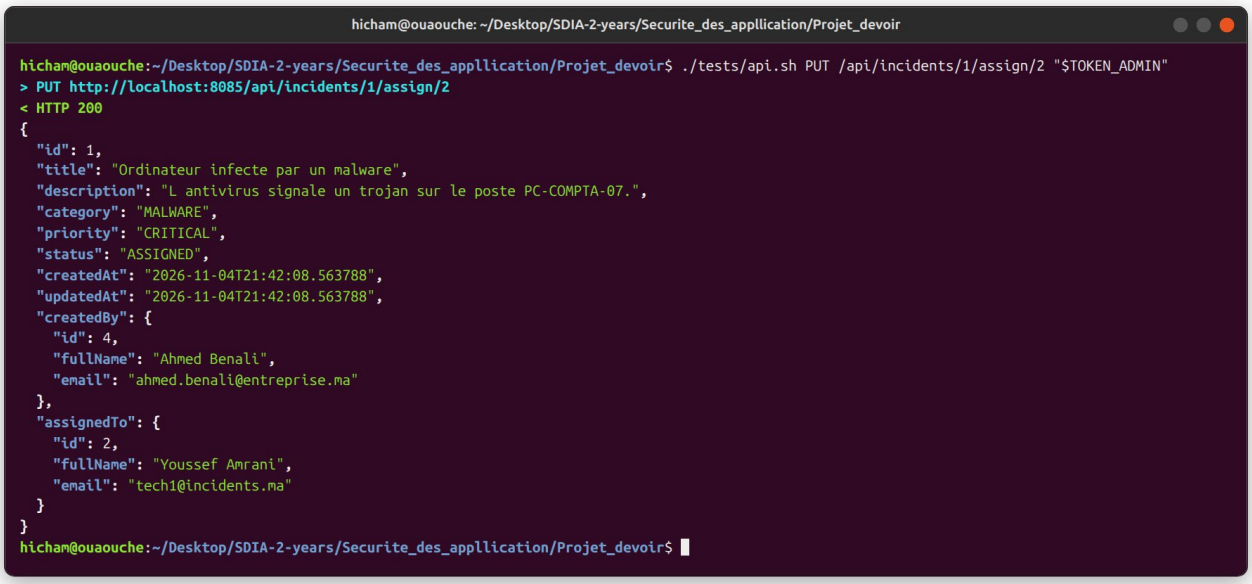


Figure 19 : *Préparation : l'ADMIN affecte l'incident 1 au technicien Youssef Amrani (OPEN → ASSIGNED)*

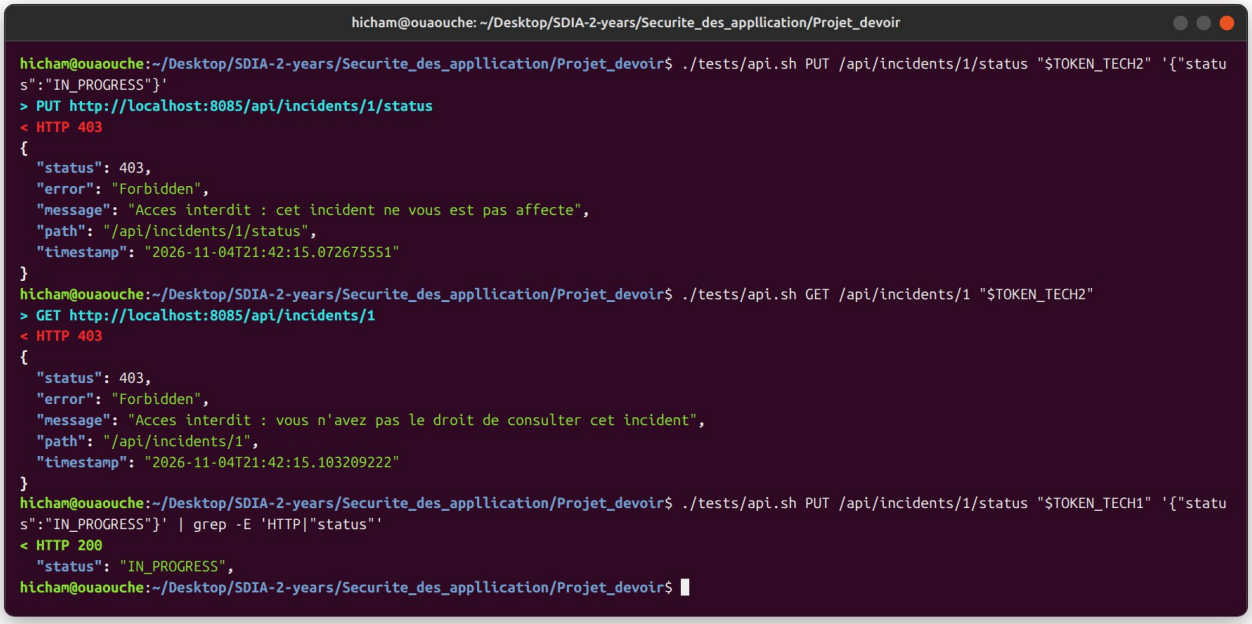


Figure 20 : *Test 6 : Karim (non affecté) → 403 en écriture et en lecture ; Youssef (affecté) → 200*

Test 7 : utilisateur désactivé

L'administrateur désactive le compte de Sara. Sa tentative de connexion échoue (**403 Compte désactivé**, DisabledException levée par le DaoAuthenticationProvider). De plus, le JWT qu'elle avait obtenu **avant** la désactivation, pourtant encore valide, est refusé (**401**) car le filtre recharge l'utilisateur en base à chaque requête.

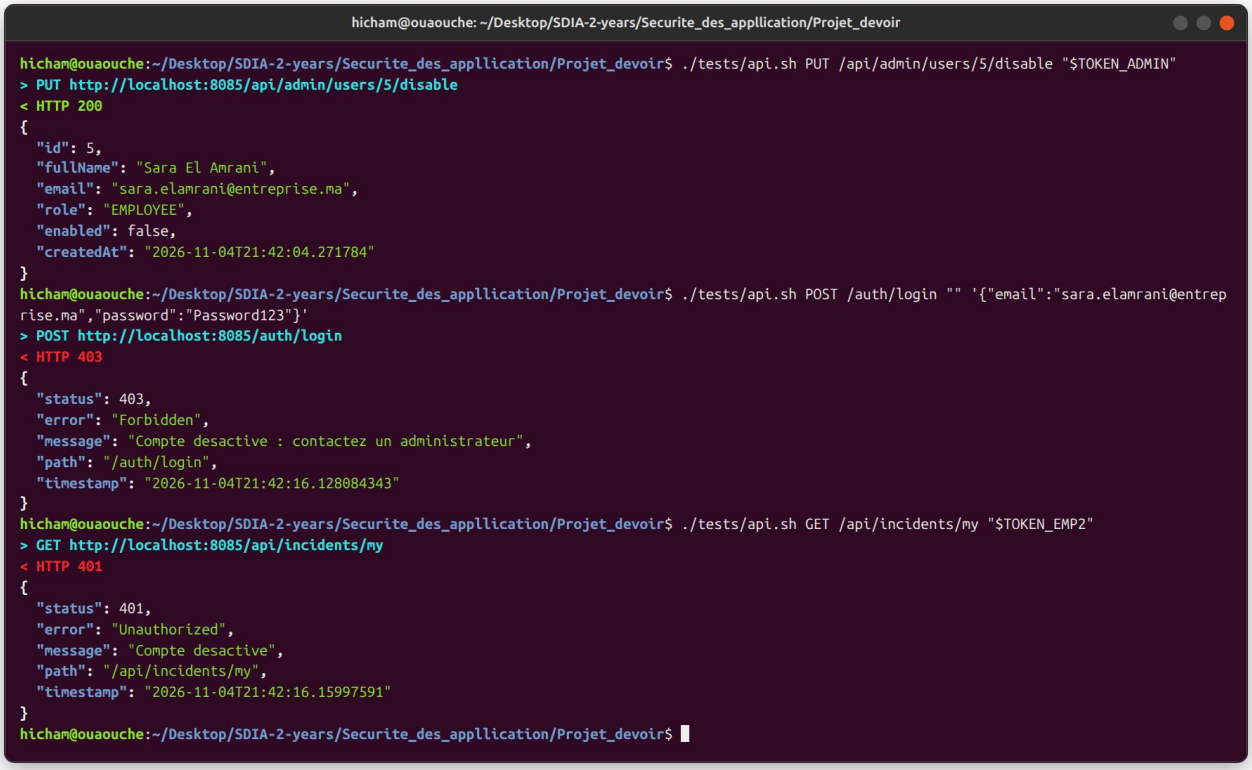


Figure 21 : *Test 7 : désactivation, connexion refusée, ancien JWT refusé*

Test 8 : inscription avec un email déjà existant

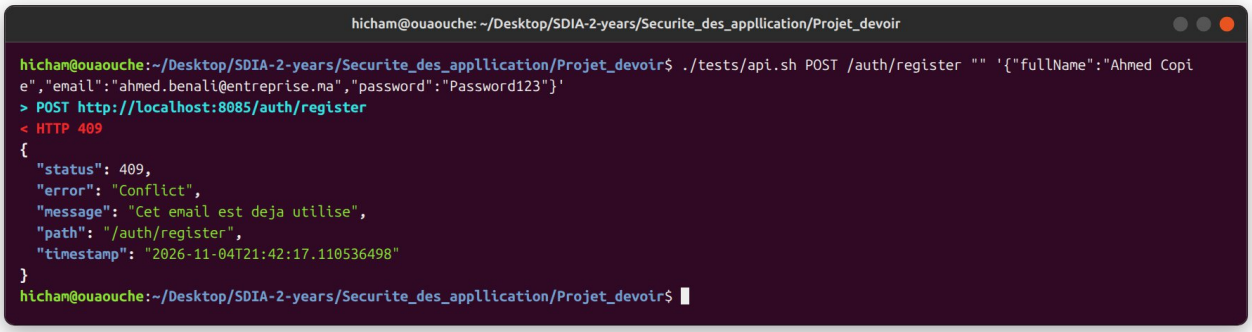


Figure 22 : Test 8 : email déjà utilisé → 409 Conflict

Test 9 : données invalides

Titre vide, description trop courte, priorité absente : la réponse 400 détaille l'erreur de **chaque champ**. Une valeur d'énumération inconnue (VIRUS) et une inscription avec email et mot de passe invalides sont également rejetées.

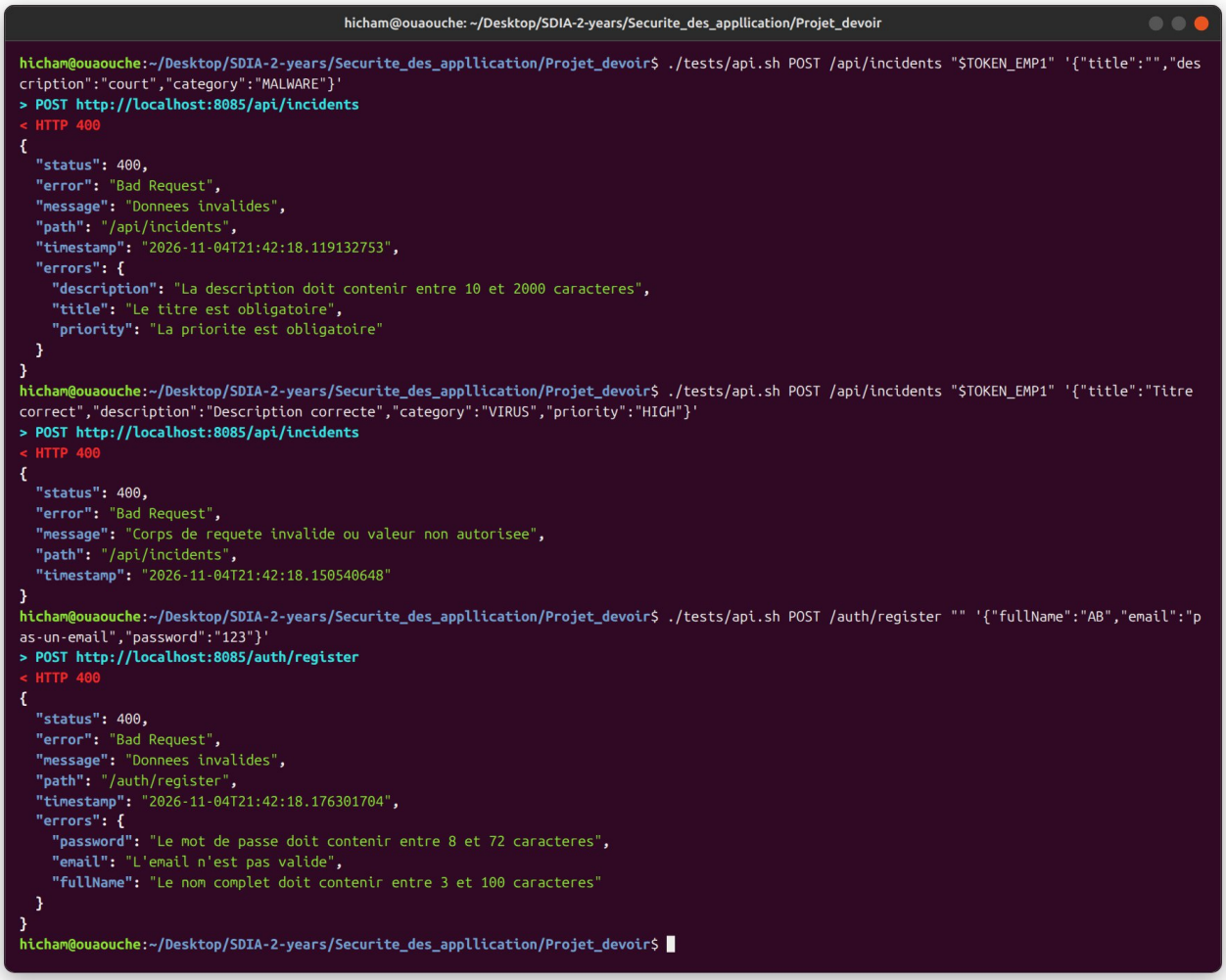


Figure 23 : Test 9 : création d'incident et inscription avec des données invalides → 400

8.5 Tests complémentaires

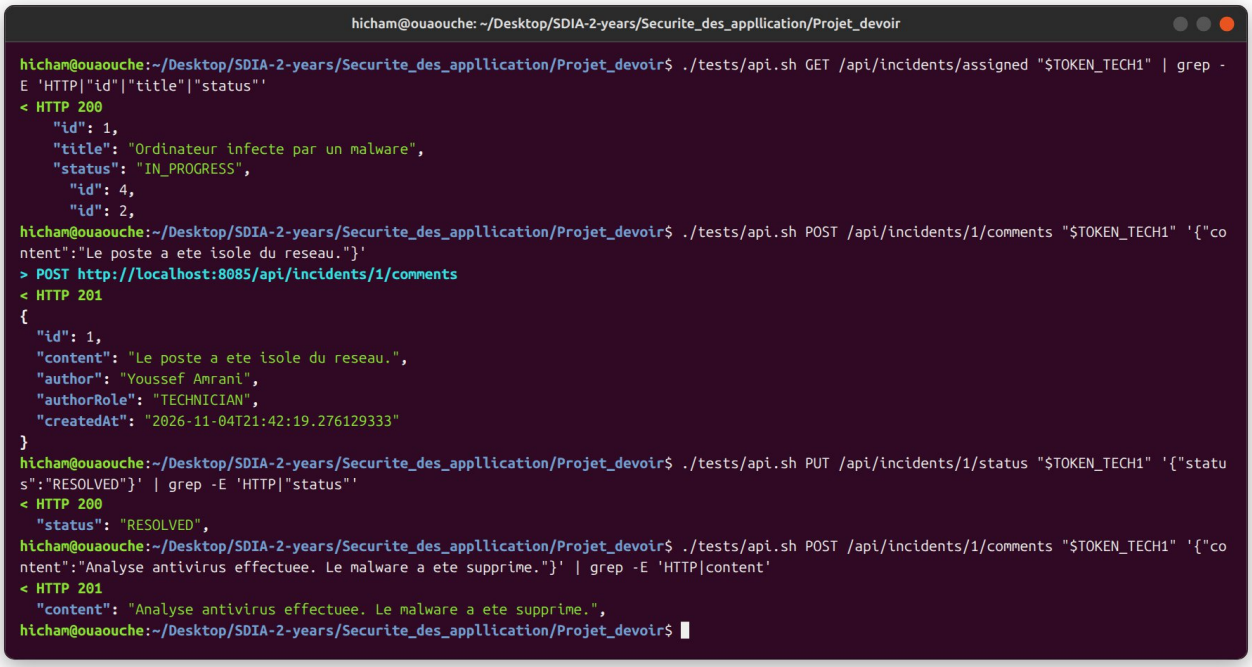


Figure 24 : *Traitement par le technicien : incidents affectés, note de traitement, passage à RESOLVED*

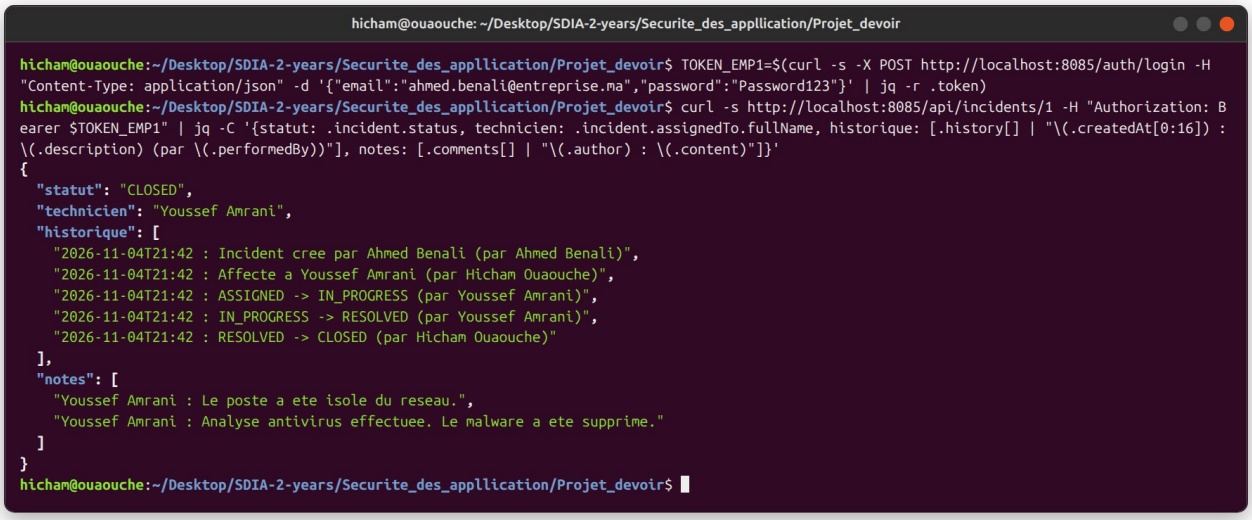


Figure 25 : *L'employé consulte l'état final (CLOSED après clôture par l'admin), l'historique complet et les notes de son incident*

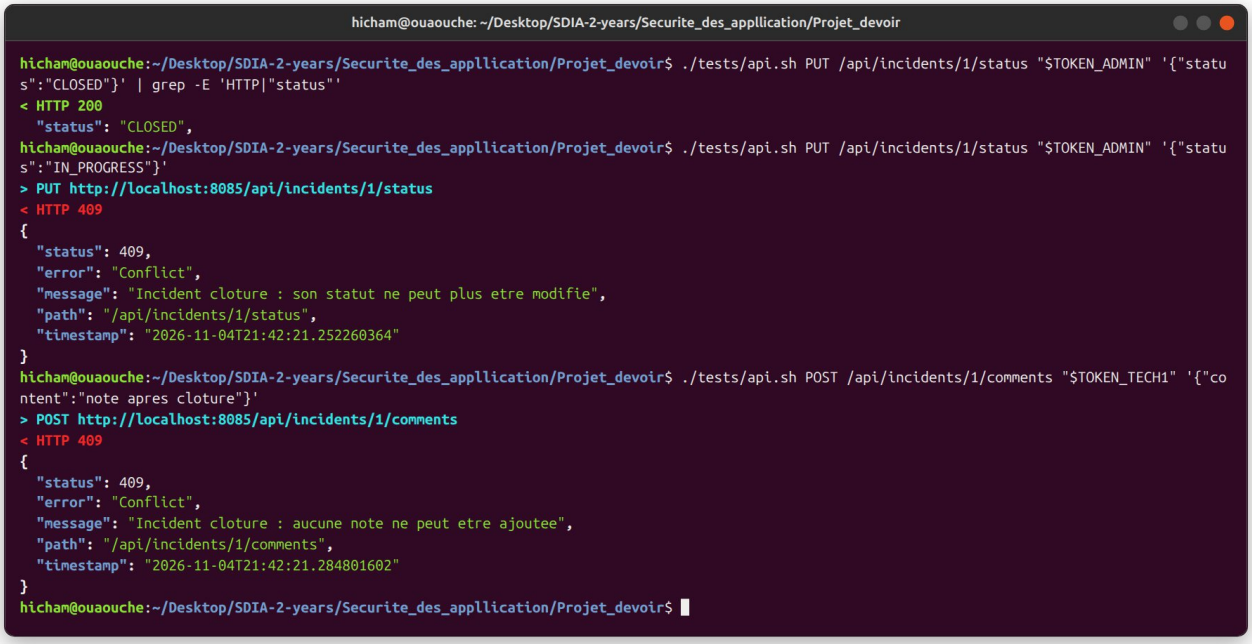


Figure 26 : *Règle 8 : après CLOSED, plus aucune modification de statut ni ajout de note (409)*

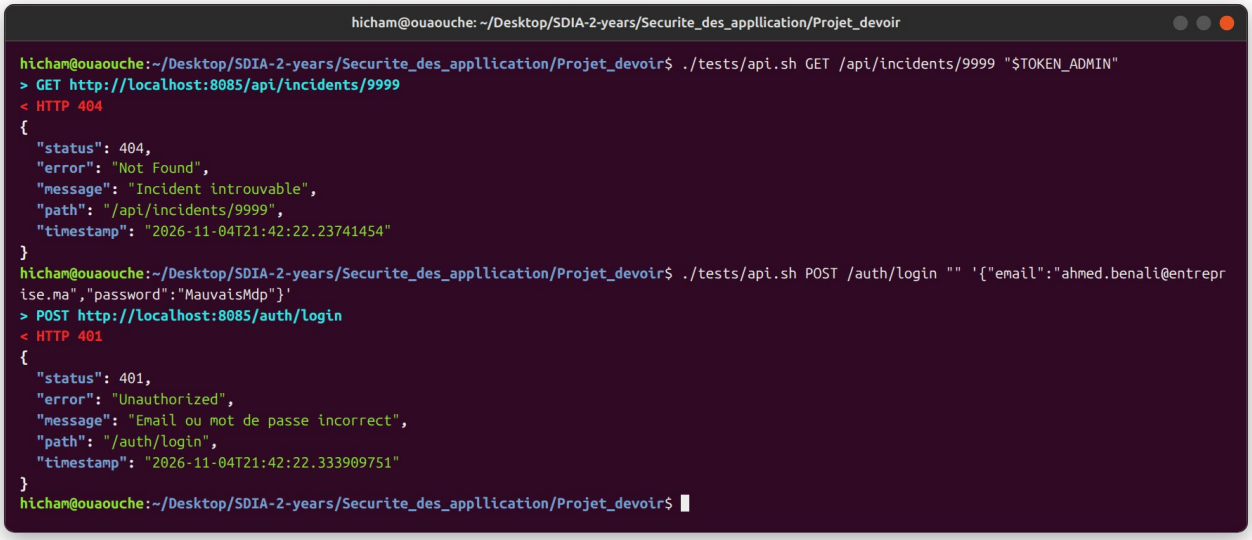


Figure 27 : *Incident inexistant (404) et mauvais mot de passe (401)*

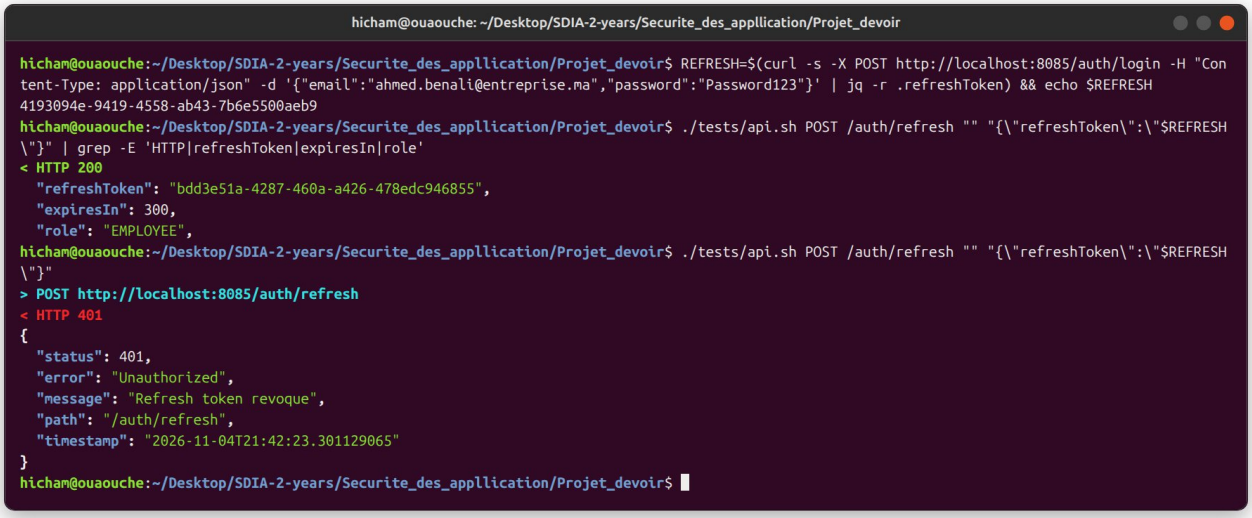


Figure 28 : *Refresh token : obtention d'un nouvel access token puis rejet de l'ancien refresh token (rotation)*

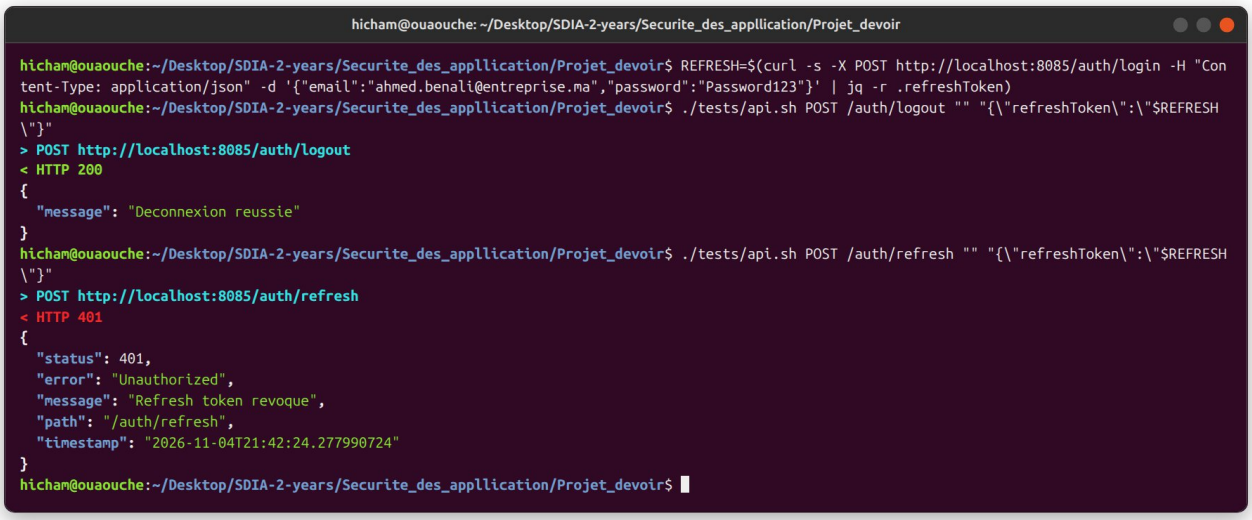


Figure 29 : *Déconnexion : le refresh token révoqué ne permet plus d'obtenir de token*

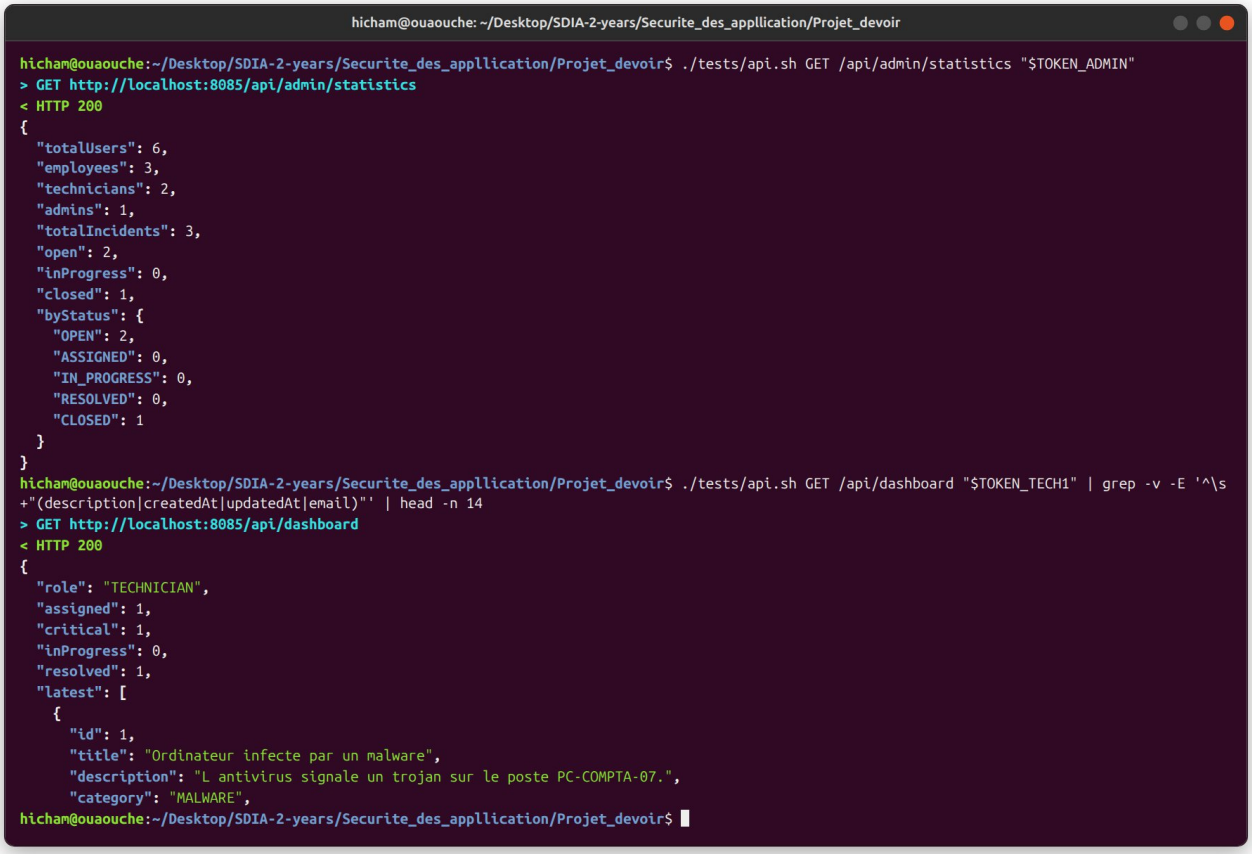


Figure 30 : *Statistiques ADMIN et dashboard TECHNICIAN calculés côté serveur*



Figure 31 : *Changement de mot de passe : ancien mot de passe vérifié, confirmation vérifiée*

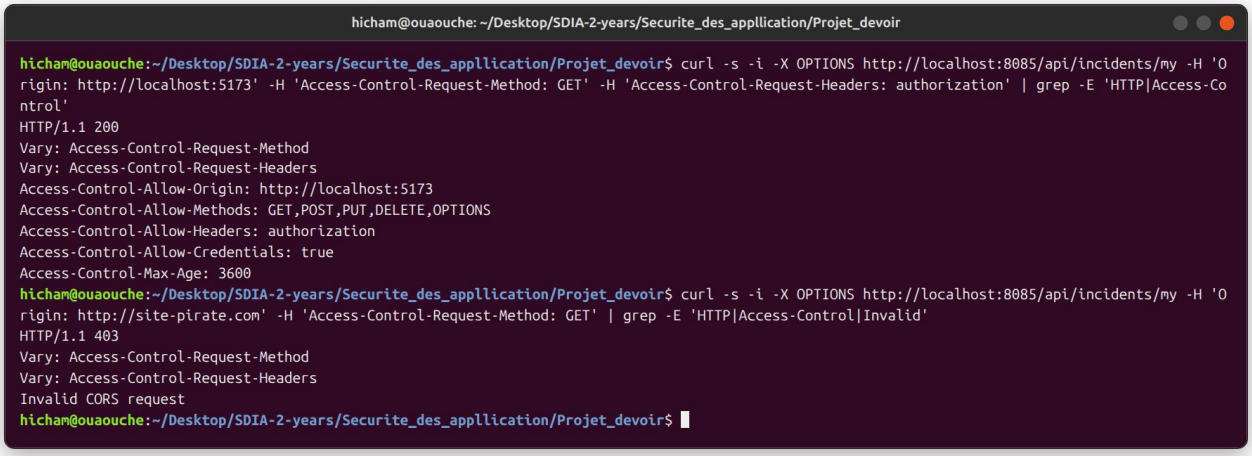


Figure 32 : *CORS : l'origine http://localhost:5173 est autorisée, une origine inconnue est refusée (403)*

8.6 Collection Postman exécutée avec Newman

La collection Postman Incidents_JWT.postman_collection.json a été préparée pour ce projet. Elle contient deux dossiers : le **scénario de test** du sujet (25 requêtes) et les **tests de sécurité** (27 requêtes). Chaque requête possède des **assertions** (onglet Tests de Postman) et les tokens sont chaînés automatiquement via des variables de collection. Le JWT expiré est fabriqué par un script de pré-requête qui signe un token dont exp est dans le passé, ce qui évite d'attendre 5 minutes.

Newman est l'exécuteur en ligne de commande officiel de Postman : il lance exactement la même collection et affiche le résultat de chaque assertion.



Figure 33 : *Newman : scénario de test (1/2)*



Figure 34 : *Newman : scénario de test (2/2)*

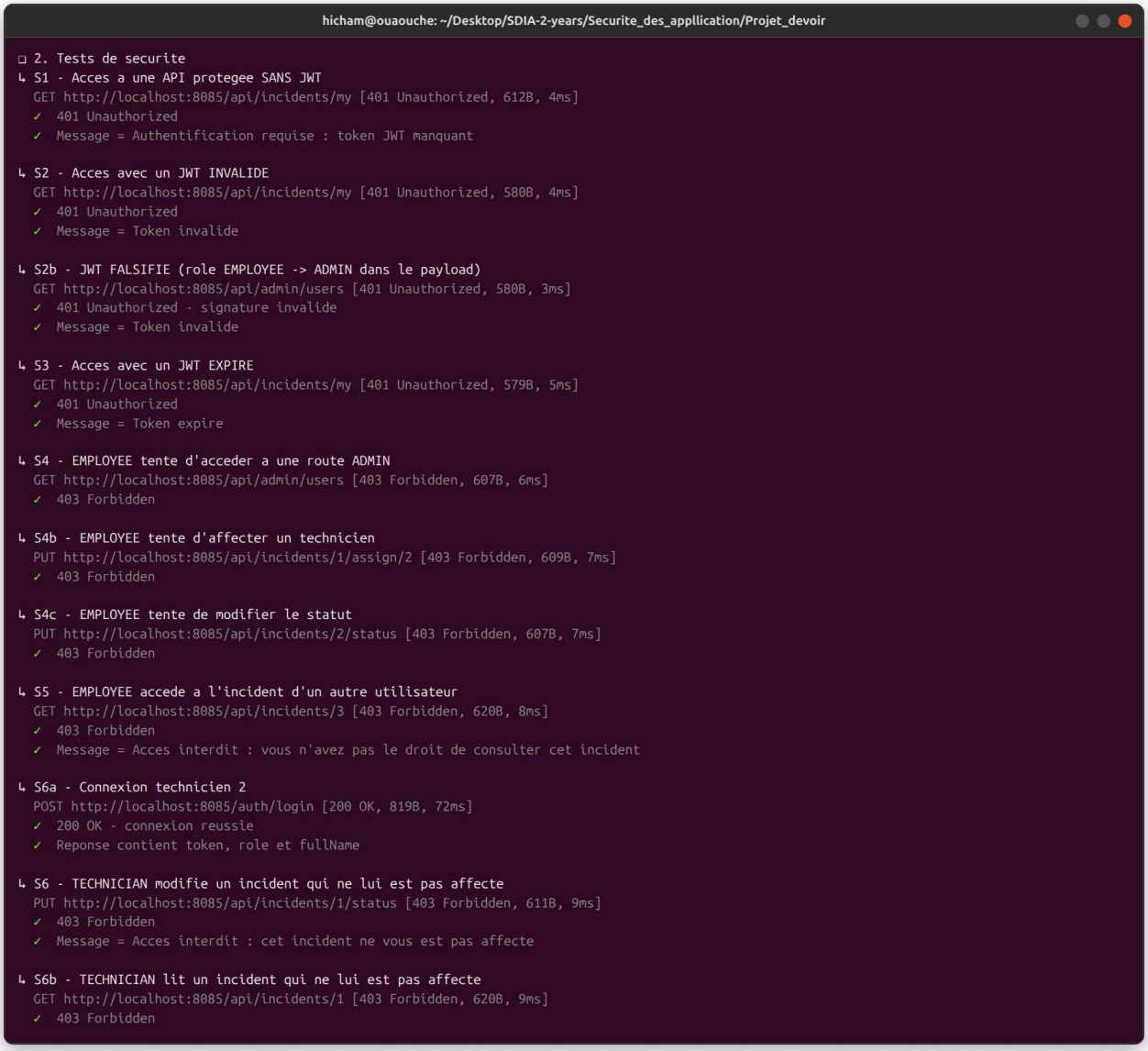


Figure 35 : *Newman : tests de sécurité (1/2)*



Figure 36 : *Newman : tests de sécurité (2/2)*

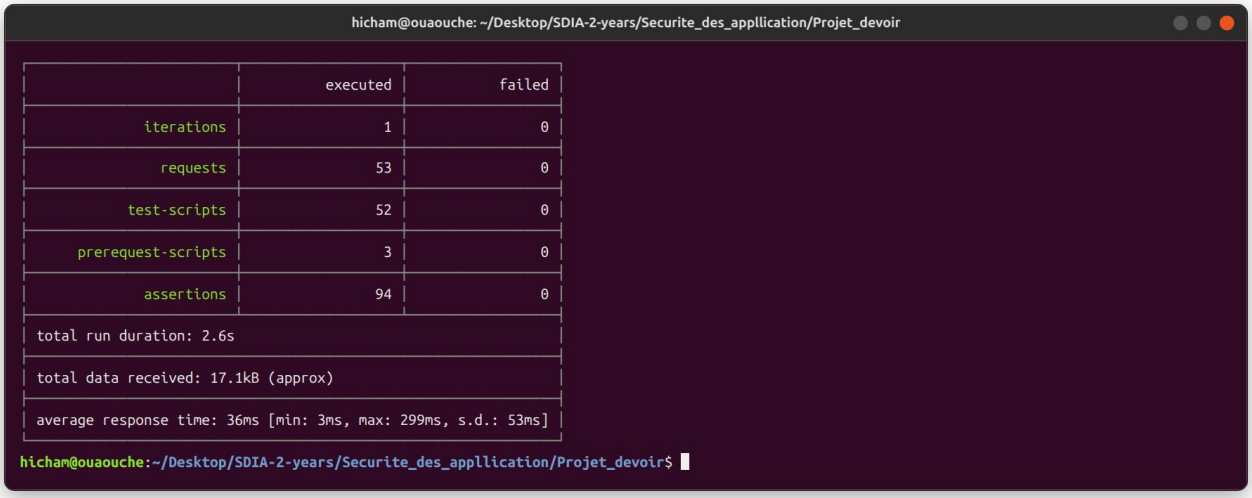


Figure 37 : *Newman : résumé, 53 requêtes, 94 assertions, 0 échec*

8.7 Tests réalisés dans l'application Postman

La collection Incidents_JWT.postman_collection.json a été importée dans l'application **Postman** (File → Import). Le dossier « 1. Scenario de test » a d'abord été exécuté avec le **Collection Runner** pour créer les comptes, les incidents et les tokens. Les tests de sécurité ont ensuite été lancés un par un avec le bouton **Send**.

Chaque capture montre la requête (méthode et URL), le **code HTTP** obtenu, le **corps JSON** renvoyé par l'API et le résultat des assertions dans l'onglet **Test Results**.

> **Remarque : expiration du JWT pendant les tests manuels**
>
> L'access token ne vit que **5 minutes**. Lors d'un premier essai manuel, les tokens obtenus pendant le scénario avaient expiré avant certains tests, qui renvoyaient alors 401 Token expire. C'est le comportement attendu, mais cela masquait le test visé. Les requêtes de la collection contiennent donc un **script de pré-requête** qui se reconnecte automatiquement (pm.sendRequest vers /auth/login) juste avant l'envoi : chaque test dispose d'un JWT valide, quel que soit le temps écoulé.

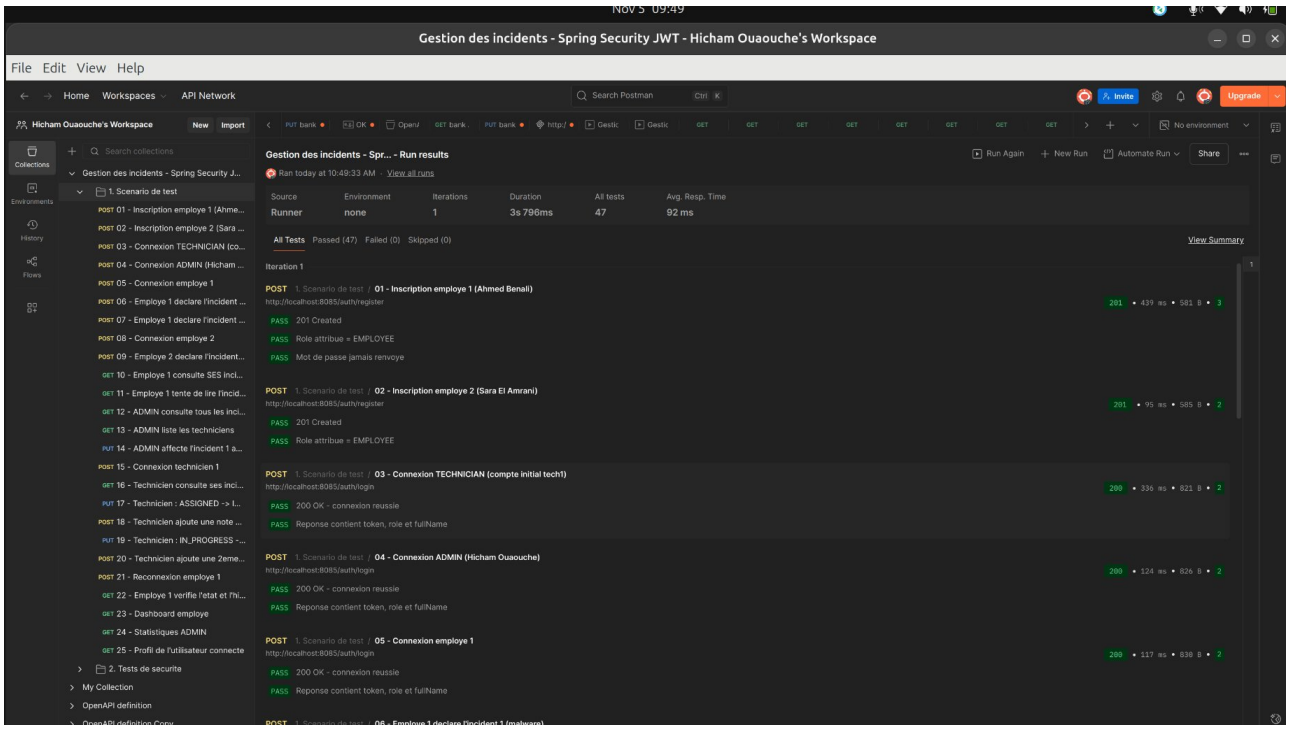


Figure 38 : *Postman Collection Runner : dossier « 1. Scenario de test », 47 assertions réussies, 0 échec (1/2)*

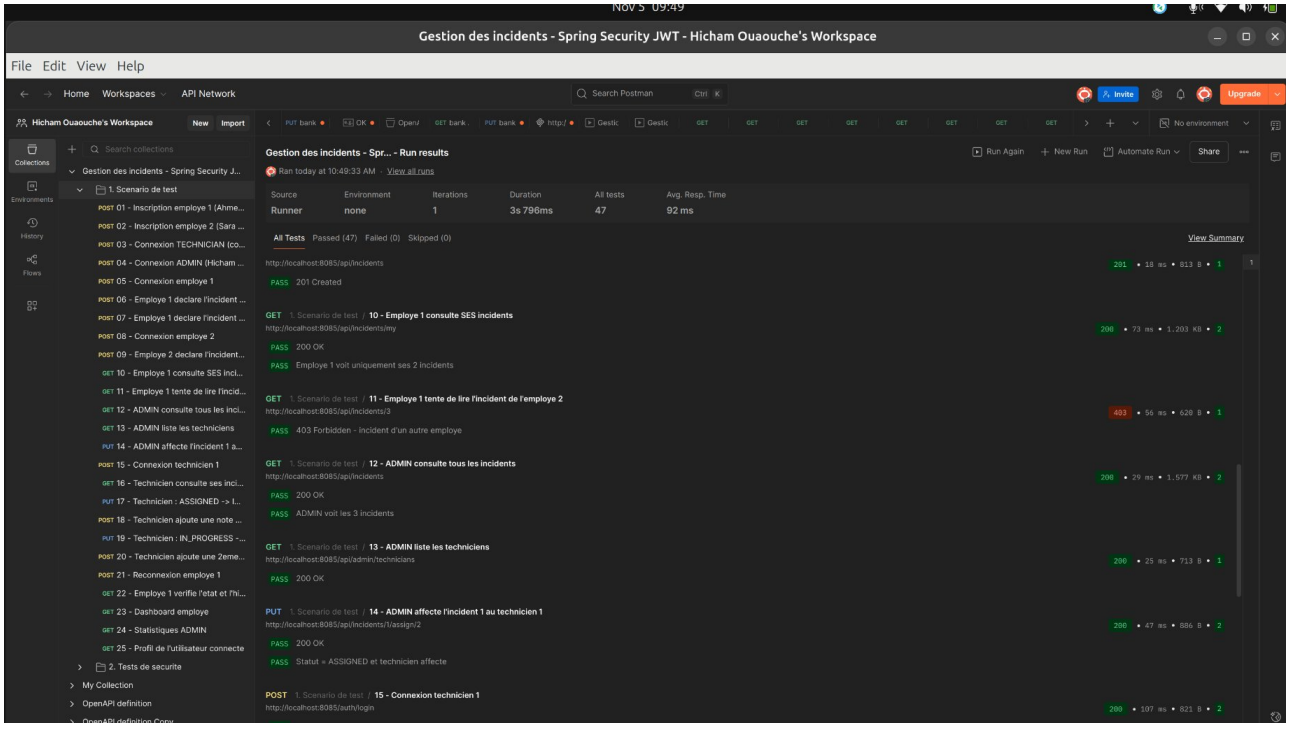


Figure 39 : *Postman Collection Runner : suite du scénario (dont la requête 11 : 403 sur l'incident d'un autre employé) (2/2)*

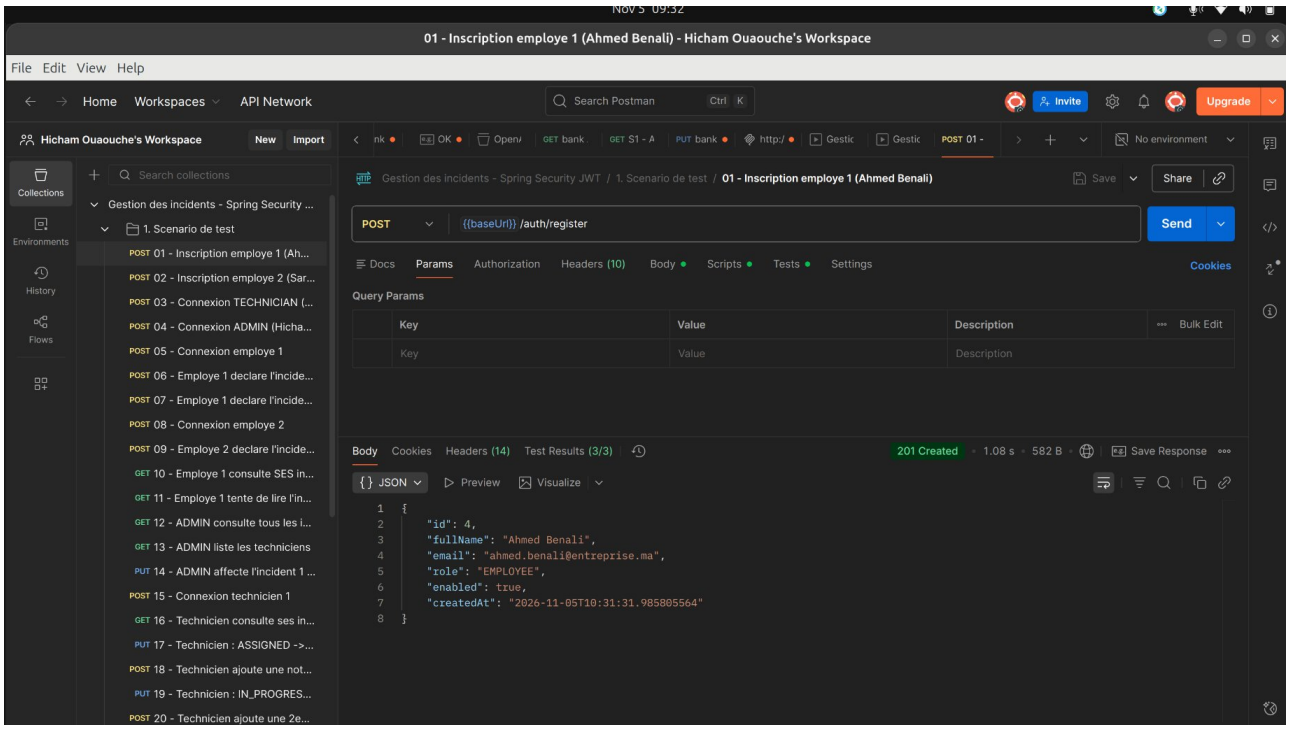


Figure 40 : *Postman : inscription d'un employé → 201 Created, rôle EMPLOYEE, mot de passe absent de la réponse*

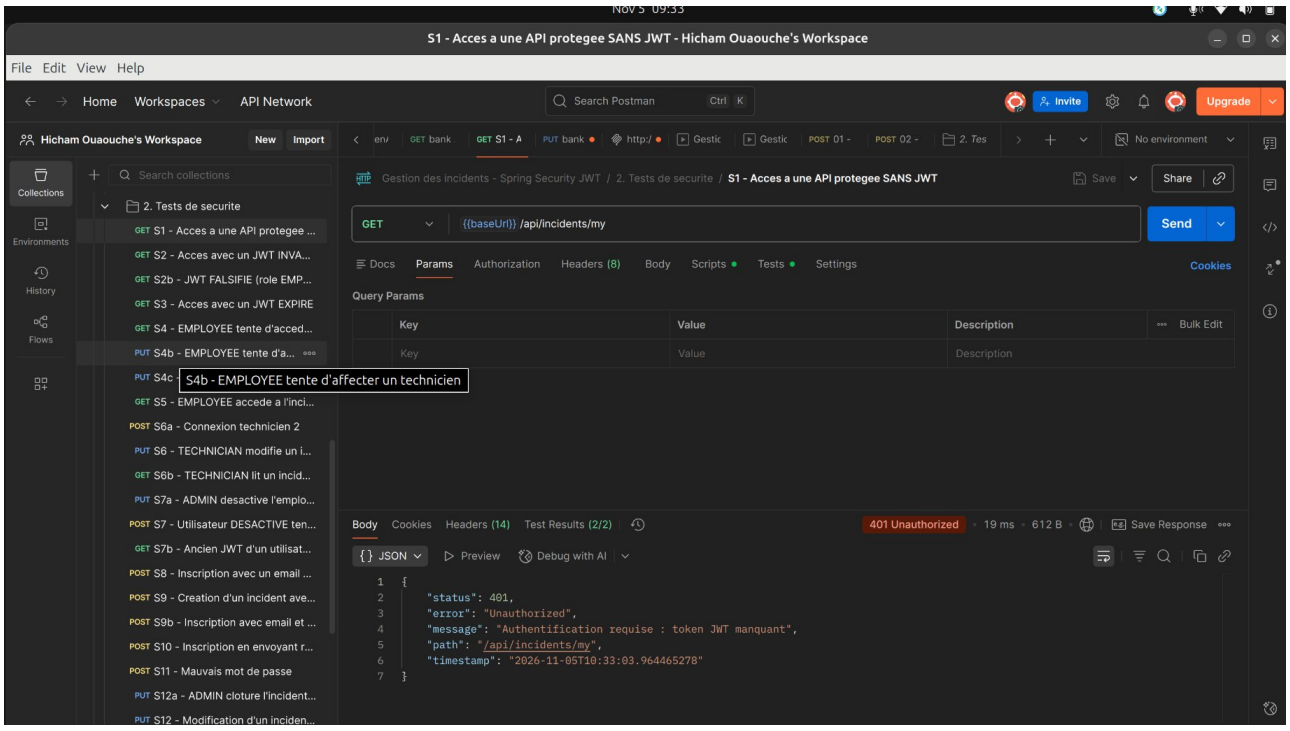


Figure 41 : *Postman, test 1 : accès à une API protégée sans JWT → 401 « token JWT manquant »*

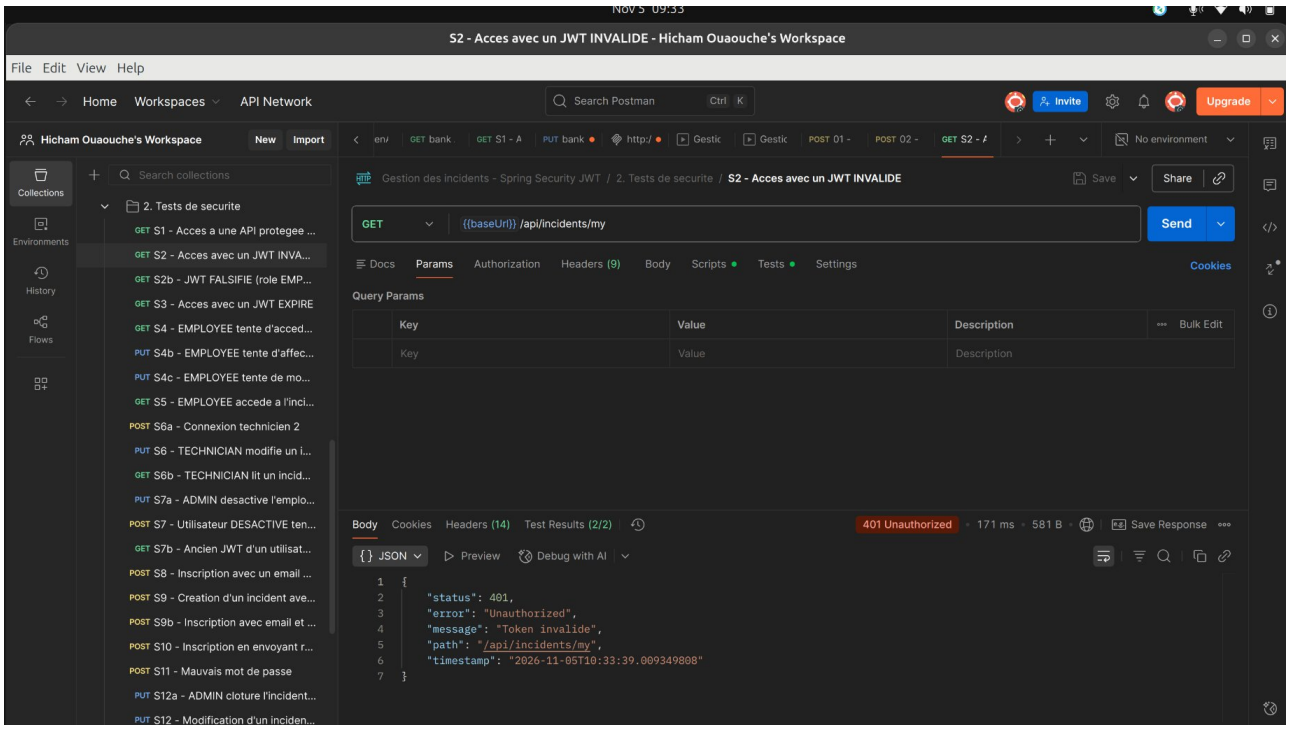


Figure 42 : *Postman, test 2 : accès avec un JWT invalide → 401 « Token invalide »*

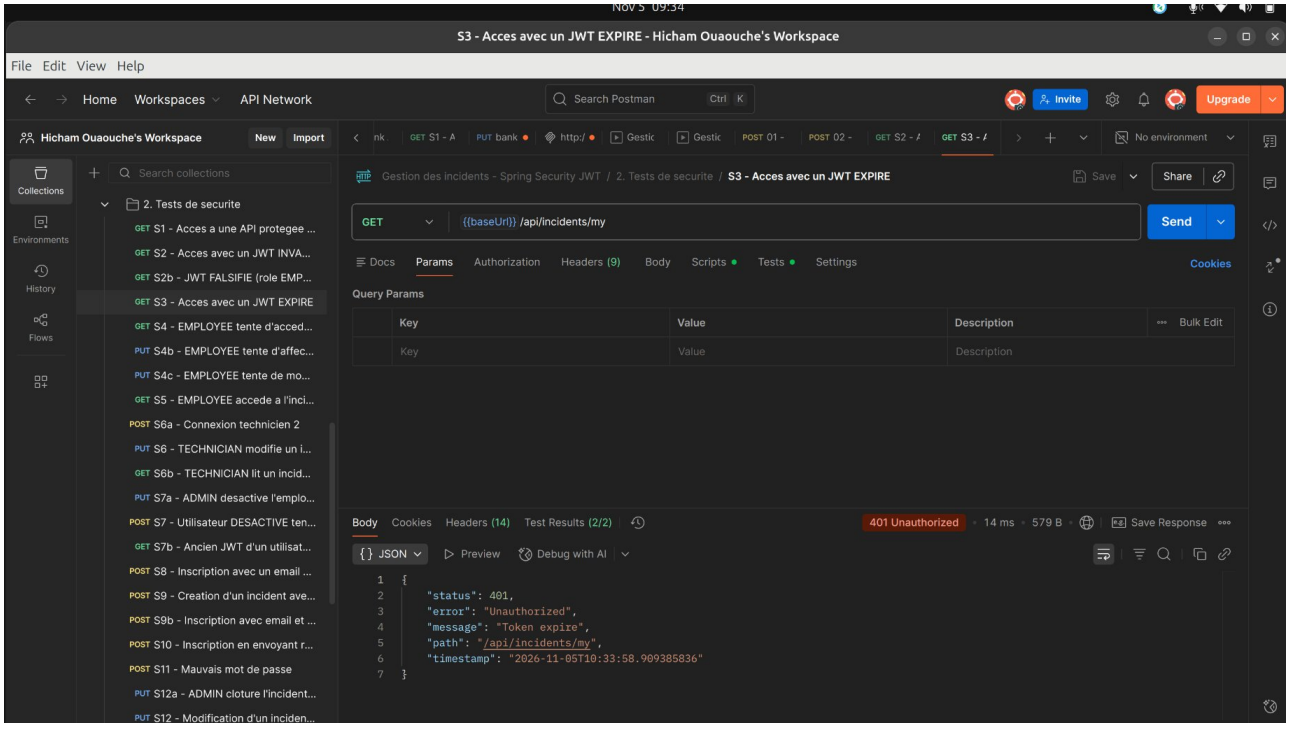


Figure 43 : *Postman, test 3 : accès avec un JWT expiré → 401 « Token expire »*

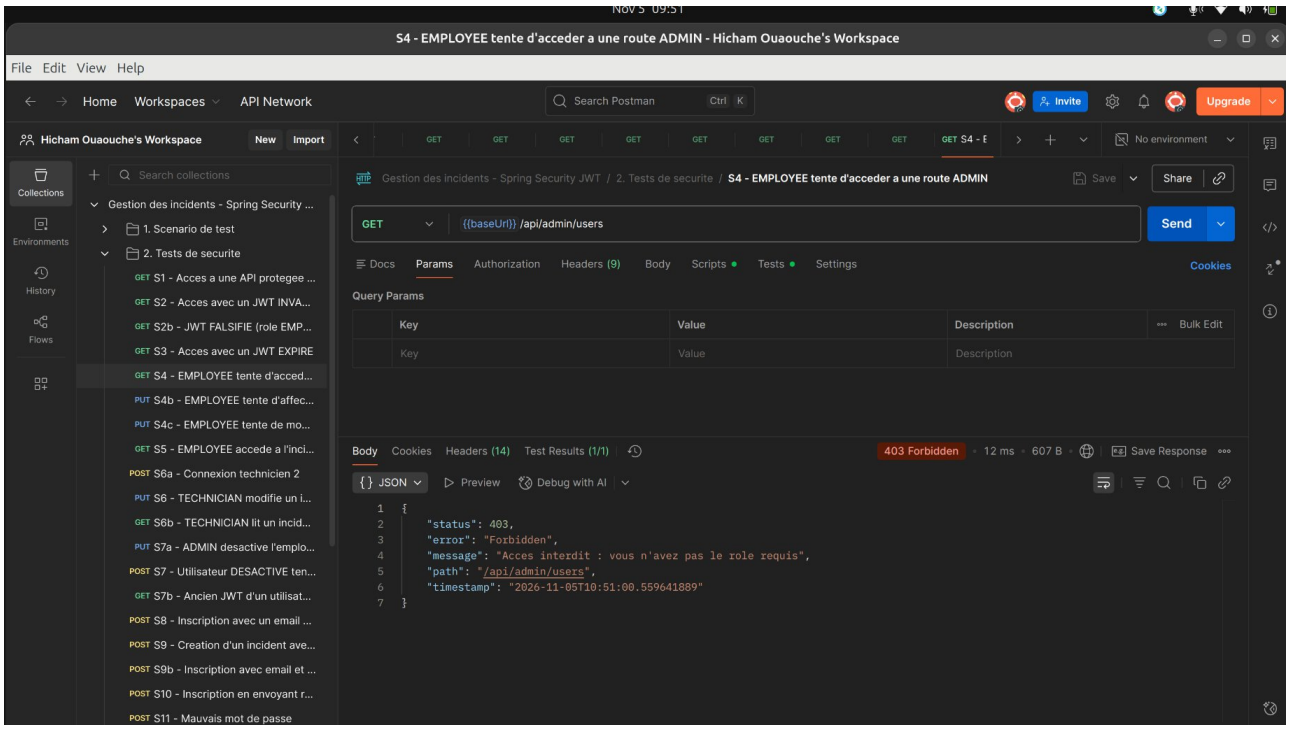


Figure 44 : *Postman, test 4 : EMPLOYEE sur la route ADMIN /api/admin/users → 403 Forbidden*

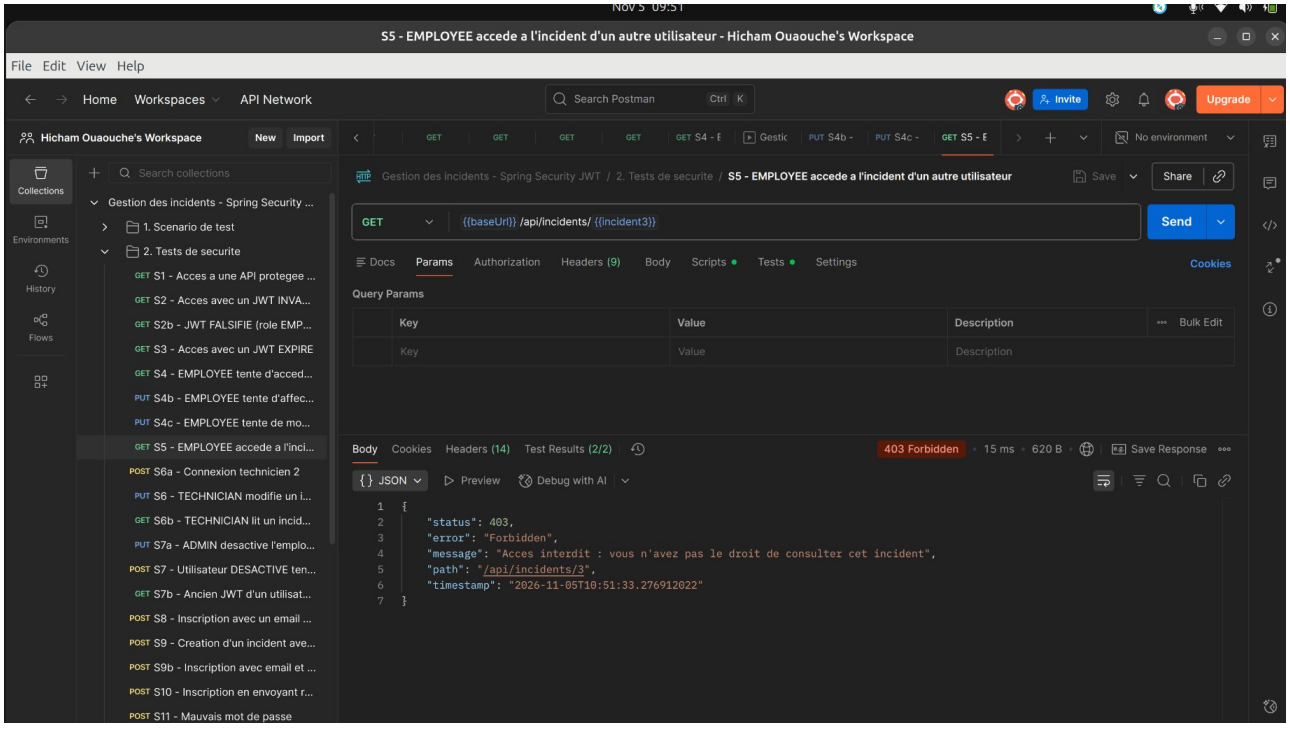


Figure 45 : *Postman, test 5 : EMPLOYEE sur l'incident d'un autre utilisateur → 403 Forbidden*

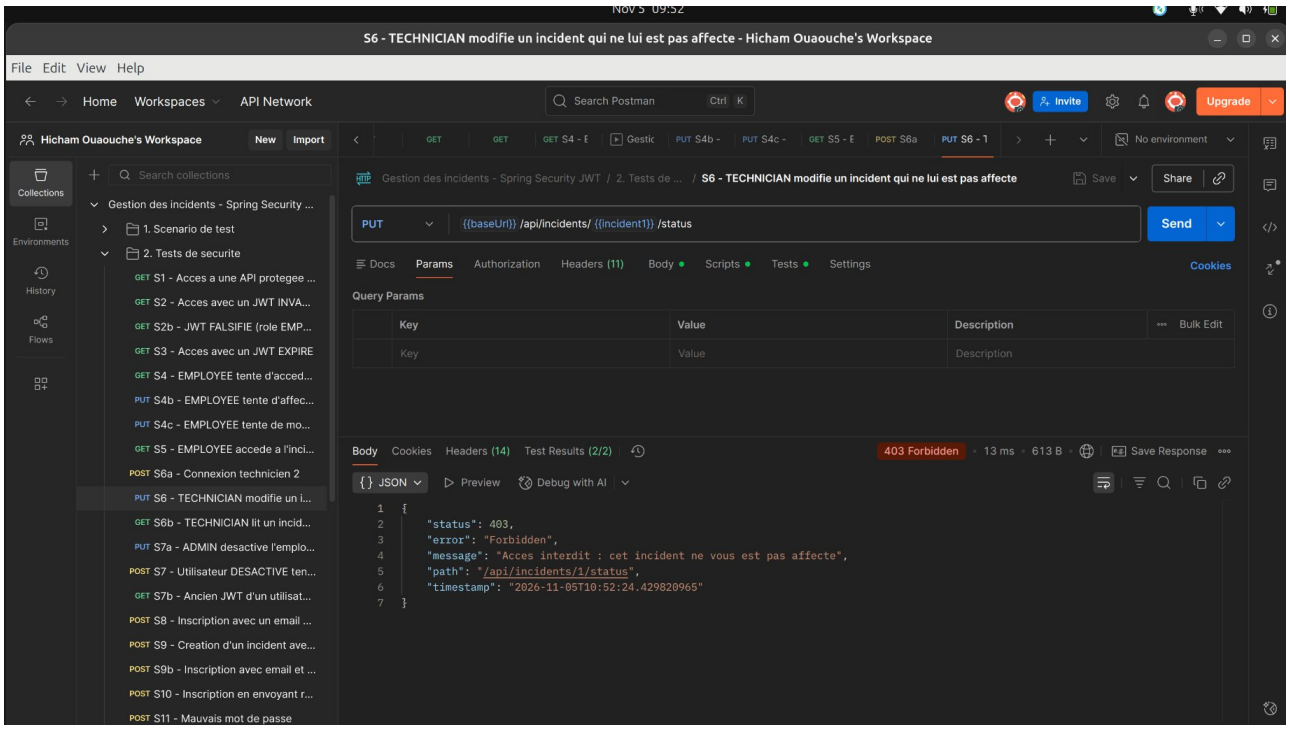


Figure 46 : *Postman, test 6 : TECHNICIAN modifie un incident qui ne lui est pas affecté → 403 Forbidden*

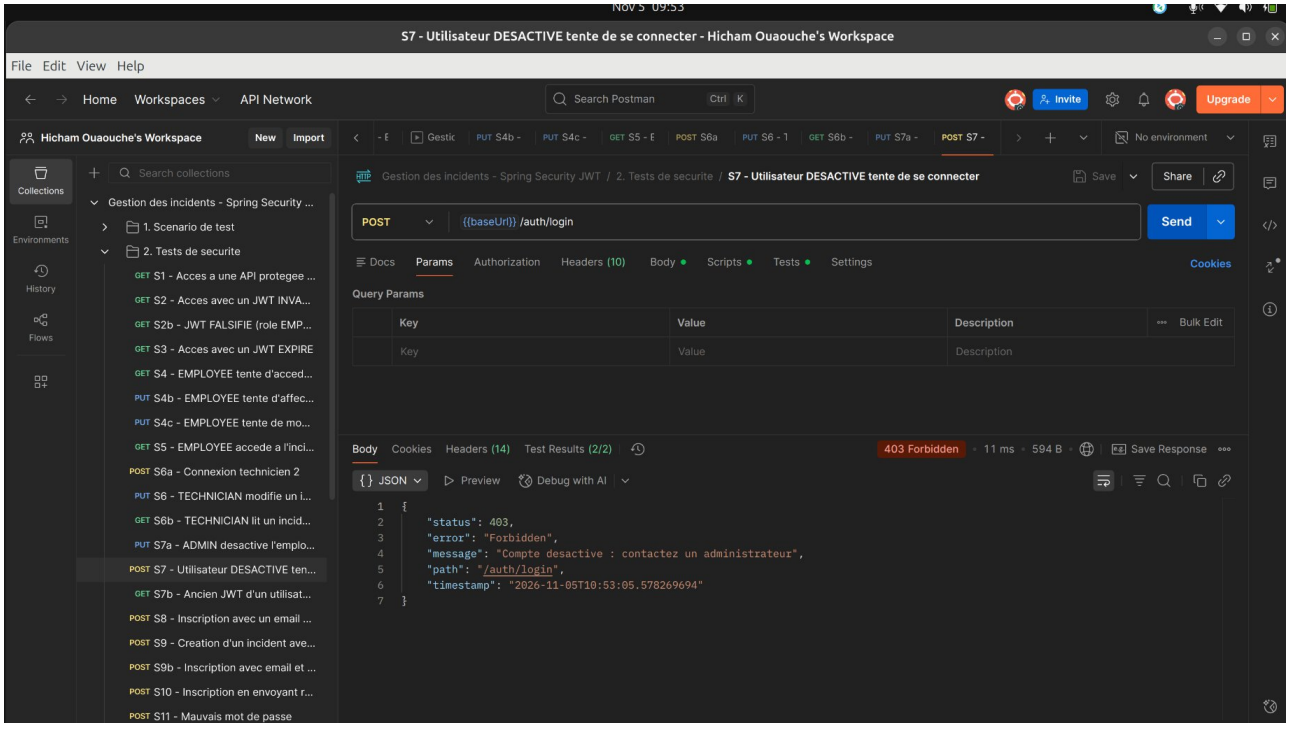


Figure 47 : *Postman, test 7 : utilisateur désactivé qui tente de se connecter → 403 « Compte desactive »*

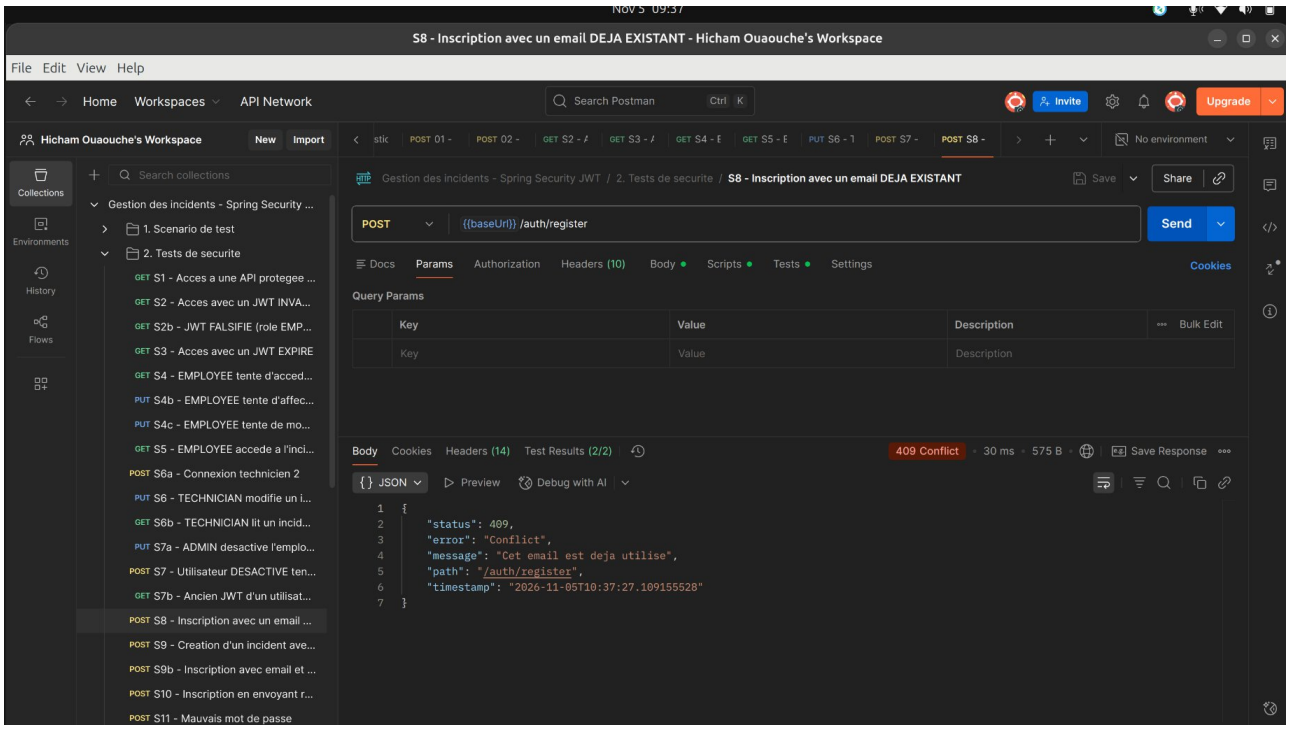


Figure 48 : *Postman, test 8 : inscription avec un email déjà existant → 409 Conflict*

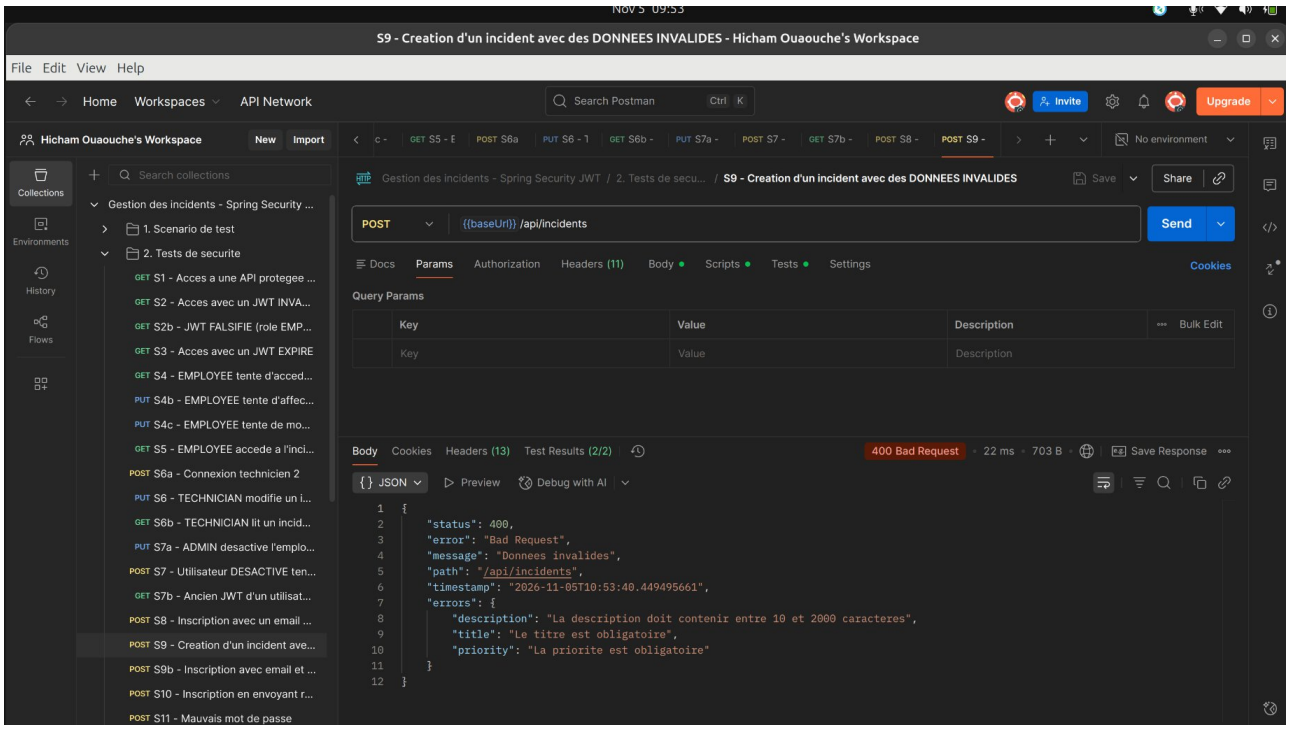


Figure 49 : *Postman, test 9 : création d'un incident avec des données invalides → 400 et détail des erreurs*

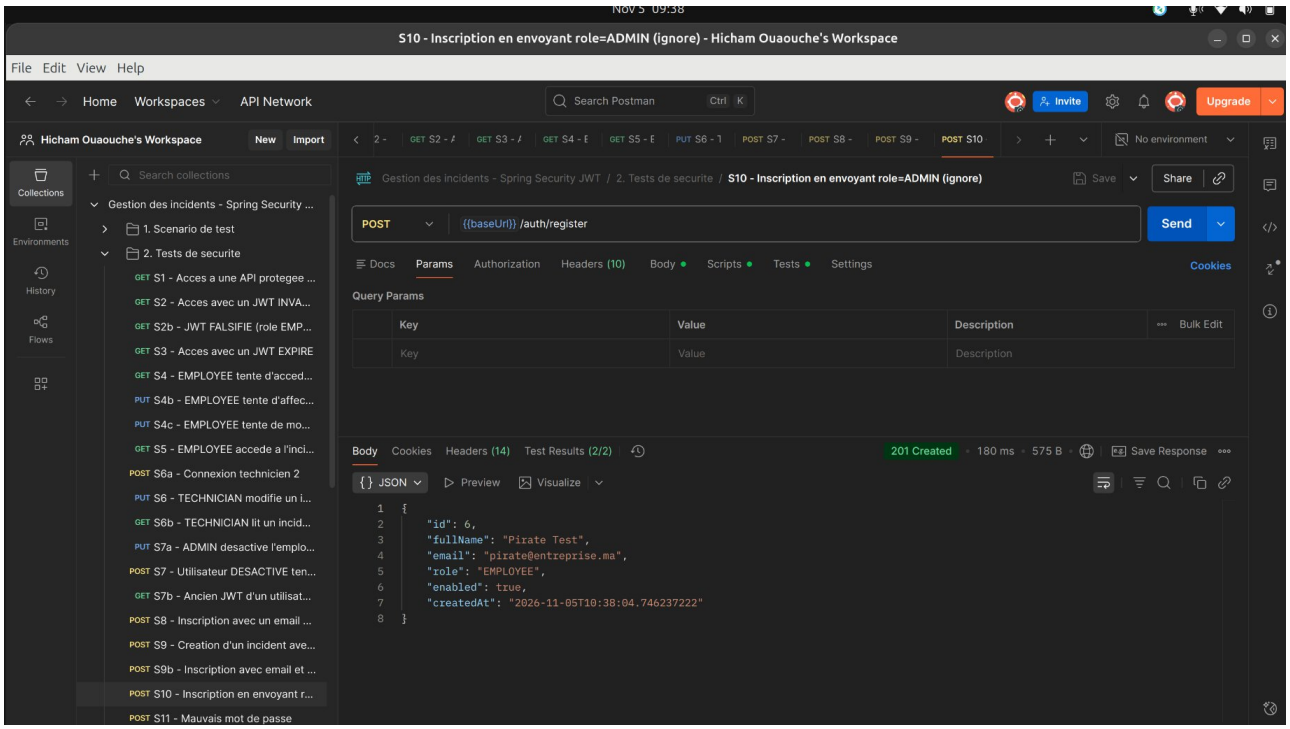


Figure 50 : *Postman, test complémentaire : inscription avec "role":"ADMIN" → le rôle reste EMPLOYEE*

9. Scénario de test et captures de l'application

Ce chapitre déroule le scénario de la section 24 du sujet dans l'interface React. Les captures ont été réalisées dans le navigateur Google Chrome (piloté par Puppeteer), sur l'application en fonctionnement (frontend sur http://localhost:5173, backend sur http://localhost:8085). La barre d'adresse de chaque capture montre l'URL réelle de la page.

Tableau 15 : *Correspondance scénario / preuves*

Étape du scénario	Réalisation	Preuve
1. Créer deux comptes EMPLOYEE	Inscriptions Ahmed Benali et Sara El Amrani	T05, N1
2. Créer au moins un compte TECHNICIAN	Youssef Amrani et Karim Idrissi (initialisation)	T02, N1
3. Créer un compte ADMIN	Hicham Ouaouche (initialisation)	T02, N1
4-5. Employé 1 : déclarer deux incidents	Malware (CRITICAL) et phishing (MEDIUM)	T09, N1, B01
6-7. Employé 2 : déclarer un incident	Panne réseau (HIGH)	N1
8. Employé 1 ne peut pas lire l'incident du 2	403 côté API et message dans l'UI	S5, B07
9-10. ADMIN consulte tous les incidents	Liste complète	N2, C02
11. ADMIN affecte un incident à un technicien	OPEN → ASSIGNED	S6a, C04
12-13. Le technicien consulte ses incidents	Liste des incidents affectés	N2, D02
14. OPEN → IN_PROGRESS puis IN_PROGRESS → RESOLVED	Boutons « Passer à … »	N2, D03
15. Ajouter une note de traitement	Notes de traitement	E1, D03
16-17. L'employé vérifie l'état et l'historique	Historique complet + notes	E2, B10

9.1 Pages publiques

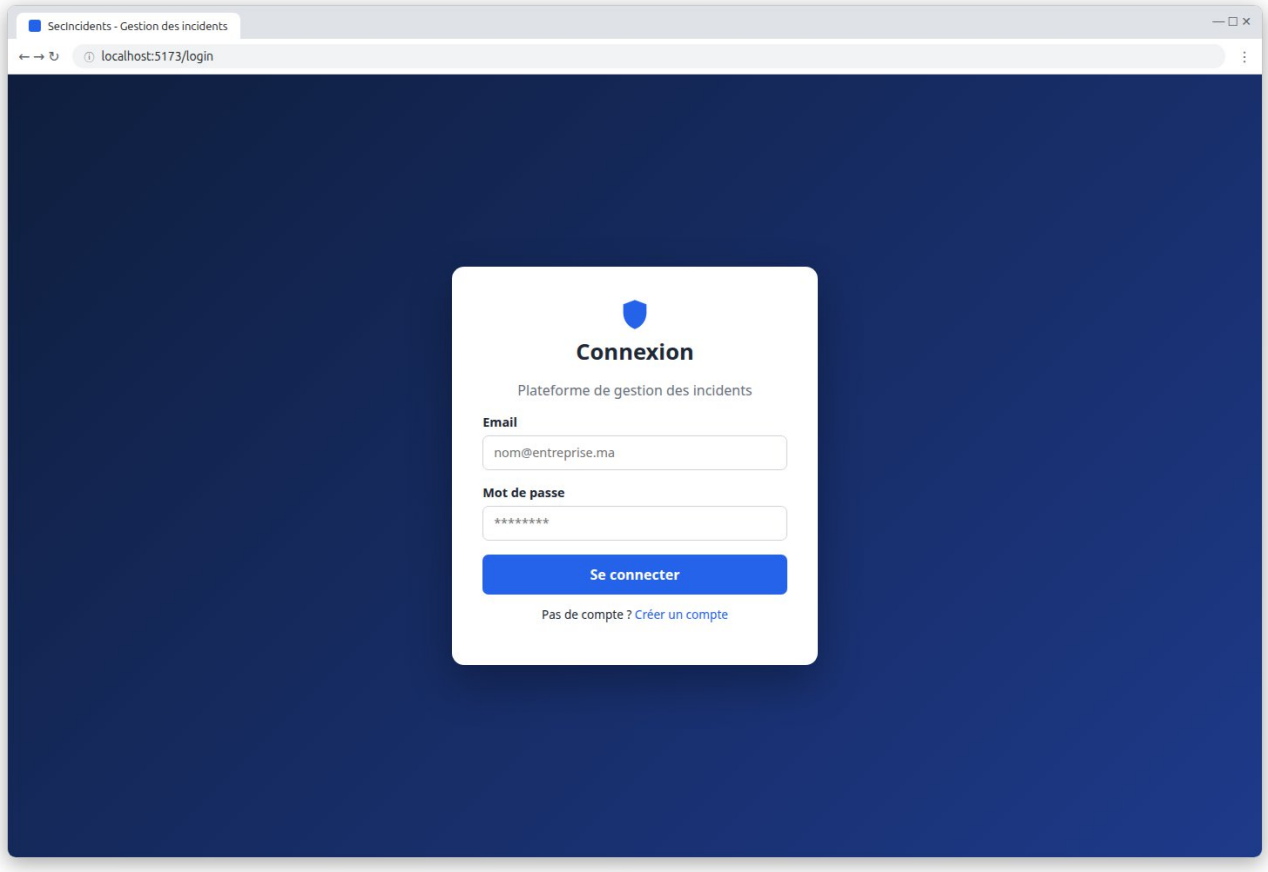


Figure 51 : *Page de connexion (accès à /dashboard sans être connecté → redirection vers /login)*

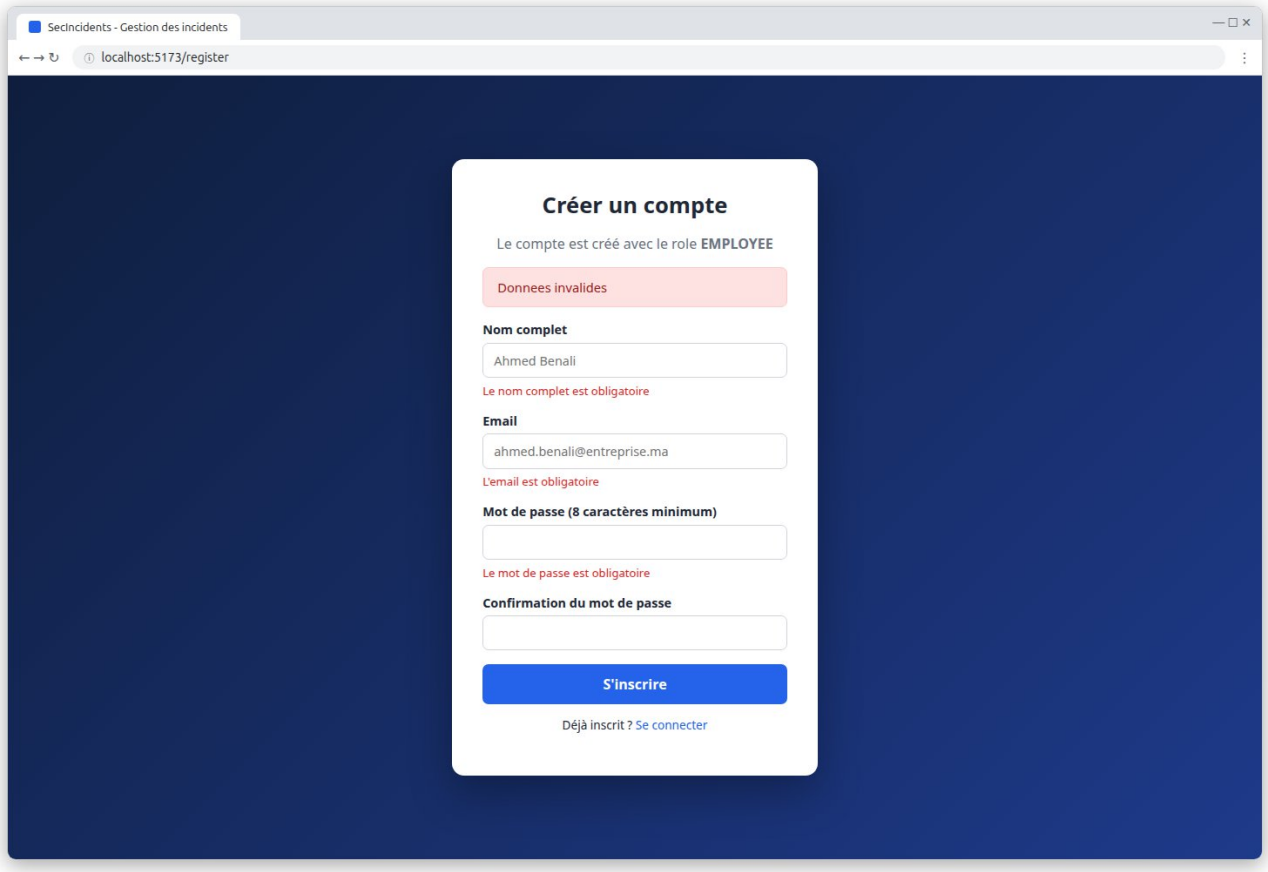


Figure 52 : *Inscription : erreurs de validation renvoyées par le backend et affichées sous chaque champ*

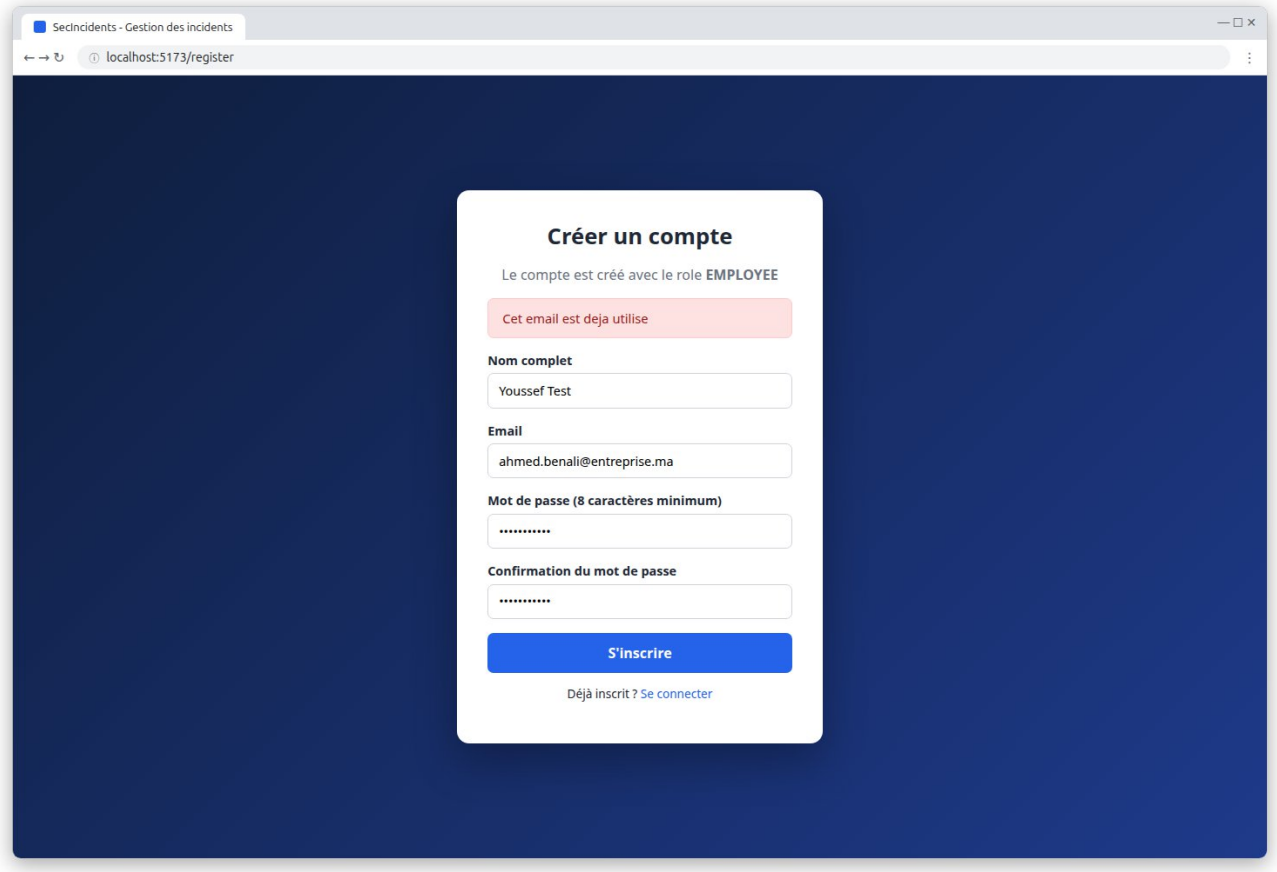


Figure 53 : *Inscription avec un email existant : message 409 du backend*

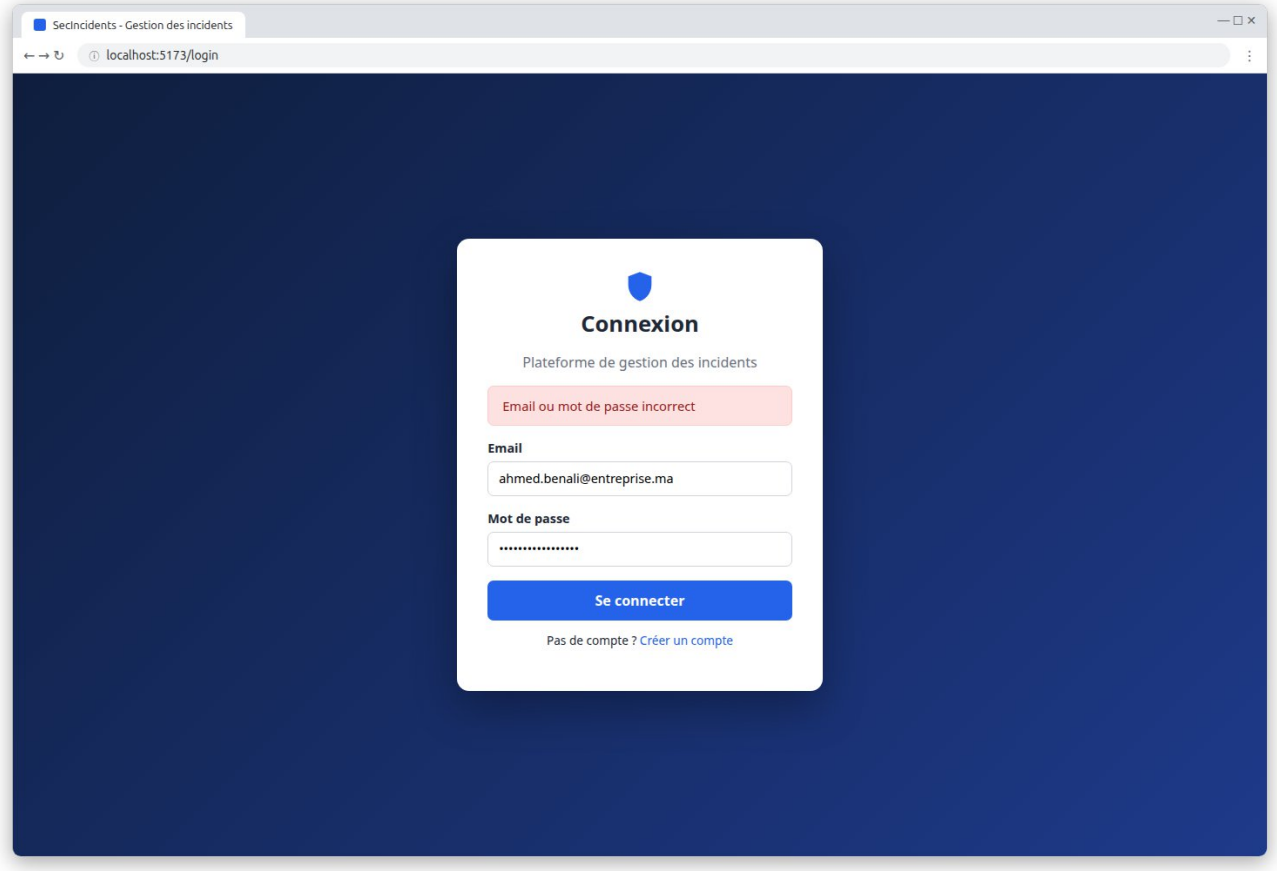


Figure 54 : *Connexion avec un mauvais mot de passe*

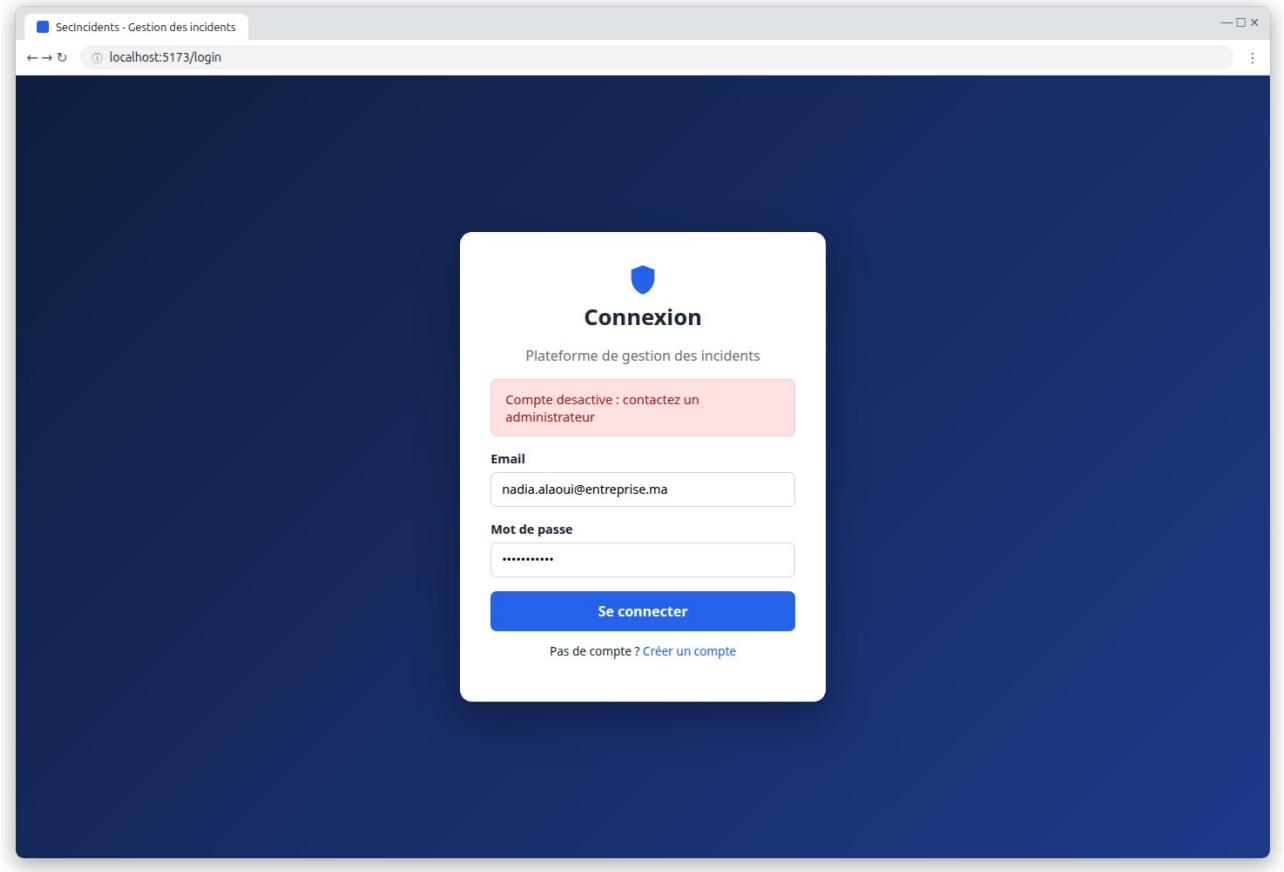


Figure 55 : *Connexion d'un compte désactivé par l'administrateur*

9.2 Employé (EMPLOYEE)

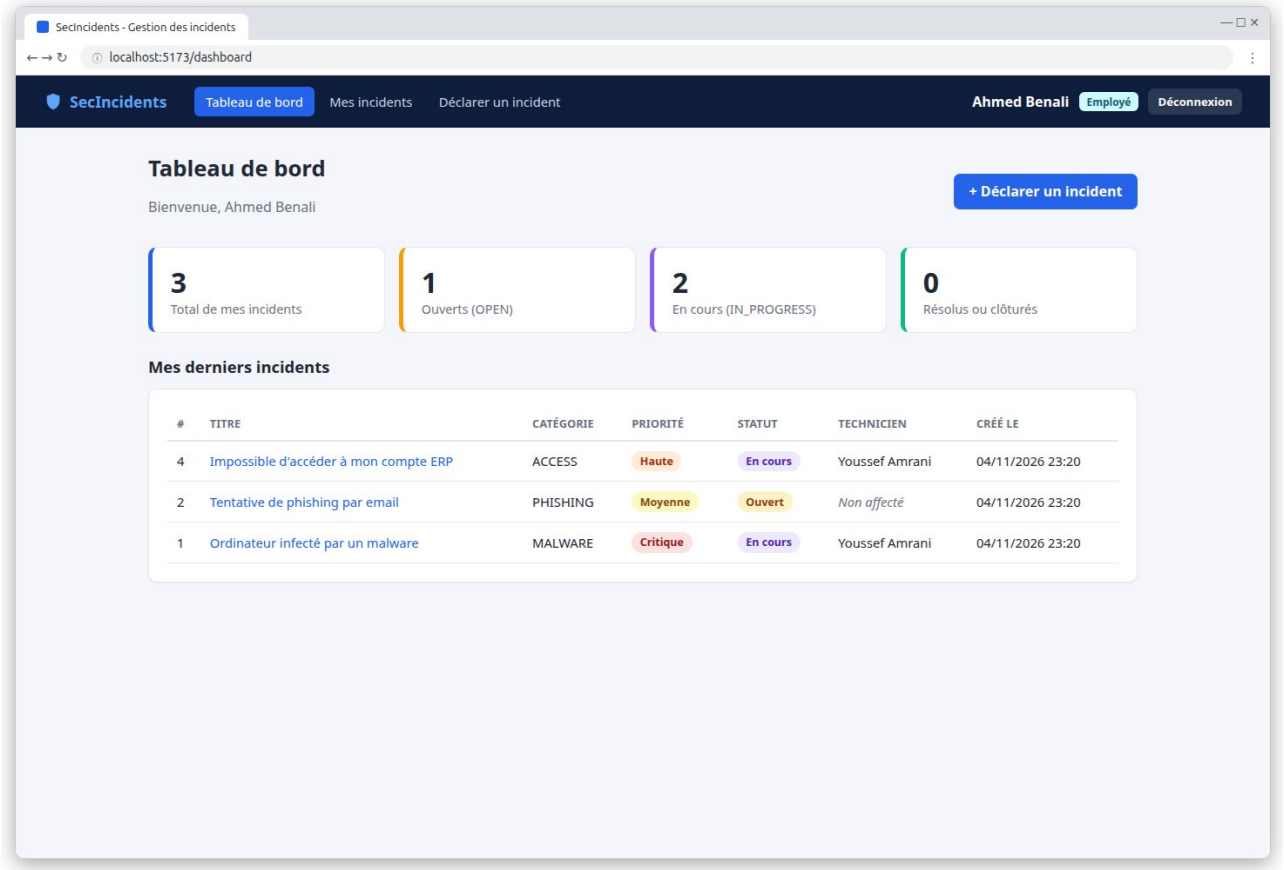


Figure 56 : *Dashboard EMPLOYEE : total, OPEN, IN_PROGRESS, RESOLVED/CLOSED et derniers incidents*

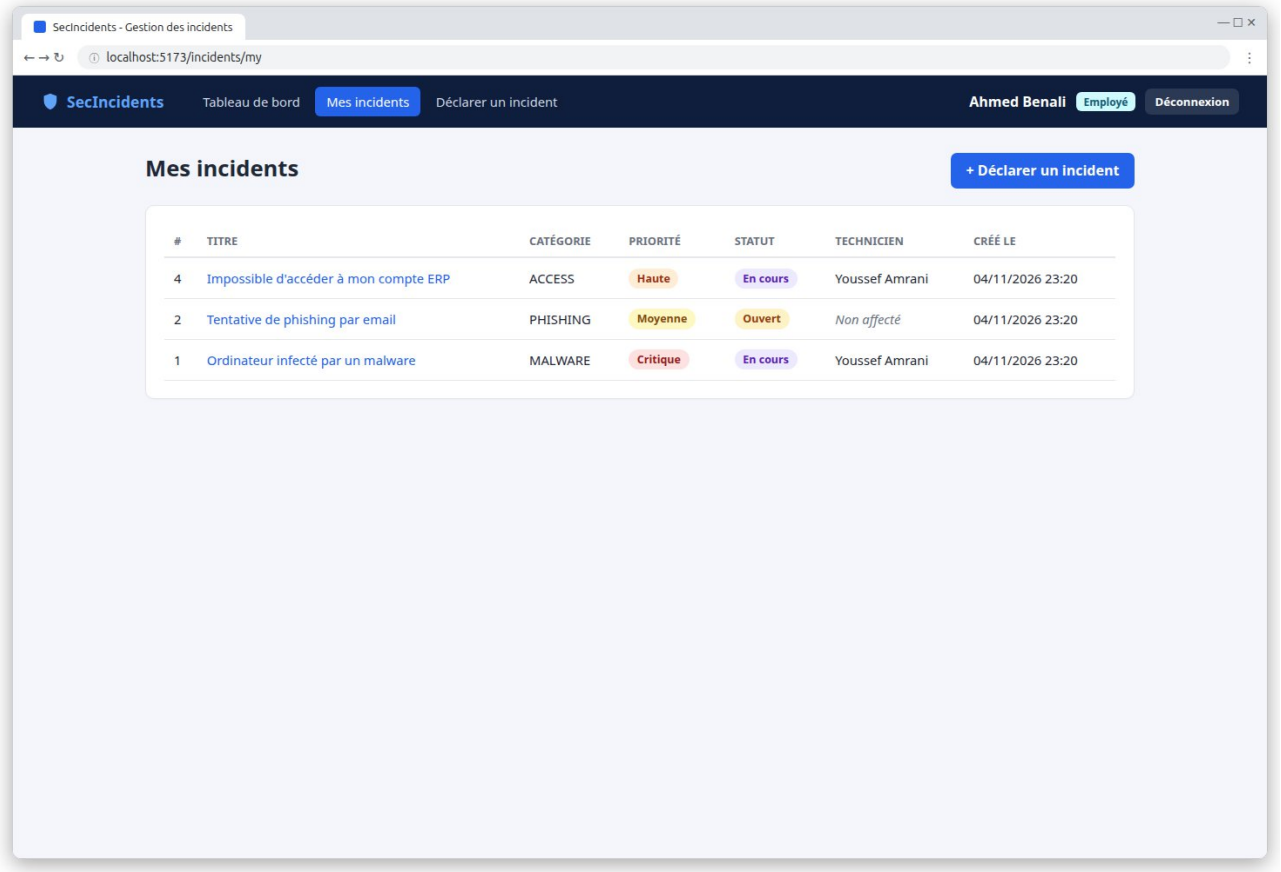


Figure 57 : *Mes incidents*

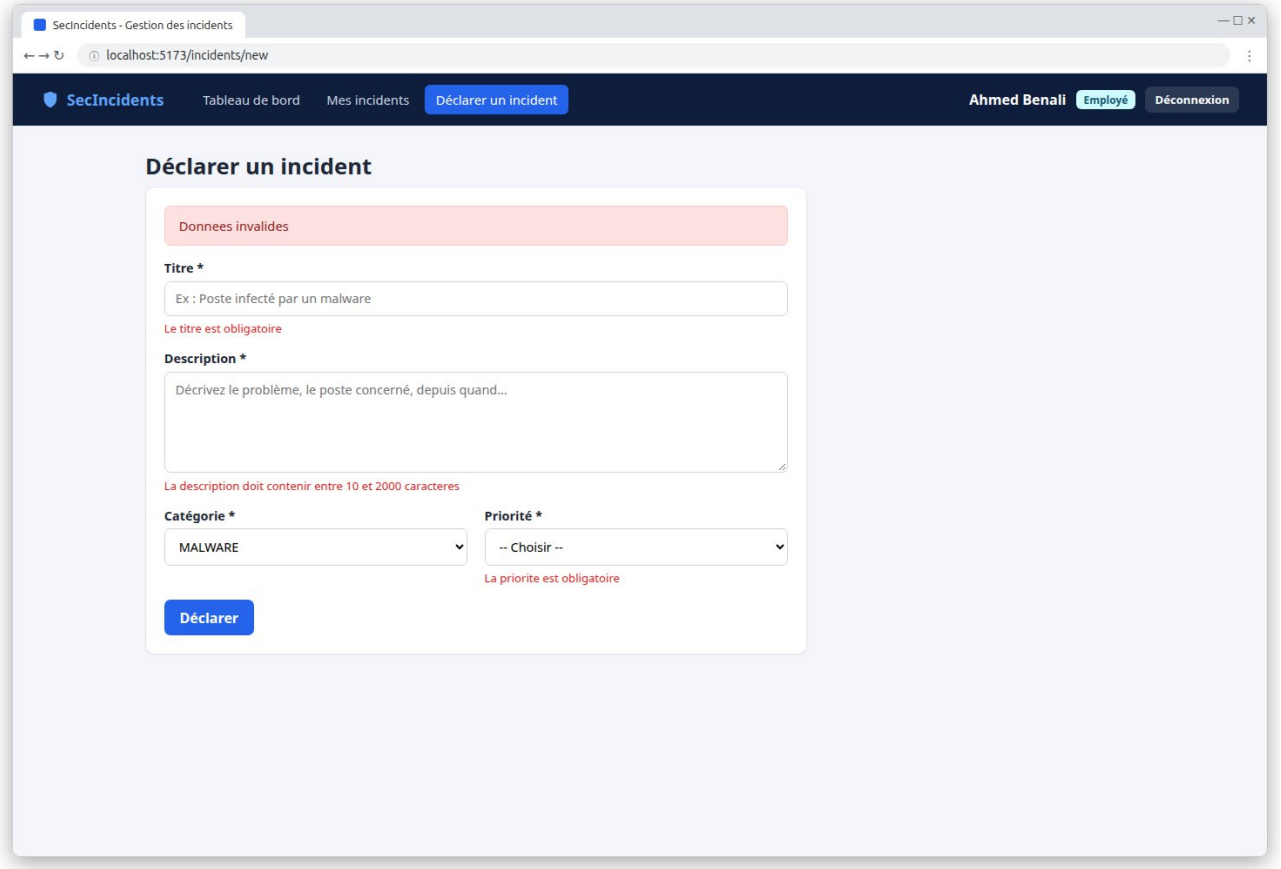


Figure 58 : *Déclaration d'un incident : validation (titre, description, priorité obligatoires)*

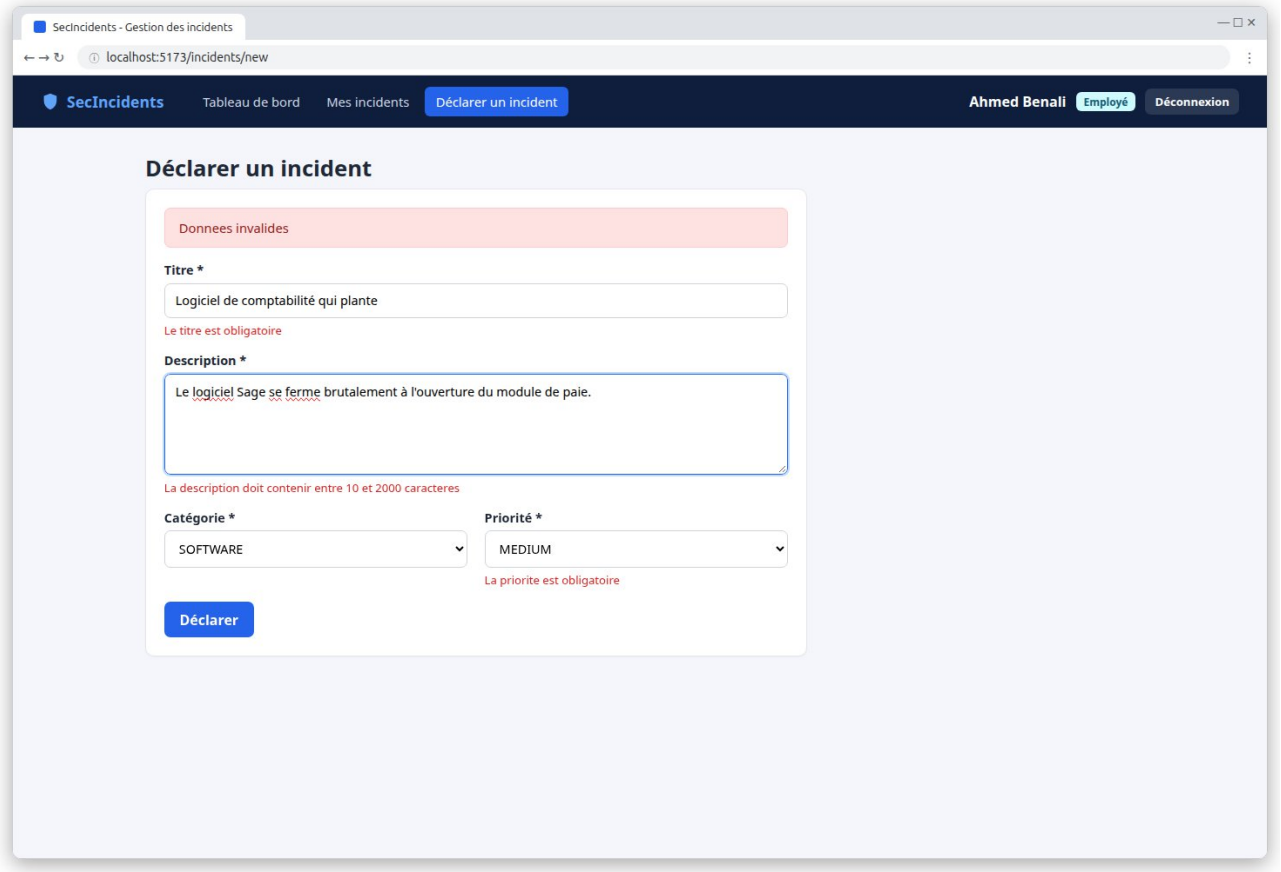


Figure 59 : *Formulaire de déclaration rempli*

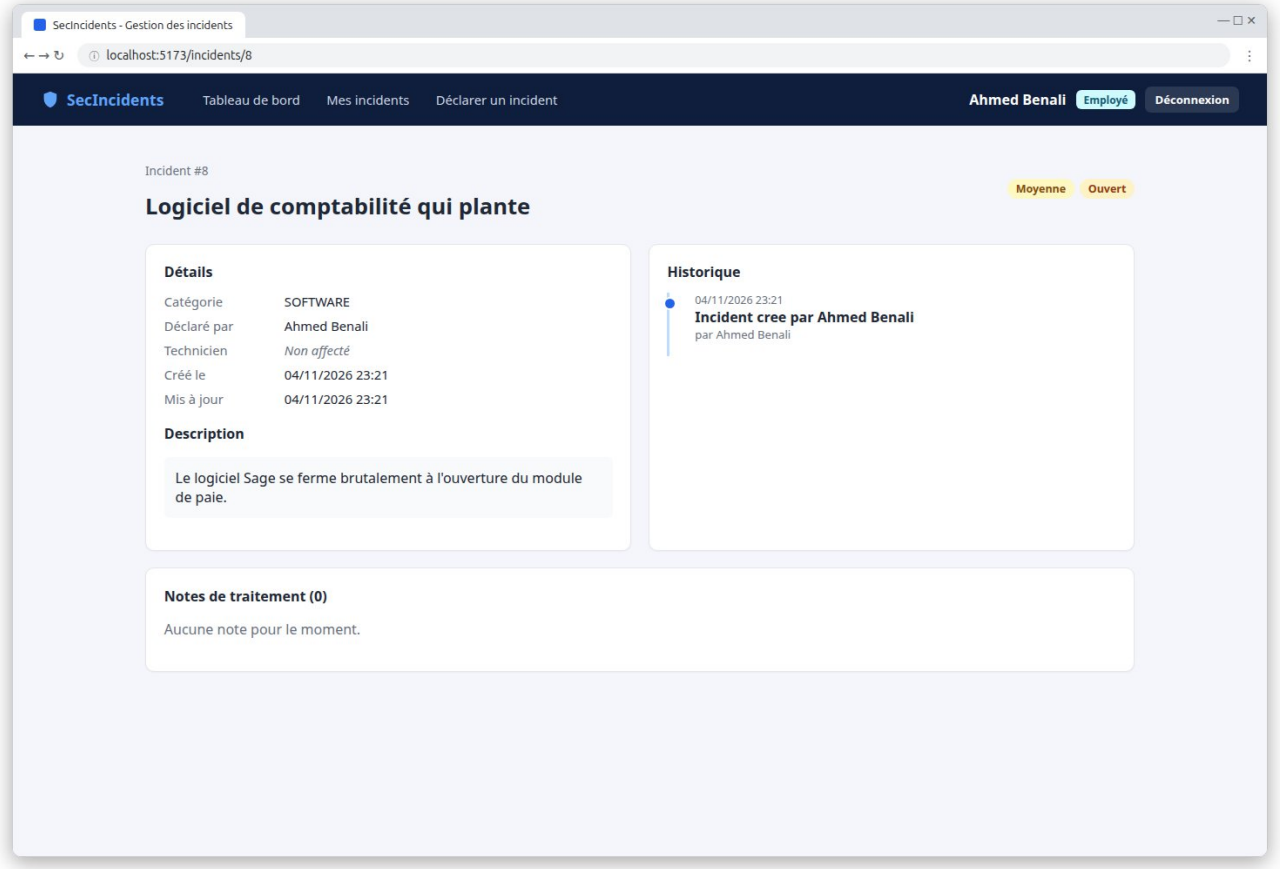


Figure 60 : *Incident créé : détail et première entrée d'historique*

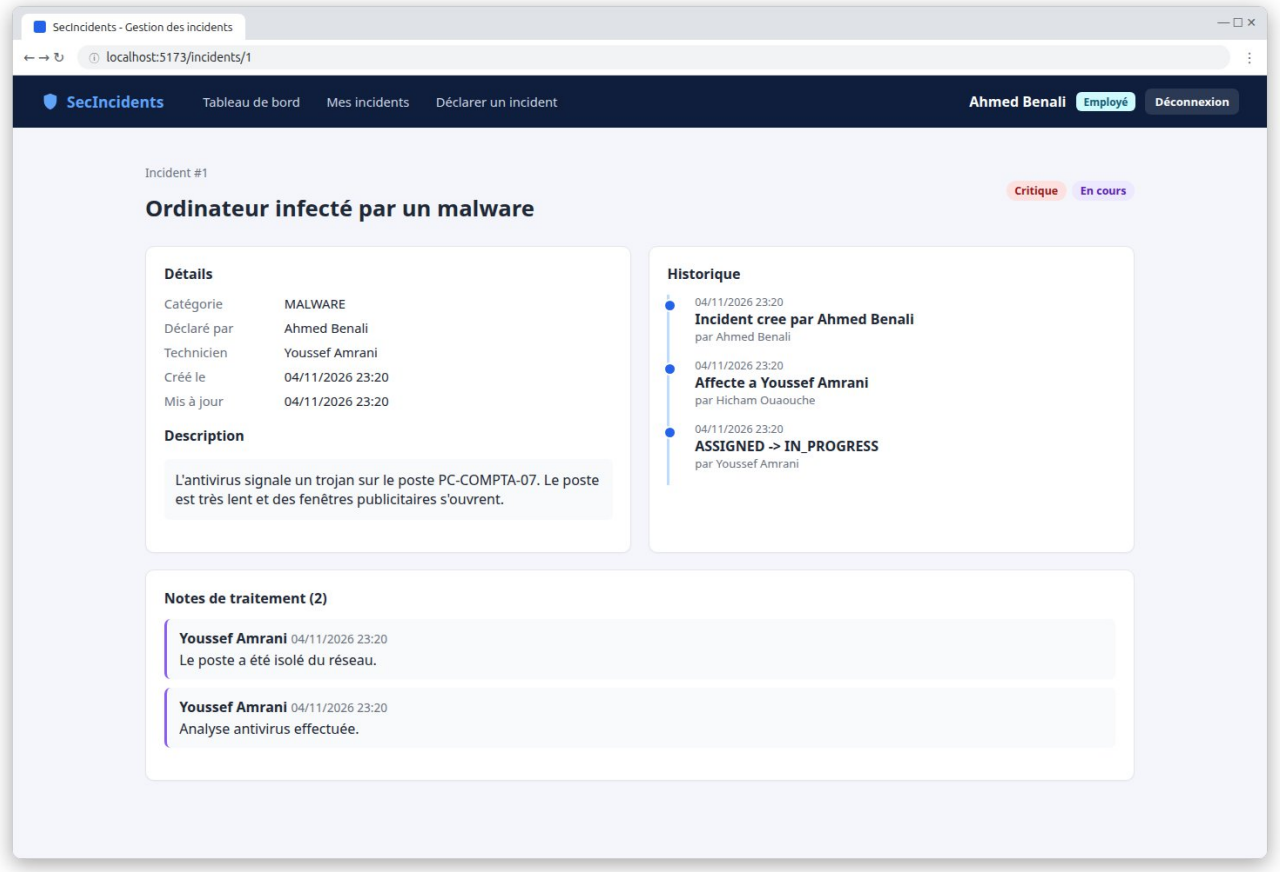


Figure 61 : *Détail d'un incident en cours de traitement, vu par son propriétaire*

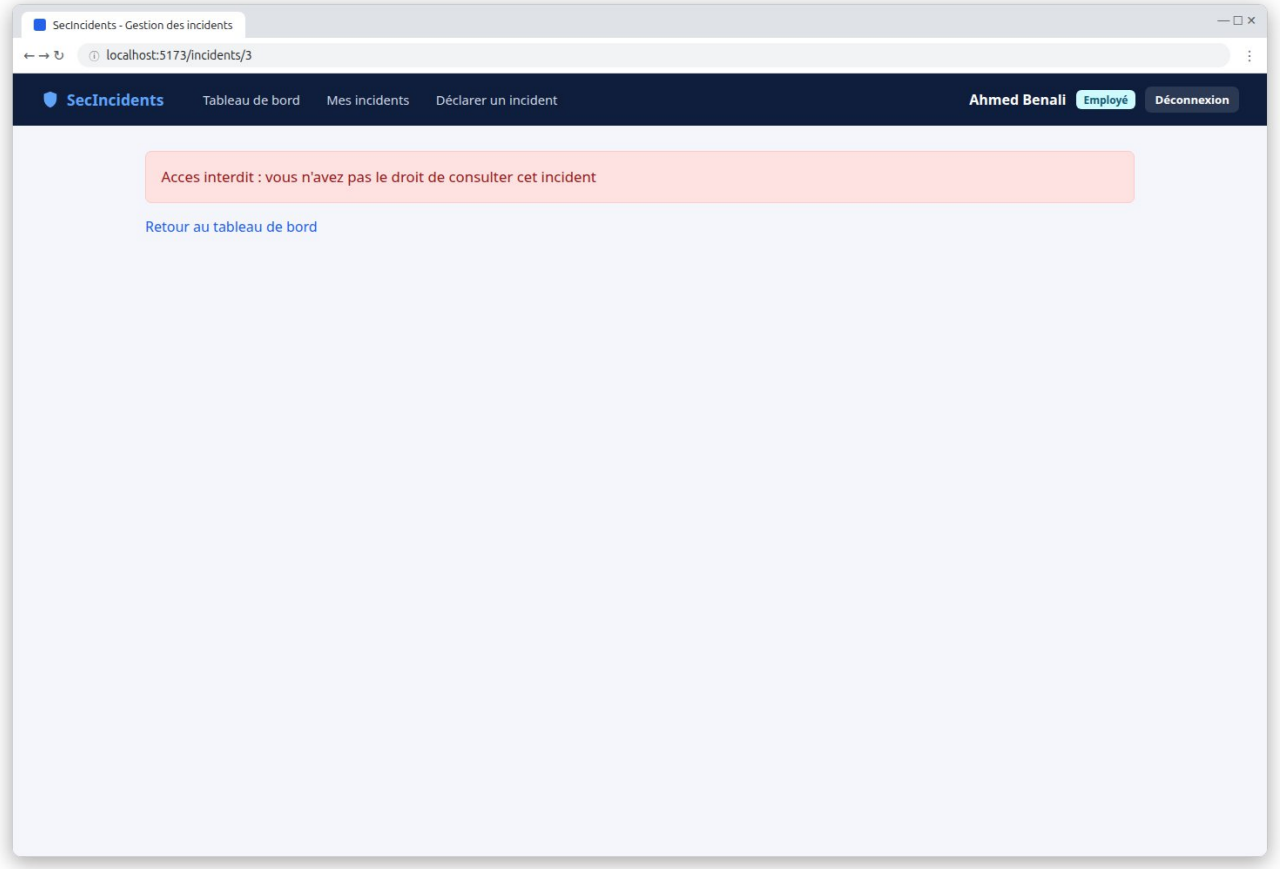


Figure 62 : *Accès direct par URL à l'incident d'un autre employé : refus du backend (403)*

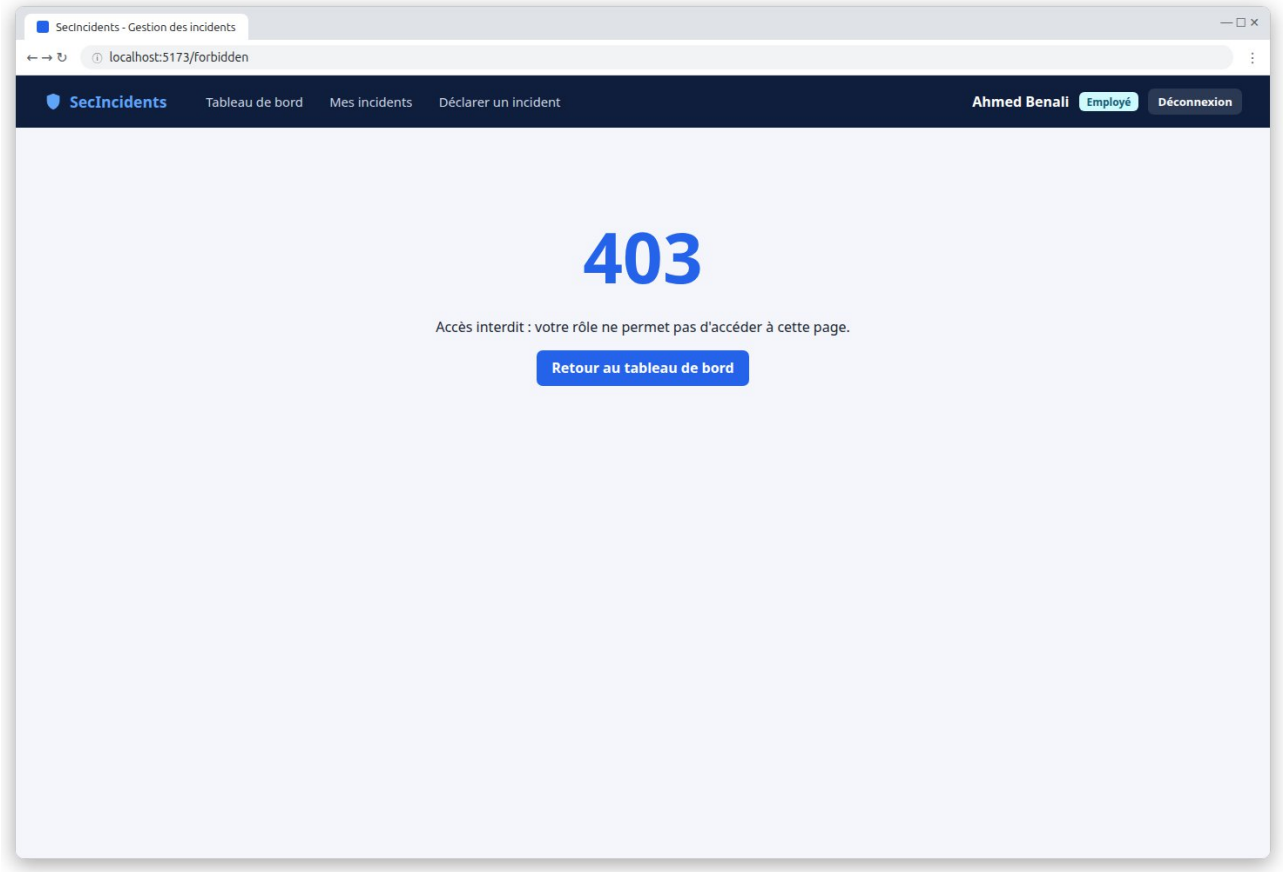


Figure 63 : *Accès à /admin/users avec le rôle EMPLOYEE : RoleRoute redirige vers la page 403*

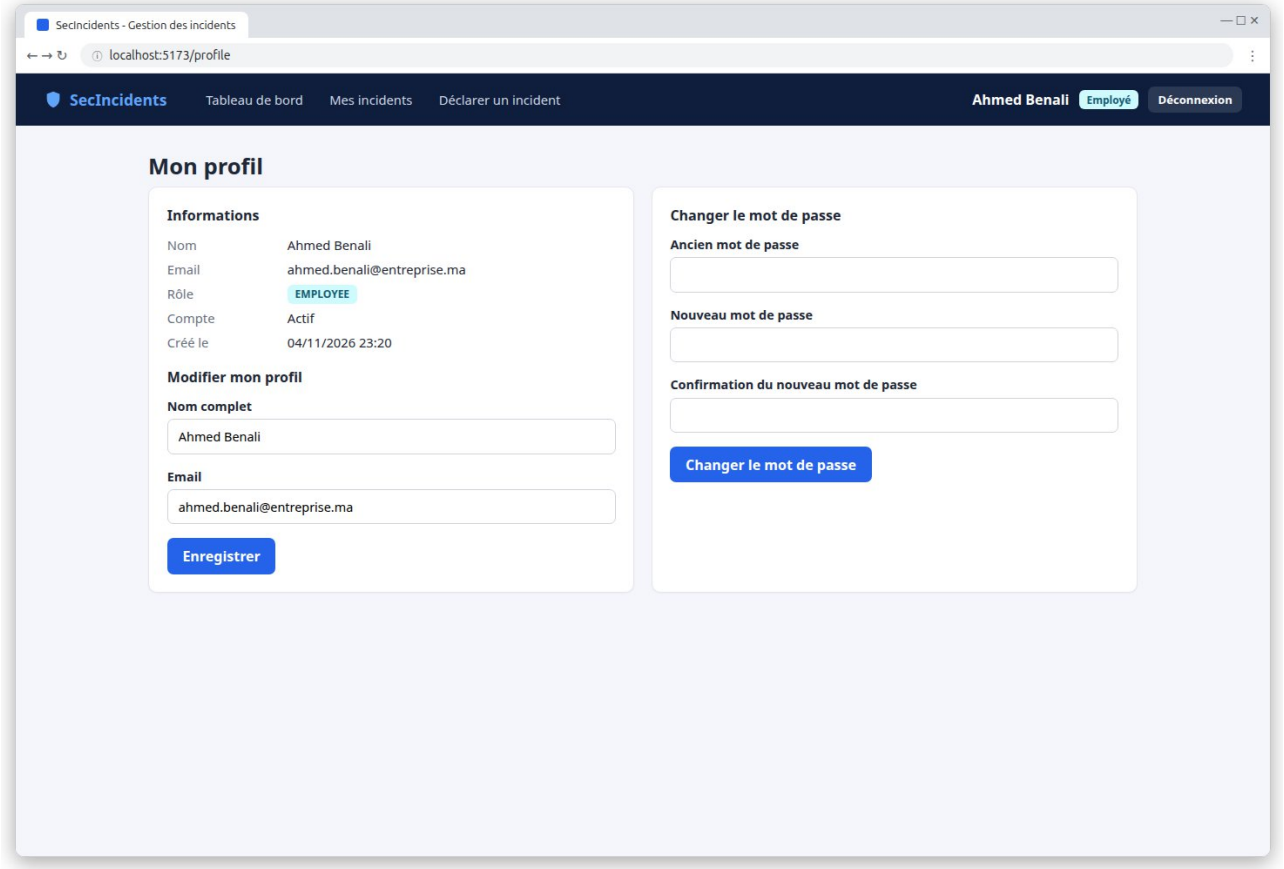


Figure 64 : *Profil : modification du nom et de l'email, changement de mot de passe*

9.3 Administrateur (ADMIN)

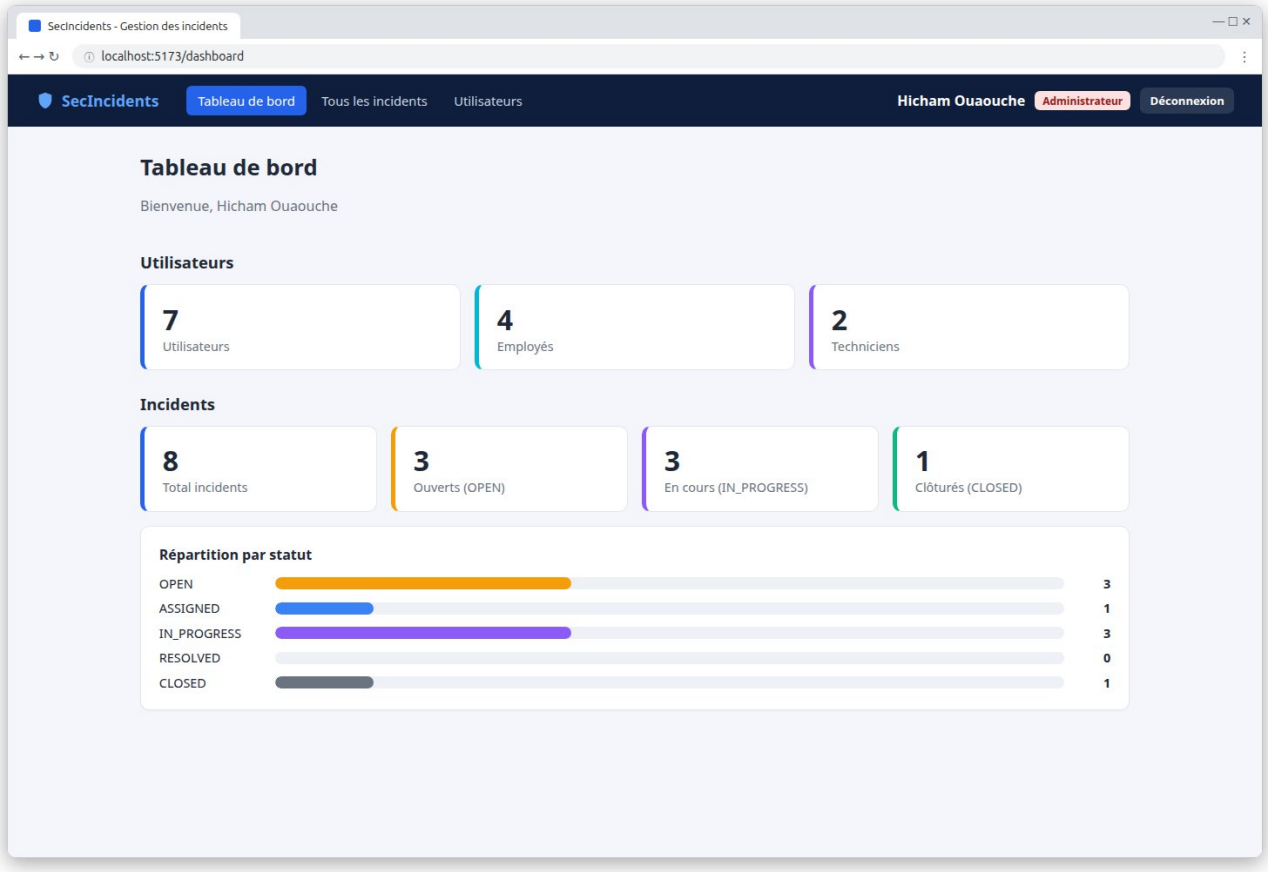


Figure 65 : *Dashboard ADMIN : utilisateurs, employés, techniciens, incidents par statut*

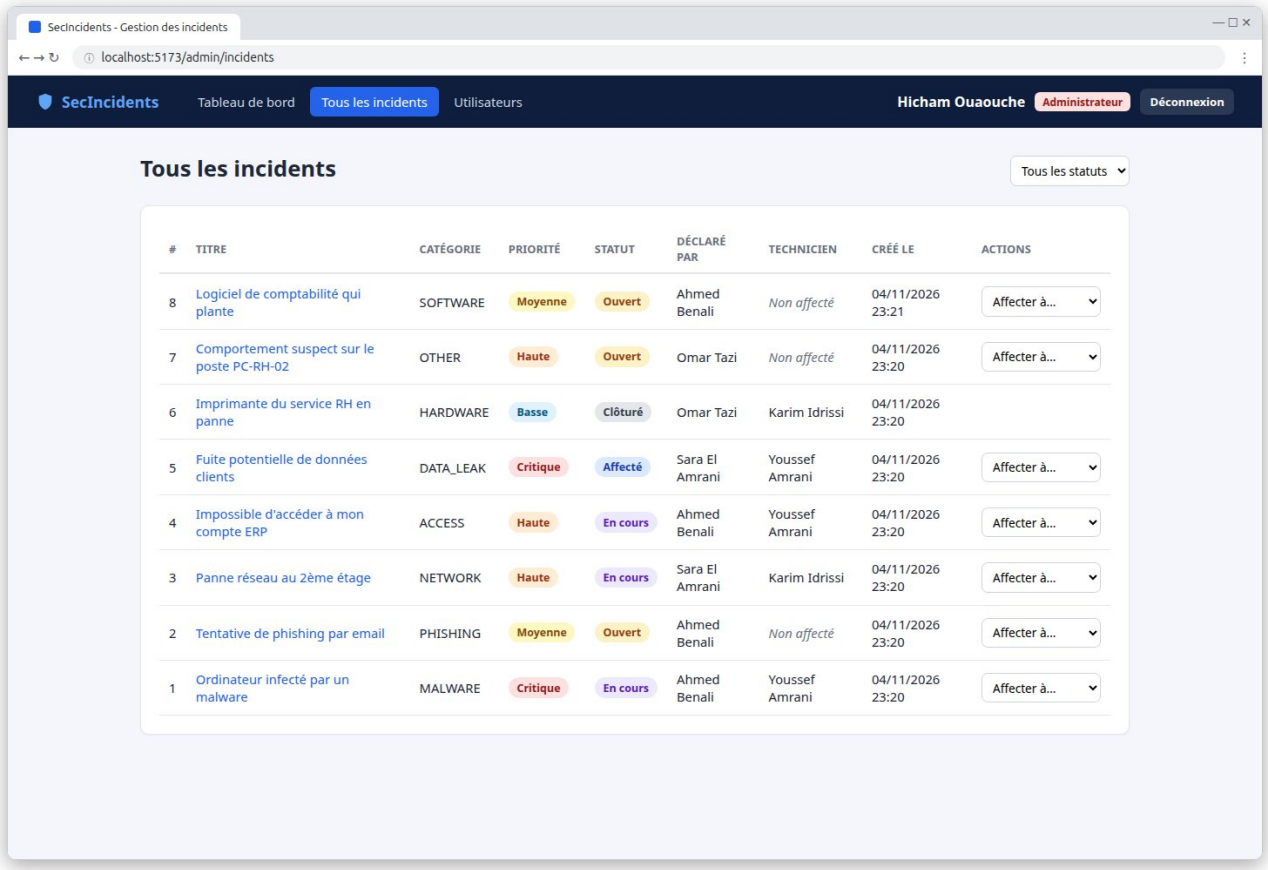


Figure 66 : *Tous les incidents, avec filtre par statut et affectation rapide*

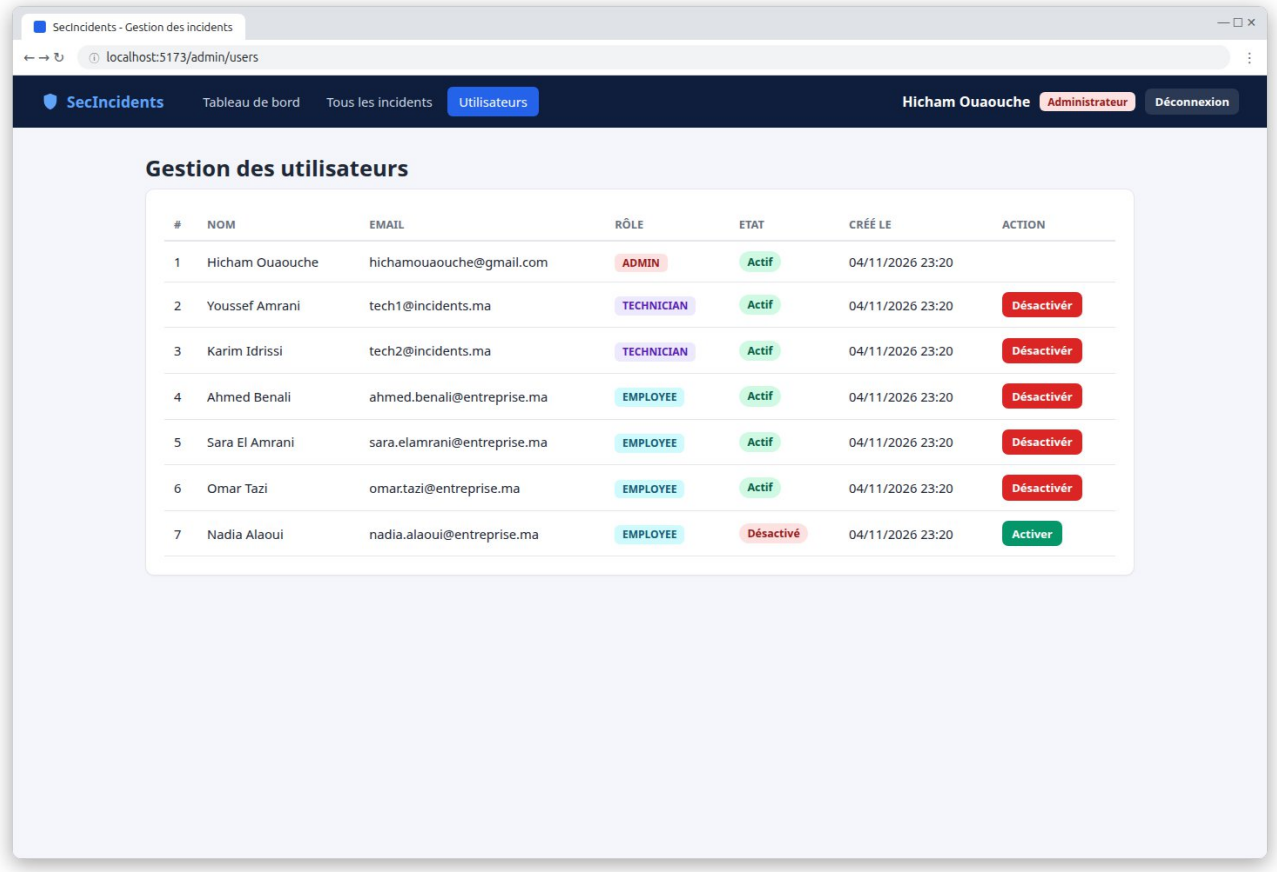


Figure 67 : *Gestion des utilisateurs : activation et désactivation*

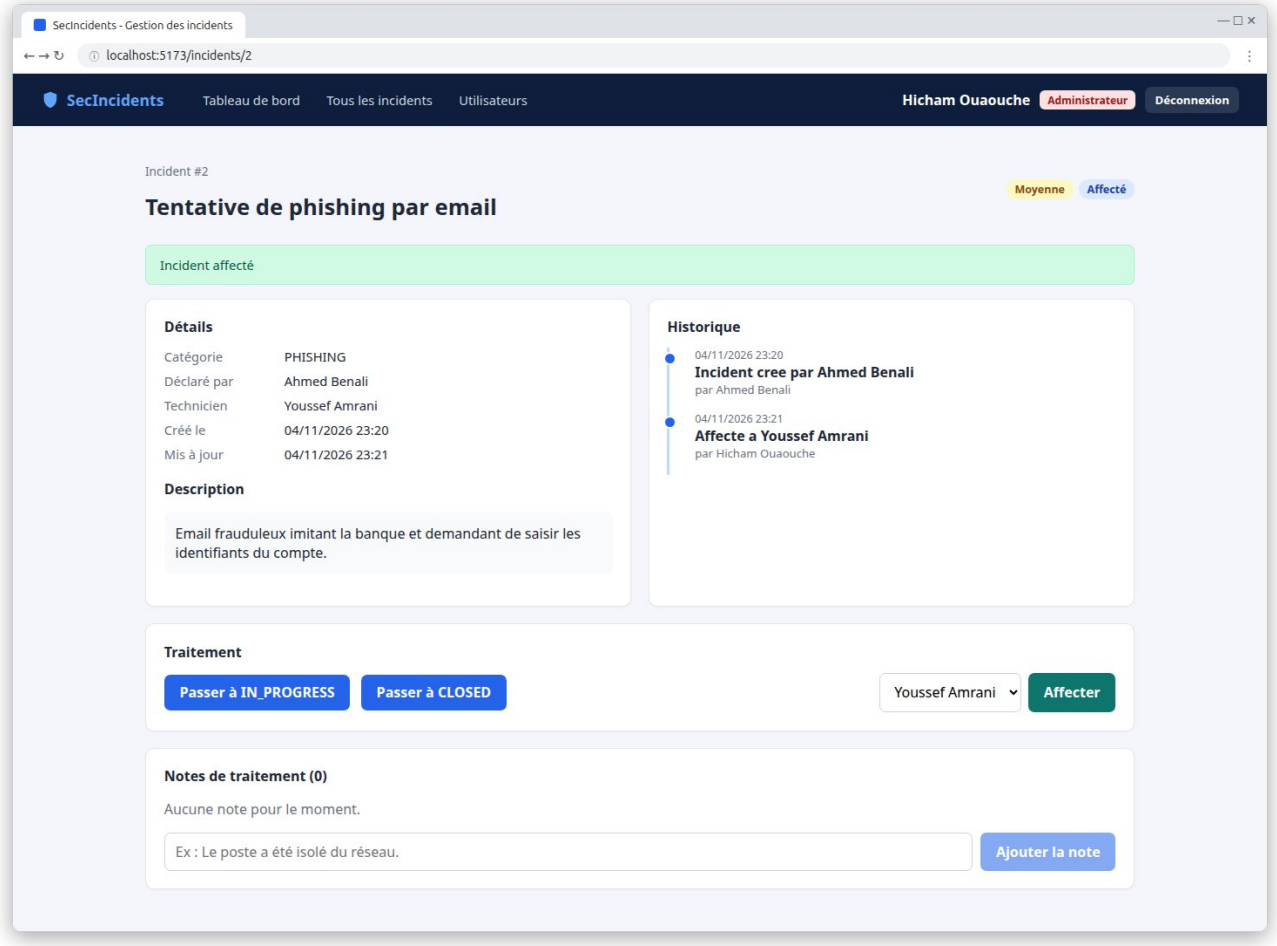


Figure 68 : *Affectation d'un incident à un technicien depuis la page de détail*

9.4 Technicien (TECHNICIAN)

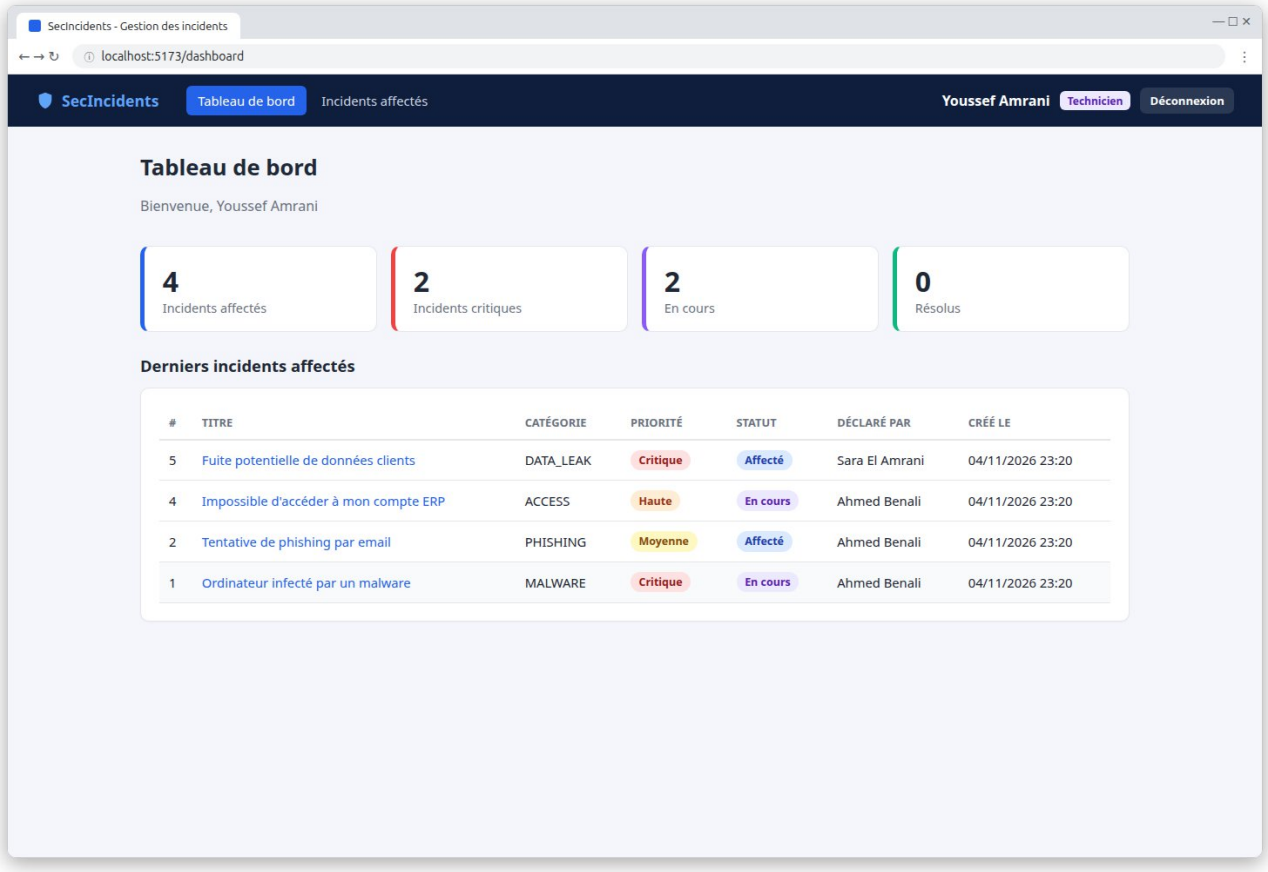


Figure 69 : *Dashboard TECHNICIAN : affectés, critiques, en cours, résolus*

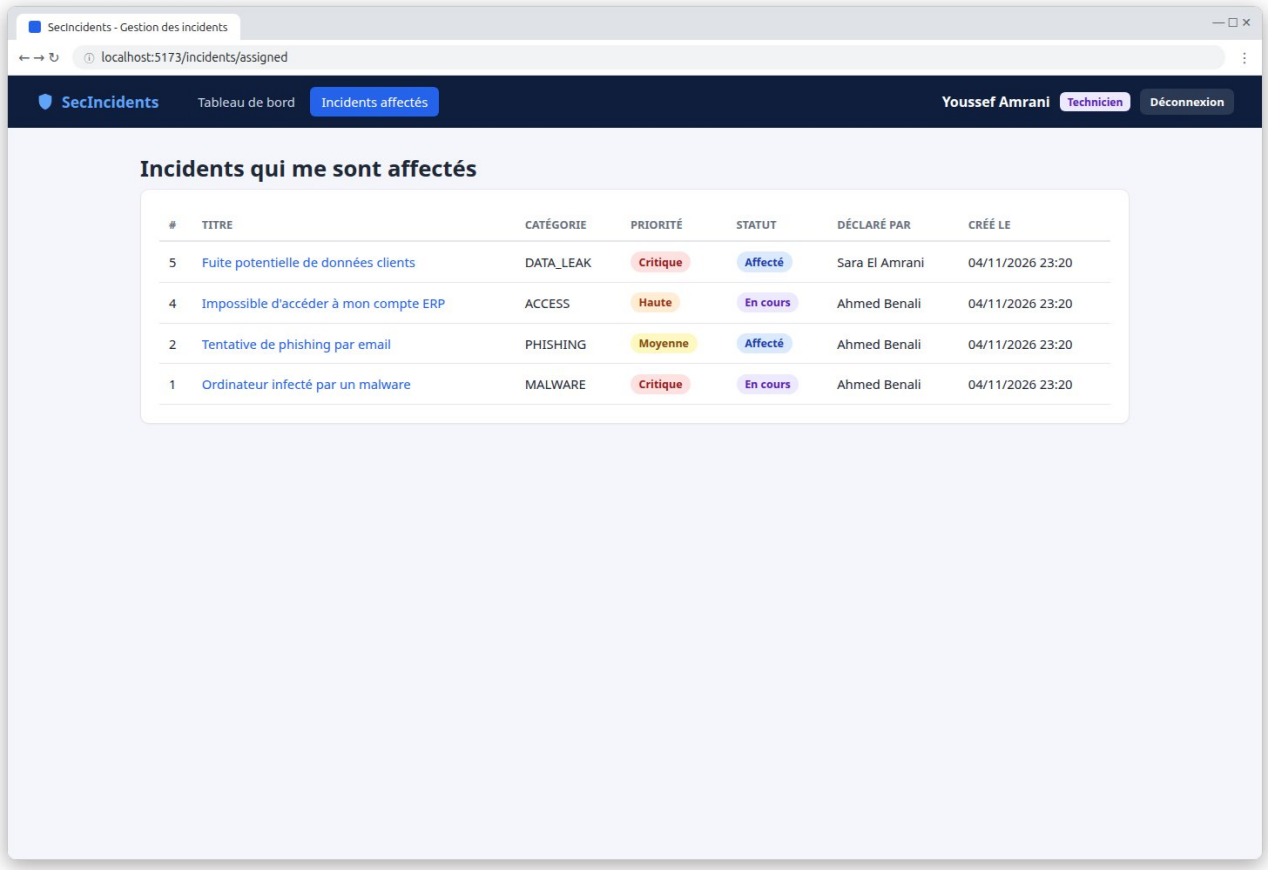


Figure 70 : *Incidents affectés au technicien*

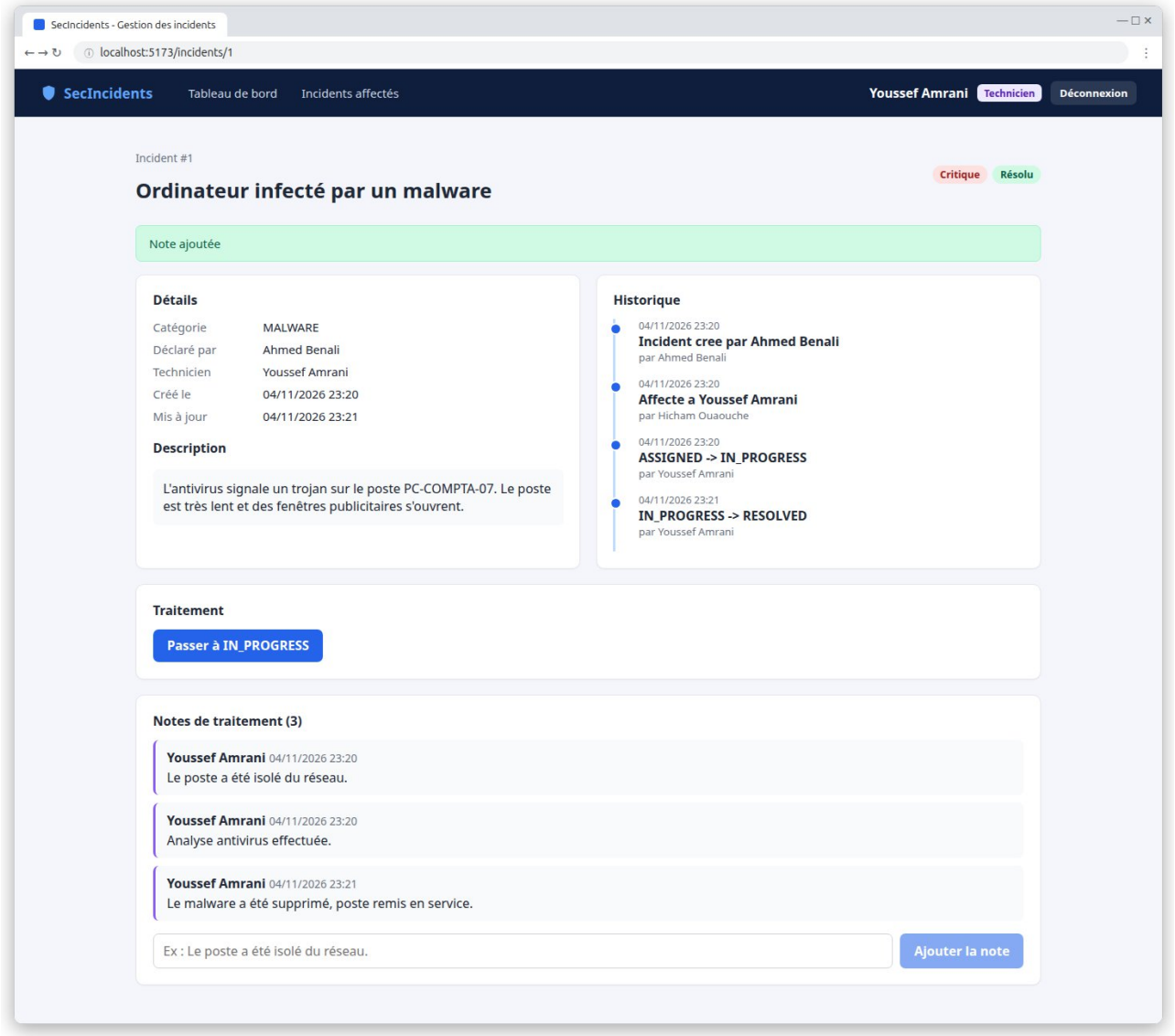


Figure 71 : *Traitement : passage à RESOLVED et ajout d'une note*

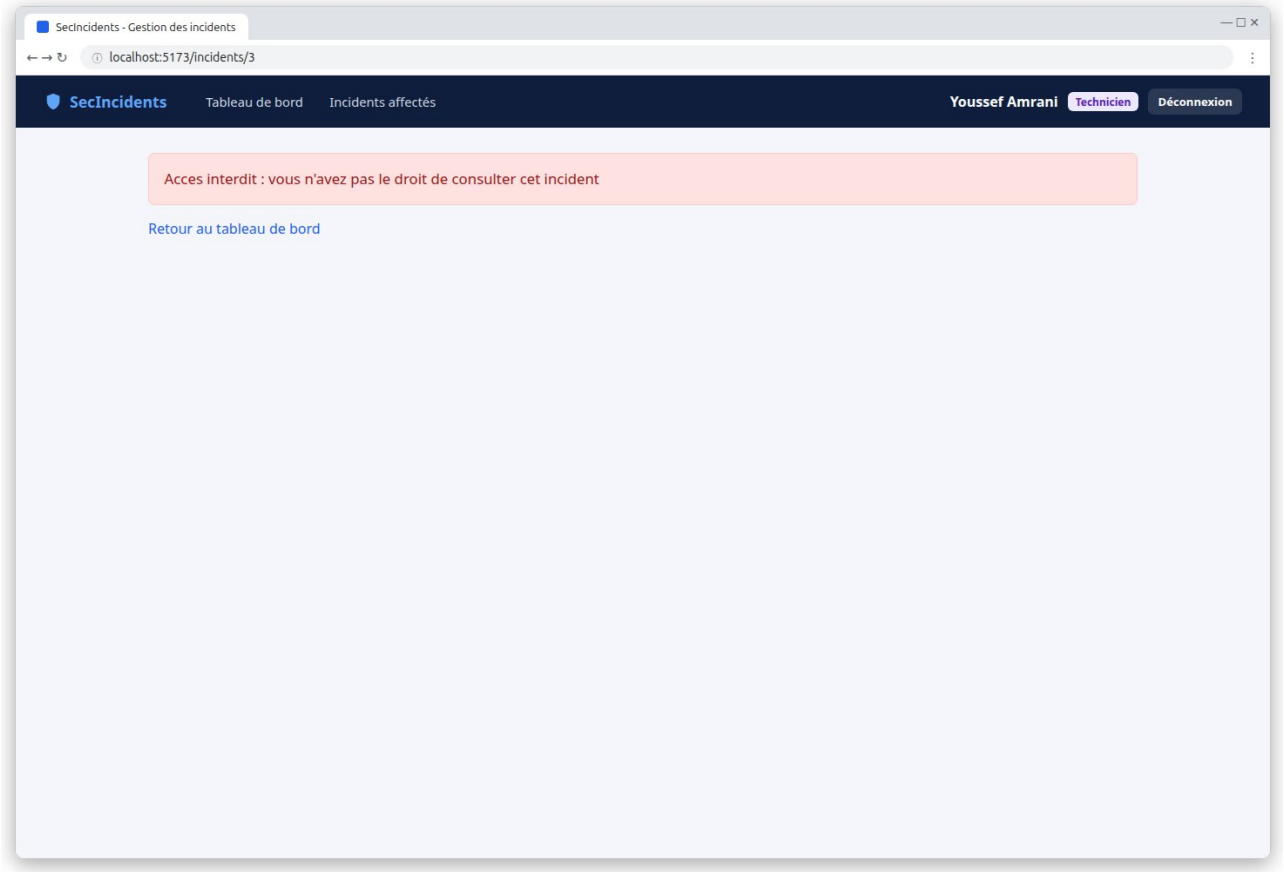


Figure 72 : *Un technicien ne peut pas consulter un incident affecté à un autre technicien*

9.5 Vérification finale et expiration de session

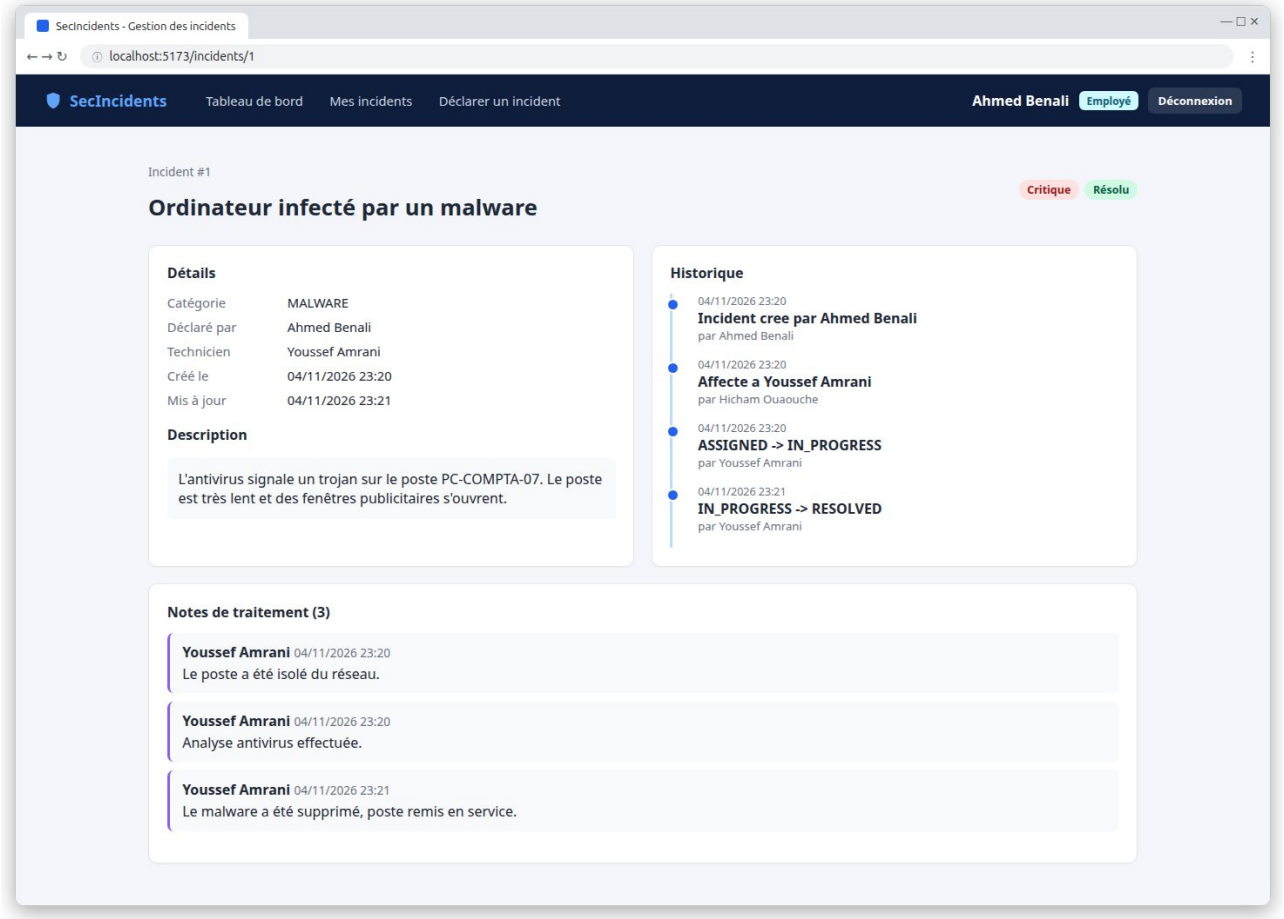


Figure 73 : *L'employé constate l'état RESOLVED, l'historique complet et les notes de traitement*

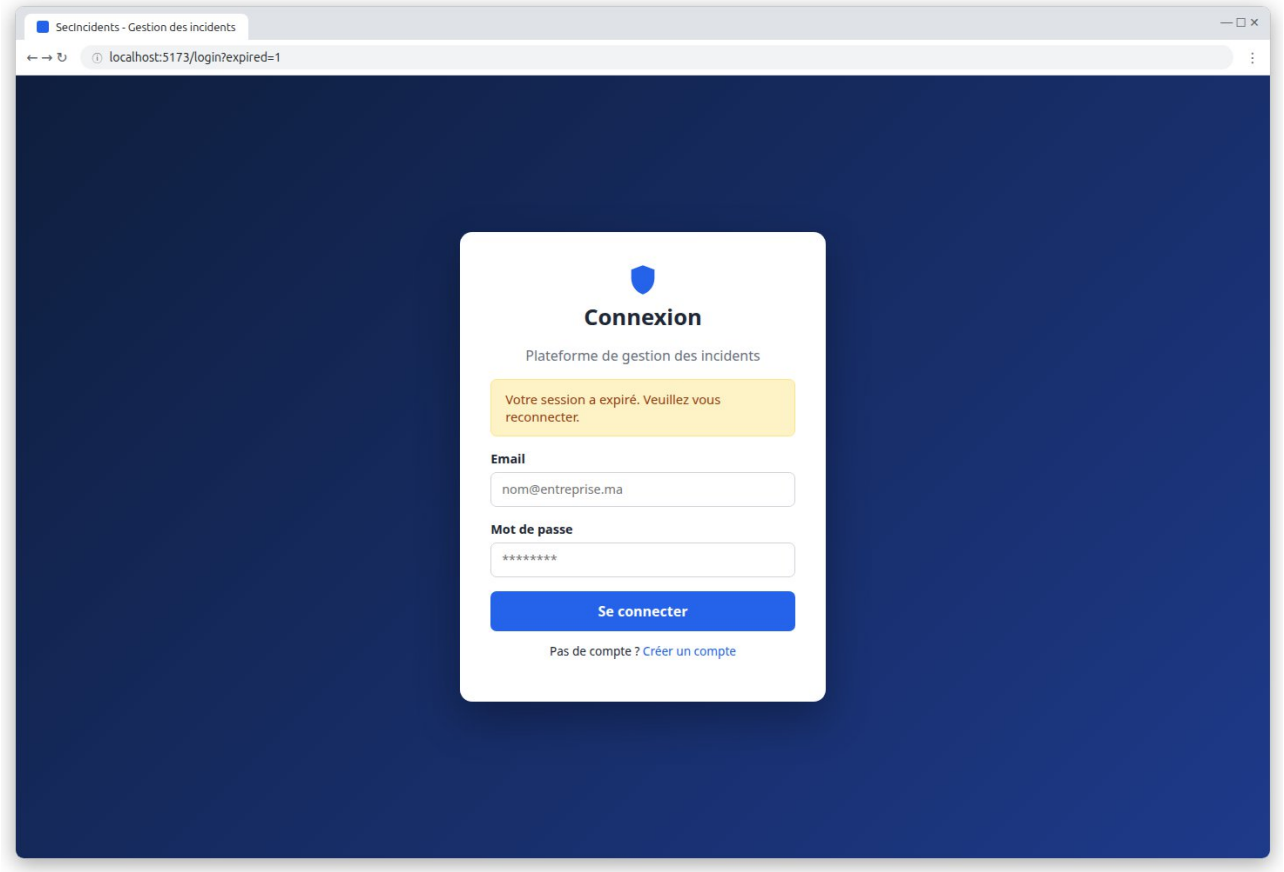


Figure 74 : *JWT expiré sans refresh token : JWT supprimé, redirection vers /login et message « session expirée »*

10. Bilan de sécurité

10.1 Mesures mises en place

Tableau 16 : Menaces et contre-mesures

Menace	Contre-mesure
Vol de mot de passe en base	Hachage BCrypt salé (coût 10) ; le mot de passe n'est jamais renvoyé par l'API
Falsification du token (élévation de privilèges)	Signature HMAC-SHA256 avec clé de 256 bits ; tout payload modifié est rejeté (testé)
Rejeu d'un token volé	Durée de vie courte (5 min) ; rotation et révocation du refresh token
Compte compromis	Désactivation par l'admin, effet immédiat (filtre qui recharge l'utilisateur, refresh tokens révoqués)
IDOR (accès direct par identifiant)	Contrôle du propriétaire / de l'affectation dans le service, pour chaque lecture et écriture
Escalade de rôle à l'inscription	Aucun champ role dans le DTO ; rôle EMPLOYEE imposé
Contournement du frontend	Toutes les règles sont dans le backend (URL + @PreAuthorize + service)
Injection / données malformées	Bean Validation, enums typés, requêtes JPA paramétrées
Fuite d'informations techniques	GlobalExceptionHandler : messages maîtrisés, pas de stack trace
Requêtes cross-origin	CORS limité à http://localhost:5173
Énumération des comptes	Message identique pour « email inconnu » et « mauvais mot de passe » (401)

10.2 Limites et améliorations possibles

- **Stockage du JWT** : le localStorage est accessible au JavaScript, donc exposé en cas de faille XSS. En production, il serait préférable de placer le refresh token dans un cookie HttpOnly; Secure; SameSite=Strict et de garder l'access token en mémoire.
- **HTTPS obligatoire** en production : sans TLS, un JWT intercepté est directement réutilisable.
- **Secret JWT** : il est dans application.properties pour faciliter la correction ; en production il doit venir d'une variable d'environnement ou d'un coffre-fort (Vault).
- **Limitation du nombre de tentatives** de connexion (rate limiting / verrouillage temporaire) contre la force brute.
- **403 ou 404** sur un incident non autorisé : renvoyer 404 masquerait l'existence de la ressource. Le 403 a été choisi pour coller au sujet (« accès interdit »).
- **Liste noire des access tokens** à la déconnexion (par exemple avec Redis) pour une révocation immédiate.
- **Journalisation et audit** des actions sensibles (désactivation, affectation) et supervision.

11. Conclusion

Ce projet a permis de construire une application complète et sécurisée de gestion d'incidents en appliquant les notions de l'atelier JWT : authentification **stateless** par JWT signé, autorisation par **rôles** sur deux niveaux (URL et méthodes), **contrôle du propriétaire** des données côté serveur, validation systématique des entrées, gestion centralisée des erreurs, ainsi que des mécanismes avancés comme le **refresh token** avec rotation et l'historique des incidents.

Côté React, les intercepteurs Axios automatisent l'envoi du JWT et la gestion de l'expiration, tandis que ProtectedRoute et RoleRoute améliorent l'expérience utilisateur sans jamais remplacer les contrôles du backend.

La démarche de test (9 tests d'intégration, 53 requêtes et 94 assertions Postman/Newman, tests curl et scénario fonctionnel complet) confirme que toutes les exigences et règles métier du sujet sont respectées, en particulier les 9 tests de sécurité obligatoires.

Le principe essentiel retenu est que **la sécurité se décide toujours côté serveur** : un JWT valide prouve une identité, mais chaque accès aux données doit encore être autorisé en fonction du rôle, de la propriété et de l'état de la ressource.

Annexe : récapitulatif des endpoints

Tableau 17 : Endpoints de l'API

Méthode	URL	Accès	Description
POST	/auth/register	Public	Inscription (rôle EMPLOYEE)
POST	/auth/login	Public	Connexion → token, refreshToken, role, fullName
POST	/auth/refresh	Public (refresh token)	Nouvel access token (rotation)
POST	/auth/logout	Public (refresh token)	Révocation du refresh token
GET	/api/profile	Authentifié	Mon profil
PUT	/api/profile	Authentifié	Modifier nom et email
PUT	/api/profile/password	Authentifié	Changer le mot de passe
GET	/api/dashboard	Authentifié	Statistiques selon le rôle
POST	/api/incidents	EMPLOYEE	Déclarer un incident
GET	/api/incidents/my	EMPLOYEE	Mes incidents
GET	/api/incidents/{id}	Propriétaire / affecté / ADMIN	Détail + historique + notes
GET	/api/incidents/assigned	TECHNICIAN	Incidents affectés
PUT	/api/incidents/{id}/status	TECHNICIAN (affecté) ou ADMIN	Changer le statut
PUT	/api/incidents/{id}/assign/{techId}	ADMIN	Affecter à un technicien
POST	/api/incidents/{id}/comments	TECHNICIAN (affecté) ou ADMIN	Ajouter une note
GET	/api/incidents?status=	ADMIN	Tous les incidents (filtre)
GET	/api/admin/users	ADMIN	Liste des utilisateurs
GET	/api/admin/technicians	ADMIN	Techniciens actifs
PUT	/api/admin/users/{id}/enable	ADMIN	Activer un compte
PUT	/api/admin/users/{id}/disable	ADMIN	Désactiver un compte
GET	/api/admin/statistics	ADMIN	Statistiques globales

Livrables

Tableau 18 : Livrables du projet

Livrable	Emplacement
Projet Spring Boot	backend/
Projet React	frontend/
README	README.md
Rapport	rapport/Hicham_OUAOUCHE_JWT_Spring_Security_React.pdf
Diagramme des entités	rapport/diagrammes/entites.png (figure du chapitre 4)
Diagramme d'architecture	rapport/diagrammes/architecture.png (chapitre 3) et flux_jwt.png
Captures des tests Postman	rapport/captures/postman/ (application Postman) + captures Newman N1 à N5 (section 8.6 et 8.7)
Captures de l'application	rapport/captures/ (A*, B*, C*, D*, E*)
Démonstration	Scénario du chapitre 9, reproductible avec les comptes de test

Flux d'authentification



Figure 75 : *Diagramme de séquence : connexion, requête protégée, expiration et refresh*